

UNIVERSIDAD NACIONAL DE MISIONES
Facultad de Ingeniería

Carrera: Ingeniería Electrónica

Asignatura: Proyecto y Diseño Electrónico ET546

**Diseño y construcción de una computadora de 8 bits
basada en el microprocesador CDP1802**

Desarrollado por

Alegre Chamorro, Felipe Santiago
felpasach@gmail.com

Boher, Atilio
atilioboher@gmail.com

Vera, Uriel Adrián
uriel.vera.tec@gmail.com

Oberá, Misiones

Noviembre de 2022

Diseño y construcción de una computadora de 8 bits basada en el microprocesador CDP1802

Desarrollado por

Alegre Chamorro, Felipe Santiago
felpasach@gmail.com

Boher, Atilio
atilioboher@gmail.com

Vera, Uriel Adrián
uriel.vera.tec@gmail.com

Proyecto y Diseño Electrónico ET546

Aprobado por

Ing. Ricardo A. Korpys
Profesor tutor

Mgter. Ing. Guillermo Alfredo Fernández
Profesor lector

Dr. Ing. Javier Ernesto Kolodziej
Profesor lector

Noviembre de 2022

Resumen

Diseño y construcción de una computadora de 8 bits basada en el microprocesador CDP1802

Desarrollado por

Alegre Chamorro, Felipe Santiago

Boher, Atilio

Vera, Uriel Adrián

Universidad Nacional de Misiones

Facultad de Ingeniería

Profesor tutor: Ing. Ricardo A. Korpys

Noviembre de 2022

El presente trabajo tiene como objetivo la proyección y el diseño de una computadora basada en el microprocesador CDP1802, presentando la fundamentación teórica y la serie de pasos seguidos para su implementación. En principio, se presenta una descripción breve del microprocesador, los principios de funcionamiento y circuitos previos basados en la misma tecnología. Luego se exponen los circuitos simples donde se implementan programas en lenguaje ensamblador pensados para lograr un entendimiento de las instrucciones del microprocesador y la forma en la que este interactúa con las memorias. En tercer lugar, se exponen los esquemas, las placas de circuito impreso y programas simples que complementan los conceptos previos. Seguidamente, se presenta el desarrollo de sistemas más complejos con mayor disponibilidad de memoria e interacción con esta. Se han probado programas complejos desarrollados por terceros que ponen en evidencia la capacidad de procesamiento del sistema y los límites alcanzables por el mismo. Finalmente, se presenta el diseño final que es una evolución de los sistemas anteriores, con mayor funcionalidad y aplicabilidad en diferentes entornos, como en la lectura o escritura de datos.

Palabras clave: *microprocesador, CDP1802, computadora, 8 bits, sistema operativo.*

Acerca de Proyecto y Diseño Electrónico

El Proyecto Electrónico es un curso anual bajo la modalidad de trabajo individual o grupal supervisado, con el propósito de aplicar estrategias de diseño y análisis a un problema de temática abierta de la ingeniería electrónica.

Abstract

Design and construction of an 8-bit computer based on the CDP1802 microprocessor

Translated from the original Spanish title: “Diseño y construcción de una computadora de 8 bits basada en el microprocesador CDP1802”

by

Alegre Chamorro, Felipe Santiago

Boher, Atilio

Vera, Uriel Adrián

National University of Misiones

Engineering Faculty

Tutor: Ing. Ricardo A. Korpys

November 2022

The objective of this project is the development and design of a computer based on the CDP1802 microprocessor, and the theoretical foundation and steps followed to achieve it are presented. In the first part of the report, a quick description of the microprocessor, its operating principles and previous circuits based on the same technology are presented. Simple circuits are then developed with their corresponding assembly language programs designed to gain an understanding of the microprocessor instructions and the way in which the microprocessor interacts with the memories. At this point, schematic development processes, PCB boards and simple programs are presented. Then, the development of more complex systems with more available memory is presented. Complex programs created by third parties are tested, revealing the real capacity and limitations of the system developed. Finally, a final design is presented which exploits the capabilities exposed by previous developments and allows greater interaction with external stimuli, such as reading or writing data.

Keywords: *microprocessor, CDP1802, computer, 8-bit, operating system.*

About Proyecto y Diseño Electrónico (“Electronic Project”)

The “Electronic Project” is a one-year course of supervised individual or group work, with the purpose of applying design and analysis strategies to an open-topic problem in electronic engineering.

Índice general

Índice general	ix
Índice de figuras	xii
Índice de fotografías	xv
Índice de tablas	xv
1 Introducción	1
1.1. Objetivos	1
2 Introducción al microprocesador CDP1802	3
2.1. Microprocesador CDP1802	3
2.2. Computadoras basadas en el CDP1802	5
2.2.1. COSMAC ELF	5
2.2.2. COSMAC ELF II	6
2.2.3. Membership Card	7
2.3. CDP1802 y la arquitectura COSMAC	9
2.3.1. Características específicas	9
2.3.2. Organización del sistema	10
2.3.3. Notación de la arquitectura COSMAC	13
3 Programas utilizados	17
3.1. Entorno de programación - Visual Studio Code	17
3.2. Revisión de modificaciones con Git y Visual Studio Code	18
3.3. Ensamblador A18	21
3.4. Emulador Emma02	23
3.5. Terminal Tera Term	27
3.6. Multi Commander	28
3.7. Programador universal XGecu TL866-3G	29
3.8. KiCad EDA	31

4	Pruebas en el COSMAC ELF II	33
5	Sistema mínimo	35
5.1.	Lista de componentes	41
5.2.	Programas del sistema mínimo	42
5.2.1.	Programa 1: Parpadeo LED Q	42
5.2.2.	Programa 2: LED Q se enciende al presionar pulsador	43
5.2.3.	Programa 3: LED Q parpadea al presionar pulsador	44
5.2.4.	Programa 4: Transmisión serial de caracteres	45
5.3.	Simulaciones y resultados experimentales	48
5.4.	Programador	49
6	Sistema medio	53
6.1.	Lista de componentes	58
6.2.	Programas del sistema medio	59
6.2.1.	Intérprete Tiny Basic	59
6.2.2.	Sistema operativo ElfOS sin disco	62
6.3.	Simulación Emma02	64
6.3.1.	Intérprete Tiny Basic	64
6.3.2.	Sistema operativo ElfOS sin disco	64
6.4.	Resultados experimentales	65
6.4.1.	Intérprete Tiny Basic	65
6.4.2.	Sistema operativo ElfOS sin disco	65
7	Sistema Final	67
7.1.	Esquemáticos	67
7.1.1.	Fuente de alimentación	69
7.1.2.	Mapa de memoria	71
7.1.3.	Memorias seriales	73
7.1.4.	Convertor analógico digital	75
7.1.5.	Microprocesador	77
7.1.6.	Decodificador	79
7.1.7.	Comunicación UART	81
7.1.8.	Comunicación SPI	83
7.1.9.	Alimentación de compuertas	88
7.1.10.	Interfaz I/O	90
7.1.11.	BUS I2C	93
7.2.	Lista de componentes	95
7.3.	Placa PCB	97
8	Conclusiones	103

9	Trabajos a futuro	105
9.1.	Construcción de sistema final	105
9.2.	Programas de prueba	105
9.2.1.	Coordinación de interfaz entrada-salida	105
9.2.2.	Comunicación I2C	105
9.2.3.	Conversión Analógico-Digital	106
9.2.4.	Rutina de cambio en el mapa de memoria	106
9.2.5.	Escritura de memoria SDcard	106
9.2.6.	Copia de memoria	106
9.3.	Rediseño de circuito	106
9.3.1.	Rediseño de placa PCB	106
9.3.2.	Conversión Digital-Analógico	107
9.3.3.	Separación galvánica de entradas analógicas	107
9.3.4.	Traslado del circuito SPI del mapa de memoria principal al mapa de memoria de entrada y salida del CDP1802	107
9.4.	Desarrollo de controlador lógico programable	107
9.5.	Topología con dos microprocesadores	107
10	Lineamientos para el desarrollo del sistema operativo	109
	Bibliografía	111
A	Protocolos de Comunicación Serial	115
A.1.	UART	115
A.2.	I ² C	116
A.3.	SPI	121
B	Formato Intel (Archivo .hex)	123
C	Programas escritos para el sistema mínimo	125
C.1.	Programa 1: Parpadeo LED Q	125
C.2.	Programa 2: LED Q se enciende al presionar pulsador	127
C.3.	Programa 3: LED Q parpadea al presionar pulsador	128
C.4.	Programa 4: Transmisión serial de caracteres	129

Índice de figuras

2.1.	CDP1802 en encapsulado DIP40	4
2.2.	Distribución de pines del CDP1802	4
2.3.	Fotografía del COSMAC ELF de la revista Popular Electronics	5
2.4.	Fotografía obtenida en línea del COSMAC ELF II	6
2.5.	Fotografía obtenida en línea del Membership Card	7
2.6.	Circuitos impresos Membership Card	8
	(a). PCB Membership Card	8
	(b). PCB Membership Card Panel Frontal	8
2.7.	Diagrama de bloques de un sistema computacional típico usando el microprocesador CDP1802	11
2.8.	Estructura interna del microprocesador CDP1802	14
2.9.	Formato de instrucción	15
3.1.	Extensiones utilizadas en Visual Studio Code	17
	(a). Extensión para visualización en Assembler	17
	(b). Extensión para visualización de formato Intel	17
3.2.	Extensión para resaltar sintaxis de código en assembler	18
3.3.	Extensión para resaltar la composición de archivos en formato Intel	18
3.4.	Configuración inicial de Git	19
	(a). Abrir Git	19
	(b). Comandos para la configuración inicial	19
3.5.	Configuración de Git relacionada a difftool	20
3.6.	Comando para comparar dos archivos con Git	20
3.7.	Ejemplo de comparación	20
3.8.	Apertura de ventana de comandos en directorio	21
3.9.	Resultado del ensamblado en ventana de comandos	22
3.10.	Ensamblado con script Batch	22
	(a). Instrucciones Batch para el ensamblado con doble click	22
	(b). Uso del archivo .bat	22
3.11.	Computadoras disponibles que utilizan el CDP1802	23
3.12.	Seleccionar programa	23
3.13.	Seleccionar programa y configuración	24
3.14.	Seleccionar programa en Emma02	25

3.15. Direct Assembler	25
3.16. Registrar instrucciones ejecutadas y visualización de Registros internos	26
3.17. Captura de instrucciones ejecutadas	26
3.18. Tamaño y tipo de letra de la terminal	27
3.19. Cargado de programas a través de la terminal	28
(a). Opción de envío de archivo	28
(b). Configuración de delay	28
3.20. Herramienta para cambio de carácter de salto de línea en Multi Commander	29
3.21. Programador universal XGecu TL866-3G	30
3.22. Entorno gráfico del programa del programador universal XGecu TL866-3G	30
3.23. Entorno gráfico del editor de esquemas del software KiCad	31
3.24. Entorno gráfico del editor de circuito impreso del software KiCad	32
5.1. Diagrama de bloques del sistema mínimo	35
5.2. Mapa de memoria Sistema Mínimo	36
5.3. Esquema de sistema mínimo	38
5.4. Placa PCB diseñada para sistema mínimo	39
(a). Pista diseñada para sistema mínimo	39
(b). Máscara de componentes sistema mínimo	39
5.5. Programa 1	42
5.6. Programa 2	43
5.7. Programa 3	44
5.8. Programa 4	46
(a). Rutina principal	46
(b). Rutina de transmisión	46
5.9. Configuración utilizada en el simulador Emma02	48
5.10. Resultados de la ejecución del programa 4	49
(a). Resultado obtenido en Emma02	49
(b). Resultado experimental obtenido con el sistema mínimo	49
5.11. Esquema de programador	50
5.12. Placa PCB diseñada para el programador de memoria	51
(a). Pista diseñada para Programador	51
(b). Máscara de componentes de Programador	51
6.1. Diagrama de bloques del sistema medio	54
6.2. Mapa de memoria del sistema medio	55
6.3. Esquemático del sistema medio	56
6.4. Menú del sistema operativo ElfOS sin disco	63
6.5. Configuración utilizada en el simulador Emma02 para ejecutar el intérprete Tiny Basic y el ElfOS sin disco	64
7.1. Esquema general de sistema final	68

7.2.	Esquema de fuente de alimentación de sistema final	70
7.3.	Esquema de Mapa de memoria de sistema final	72
7.4.	Esquema de memorias seriales de sistema final	74
7.5.	Esquema de conversor analógico digital de sistema final	76
7.6.	Esquema de conexión de microprocesador de sistema final	78
7.7.	Esquema de decodificador de memoria de sistema final	80
7.8.	Esquema de comunicación UART de sistema final	82
7.10.	Proceso de guardado de dato a transmitir	84
7.11.	Proceso de transmisión SPI	84
7.12.	Proceso de lectura	84
7.9.	Esquema de comunicación SPI de sistema final	85
7.13.	Esquema de alimentación de compuertas de sistema final	89
7.14.	Esquema de interfaz de entrada/salida de sistema final	92
7.15.	Esquema de bus I2C de sistema final	94
7.16.	Pistas de PCB sistema final parte frontal	98
7.17.	Pistas de PCB sistema final parte posterior	99
7.18.	Máscara de componentes de PCB de sistema final	100
7.19.	Visión en tres dimensiones de sistema final	101
10.1.	Sistema de directorios del ElfOS	110
A.1.	Interconexión de interfaz UART	115
A.2.	Paquete de datos UART	115
A.3.	Interfaz I ² C	117
	(a). Interconexión de interfaz I ² C	117
	(b). Líneas SDA y SCL	117
A.4.	Trama y secuencia de inicio y parada I ² C	118
	(a). Ejemplo de trama I ² C: Escritura en el registro de un Esclavo (M: Maestro; E: Esclavo)	118
	(b). Secuencia de inicio y parada I ² C	118
A.5.	Trama I ² C en detalle	120
	(a). Reconocimiento de bits	120
	(b). Secuencias de START y STOP	120
	(c). Paquete de Dirección	120
	(d). Paquete de Datos	120
	(e). Combinación de Paquetes de Dirección y Datos	120
A.6.	Interconexión interfaz SPI	122
B.1.	ejemplo de archivo .hex	123

Índice de fotografías

4.1. COSMAC ELF II, propiedad de la facultad de ingeniería	33
5.1. Sistema mínimo construido	40
6.1. Sistema medio construido	57

Índice de tablas

2.1. Códigos de estado	12
2.2. Modos del CDP1802	13
2.3. Numeración decimal, binaria y hexadecimal	15
5.1. Lista de componentes del sistema mínimo	41
6.1. Lista de componentes del sistema medio	58
7.1. Instrucciones INP utilizables con el 82C55	90
7.2. Instrucciones OUT utilizables con el 82C55	91
7.3. Lista de componentes del sistema final	95

Índice de listados

5.1.	Rutina de retardo de 1 bit-time	47
6.1.	Archivo 1802reg.asm con definición de etiquetas para los registros de trabajo	60
6.2.	Segmento del código fuente tb0_test.asm con modificación en línea 25 y 28	61
6.3.	Segmento del código fuente tb0_test.asm, modificación de posición de memoria RAM en el mapa de memoria	62
6.4.	Segmento del código fuente tb0_test.asm, definición del baudrate utilizado con cristal de 4 MHz	62
7.1.	Rutina propuesta para la transmisión utilizando el circuito de comunicación SPI	86
C.1.	Programa 1	126
C.2.	Programa 2	127
C.3.	Programa 3	128
C.4.	Programa 4	129

Introducción

El siguiente trabajo se desarrolla en el marco de la materia “Proyecto y diseño electrónico” de la carrera de ingeniería electrónica de la Universidad Nacional de Misiones, en la cual se busca proyectar y diseñar circuitos electrónicos que involucren las competencias desarrolladas durante el cursado de la carrera. El objetivo principal del presente documento es contemplar todos los conceptos y conocimientos utilizados en el proyecto, diseño e implementación de una microcomputadora de uso general basada en un microprocesador específico con diferentes funcionalidades como ser conversor analógico-digital y comunicación UART, entre otras. Teniendo en cuenta la búsqueda de la integración de los conceptos adquiridos en la carrera, el desarrollo de una computadora de uso general toma una relevancia considerable ya que involucra ampliamente los conceptos estudiados, como por ejemplo: el diseño y construcción de fuentes de alimentación, circuitos analógicos, circuitos digitales, acondicionamiento de señal, programación en lenguaje ensamblador, entre otros.

Motivados por los avances obtenidos gracias a los primeros microprocesadores y con la disponibilidad de las herramientas tecnológicas de la actualidad, se plantea el diseño de una computadora de uso general basada en uno de los primeros microprocesadores: **El CDP1802, primer microprocesador de la tecnología CMOS**. Se busca comprender su principio de funcionamiento, arquitectura e instrucciones de operación para poder integrar en un circuito las diferentes funcionalidades requeridas y establecer un mecanismo por el cual incorporar tecnologías más modernas como ser las tarjetas SD o FRAM.

Durante el desarrollo del trabajo se presentarán principios de funcionamiento, bibliografía involucrada, diseño de esquemas circuitales, construcción de placas de circuito impreso (PCB), diseño de programas y pruebas de funcionamiento asociadas a los circuitos confeccionados. Para que el análisis y comprensión de las topologías planteadas sean simples, el desarrollo del trabajo incrementa progresivamente la complejidad de los circuitos implementados.

1.1. Objetivos

Objetivo general La idea básica del proyecto es diseñar una microcomputadora basada en el microprocesador CDP1802 que esté dotada de capacidad de memoria no volátil (ROM) y memorias volátiles (RAM) para poder ejecutar funciones específicas y pueda conectarse mediante un puerto a un terminal (notebook o computadora de escritorio) y utilizar la pantalla y el teclado de la misma como interfaz de

entrada y salida, para poder así usar el terminal como método principal para la comunicación entre el usuario y la computadora.

- Diseñar, proyectar y seleccionar los componentes necesarios para la microcomputadora, entre los más importantes, el microprocesador, la memoria de programa, memoria de datos, buses de comunicación, integrados, etc. Para ello debe realizarse el estudio adecuado de mercado y de tecnologías ya existentes para implementar el sistema.
- Realizar un diagrama esquemático de un sistema mínimo, de preferencia de manera manual (lápiz y papel), para así obtener un acercamiento general de los elementos mínimos e indispensables para lograr el funcionamiento de la microcomputadora e introducir a los alumnos en el diseño de la misma.
- Proyectar e implementar como primera aproximación el sistema mínimo de microcomputadora donde solamente se utilice el microprocesador, una memoria de programa y los dispositivos necesarios para el funcionamiento básico (resonadores, integrados, etc.). Para ello se debe realizar un estudio de la bibliografía correspondiente sobre la arquitectura del microprocesador y de las instrucciones en lenguaje ensamblador para el funcionamiento del mismo.
- Diseñar y proyectar una fuente de alimentación lineal para energizar la microcomputadora, cuyo suministro sea la red de alimentación (220 V/50 Hz) y analizar la necesidad de utilizar un suministro alternativo (ej. baterías).
- Partiendo del sistema mínimo, proyectar una microcomputadora que esté dotada de memorias tanto ROM como RAM para su funcionamiento, un bus de puerto serie (Protocolo UART), un puerto paralelo de 8 bits, un conversor analógico-digital de 8 bits, debe contener una memoria SD-CARD para la grabación de algún programa que se requiera y analizar el uso de memorias F-RAM.

Introducción al microprocesador CDP1802

2.1. Microprocesador CDP1802

El microprocesador utilizado es el CDP1802 [1], el mismo es un chip de gran escala de integración LSI (del inglés: *Large Scale Integration*) de tecnología CMOS fabricado originalmente por la empresa RCA en 1974, quien también le dio el nombre COSMAC [2] (del inglés: Complementary Symmetry Monolithic Array Computer) para referirse a su arquitectura, nombre que también se utilizó para referirse a varias computadoras que fueron diseñadas durante los años en base al mismo, algunas de las cuales se mencionan en este informe. El CDP1802 fue el primer microprocesador fabricado con tecnología CMOS [3] y posee una arquitectura y características que difieren significativamente de la mayoría de microprocesadores de 8 bits de la misma época. Además, posee un consumo de potencia ultra bajo, y no posee una velocidad de reloj mínima, permitiéndole funcionar con una señal de reloj reducida hasta 0 Hz y luego resumir su funcionamiento sin ningún efecto negativo.

A diferencia de la mayoría de los microprocesadores, que tienen principalmente un registro de contador de programa, un registro de índice y un acumulador, el CDP1802 posee 16 registros de 16 bits de uso general, cualquiera de los cuales puede ser designado como contador de programa o registro de índice, y también posee un acumulador de 8 bits. Gran parte del set de instrucciones se encuentra dedicado a varias operaciones utilizando estos registros de uso general.

El CDP1802 también posee internamente un controlador de acceso directo de memoria o DMA (del inglés: *Direct Memory Access*), permitiendo que la memoria sea leída o modificada por dispositivos externos sin necesidad de que el CDP1802 detenga la operación de su rutina principal.

Se ha optado por este microprocesador porque, a diferencia de la mayoría de los microprocesadores del mismo periodo, el CDP1802 todavía se fabrica en la actualidad, esto es debido a su estabilidad, resistencia a la radiación (como pulsos electromagnéticos), su ultra bajo consumo de energía y su habilidad para funcionar con señales de reloj de muy baja frecuencia, para disminuir el consumo de energía, cuando la velocidad de operación no es un requerimiento.

Otro aspecto del CDP1802 que lo hizo popular entre los diseñadores aeroespaciales es que estaba disponible en una versión resistente a la radiación, lo que lo hacía ideal para el duro entorno del espacio. Se utilizaron seis 1802 como cerebro de la sonda Galileo [4], que se lanzó en 1989 y orbitó Júpiter entre 1995 y 2003. El 1802 también ha sido utilizado en el Telescopio Espacial Hubble y además, en dispositivos comerciales como cajas registradoras, teléfonos públicos y equipamientos de audio y video.

En la figura 2.1 y 2.2 se puede apreciar una fotografía del encapsulado del CDP1802 y la distribución de pines, respectivamente.



Figura 2.1: CDP1802 en encapsulado DIP40

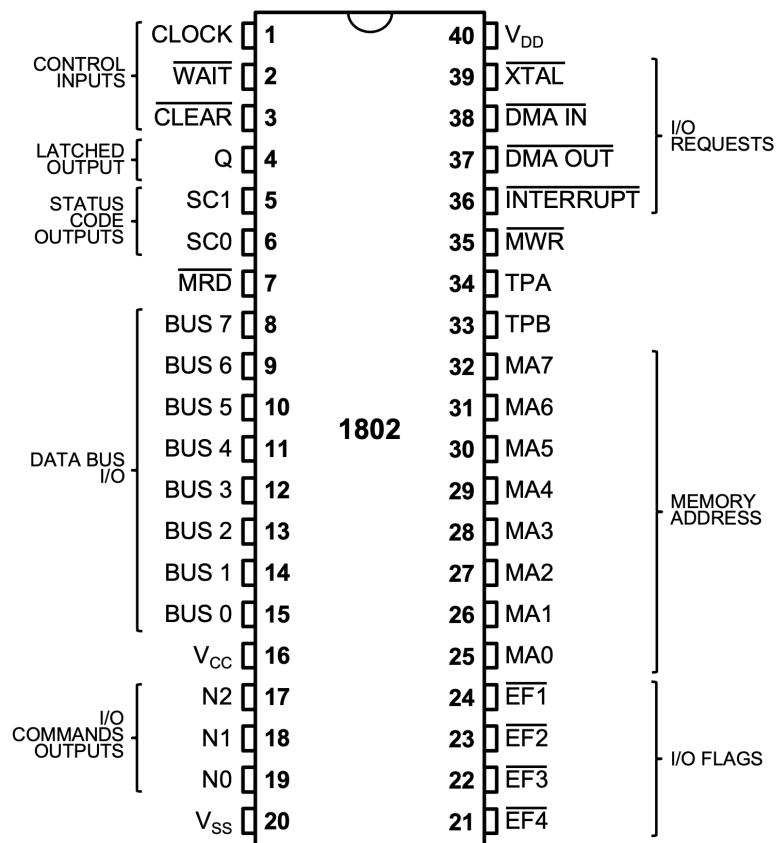


Figura 2.2: Distribución de pines del CDP1802

2.2. Computadoras basadas en el CDP1802

2.2.1. COSMAC ELF

En la edición de agosto de 1976 de la revista Popular Electronics [5], Joseph Weisbecker detalló instrucciones para construir el COSMAC ELF basado en el CDP1802. El ELF era una computadora simplificada que permitía a los usuarios ingresar programas accionando interruptores de palanca y costaba alrededor de USD\$ 80 en partes.

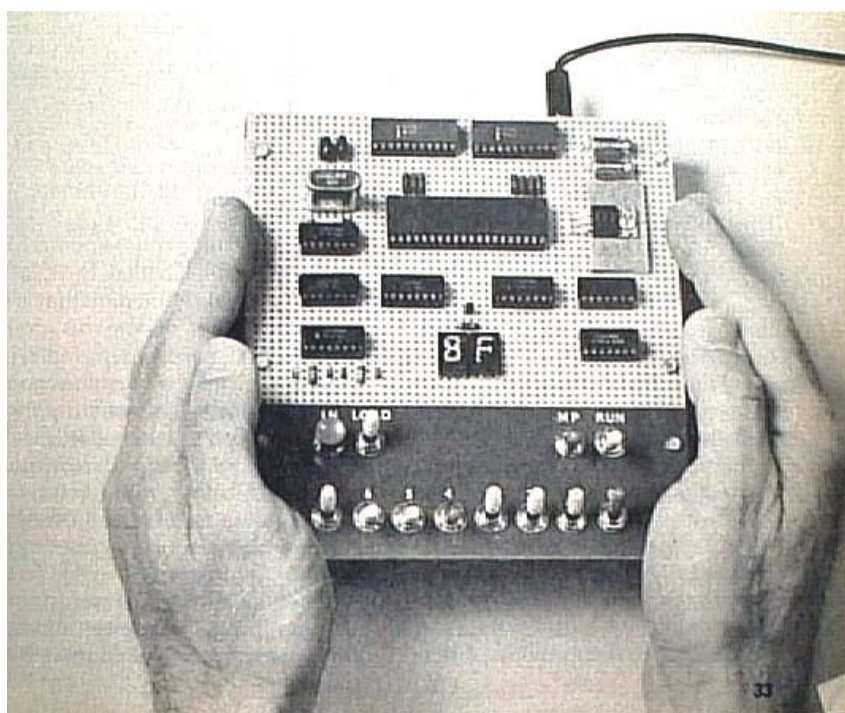


Figura 2.3: Fotografía del COSMAC ELF de la revista Popular Electronics

El diseño de la computadora COSMAC ELF es simple, lo cual es así principalmente porque el CDP1802 posee un controlador DMA integrado como así también un sistema de control de dispositivos de entrada y salida, lo cual hace que no sean necesarios muchos circuitos integrados externos que de otro modo serían imprescindibles para realizar las tareas requeridas.

Una de las razones de la popularidad del CDP1802 entre los aficionados fue que su diseño facilitó la expansión de sistemas que lo utilizan y la interfaz con el mundo exterior. Los microprocesadores típicos requieren hardware de soporte y múltiples instrucciones de código máquina para utilizar periféricos de entrada y salida. Además, cuatro pines en el 1802 actúan como entradas digitales directas y con solo una instrucción de código máquina, un programa puede verificar el estado de uno de estos pines y actuar sobre el resultado. Otro pin, denominado Q, funciona como una salida de un bit que también se puede controlar con una sola instrucción de código máquina.

Una salida de un bit puede no parecer mucho, pero permite comunicación serial y el control de todo tipo de dispositivos. Tal vez lo más interesante, como señaló Weisbecker en sus artículos de Popular Electronics, es que los bucles de tiempo cuidadosamente elaborados podrían controlar con precisión la frecuencia a la que el voltaje del bit de salida cambiaba de un estado a otro, produciendo una onda cuadrada a frecuencias audibles. Esto significaba que el bit de salida podía conectarse a un altavoz y surgirían tonos musicales. Alan Turing usó una técnica similar para crear las primeras notas musicales generadas por computadora en 1951 [6].

2.2.2. COSMAC ELF II

El COSMAC ELF II es una versión mejorada de la primera COSMAC ELF, la cual incluye un teclado en lugar de interruptores para ingresar datos, además de ranuras donde pueden agregarse diferentes placas de expansión, por ejemplo, una placa de memoria RAM.



Figura 2.4: Fotografía obtenida en línea del COSMAC ELF II

2.2.3. Membership Card

La Membership Card [7] es una reproducción de la computadora ELF original de Popular Electronics, empaquetada para caber en una lata Altoids® de tamaño de bolsillo. Está completamente construida con piezas y tecnología de 1980, utilizando solo componentes comunes de tecnología through hole (agujeros pasantes) y de bajo costo (sin circuitos integrados personalizados, ni componentes de montaje superficial). Para usarla no es necesario una PC moderna ni megabytes de software propietario.

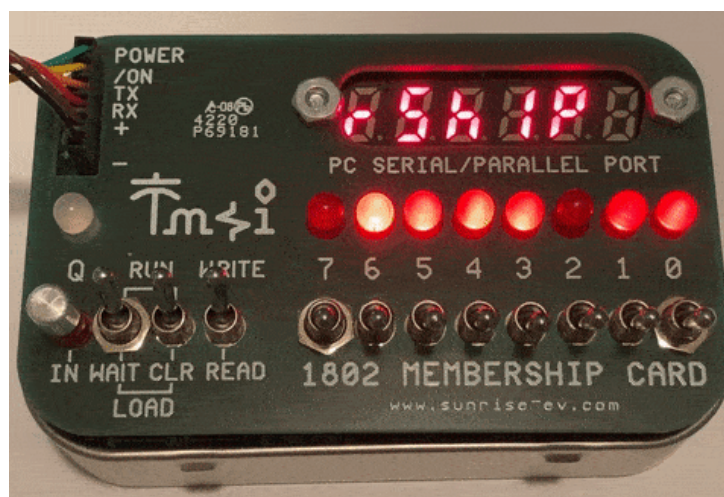
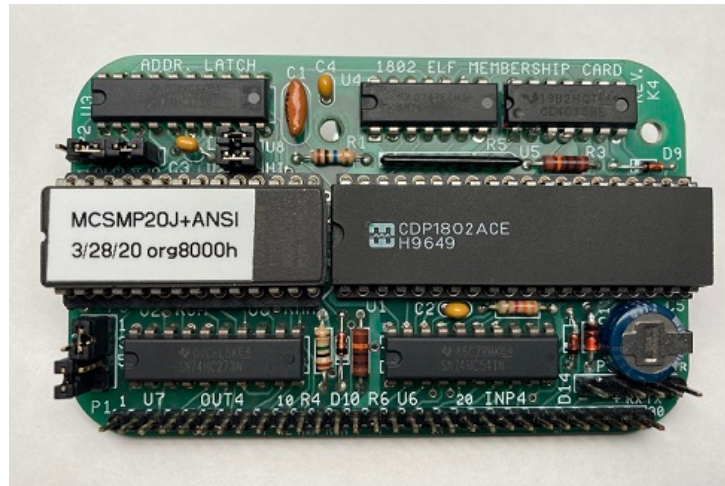


Figura 2.5: Fotografía obtenida en línea del Membership Card

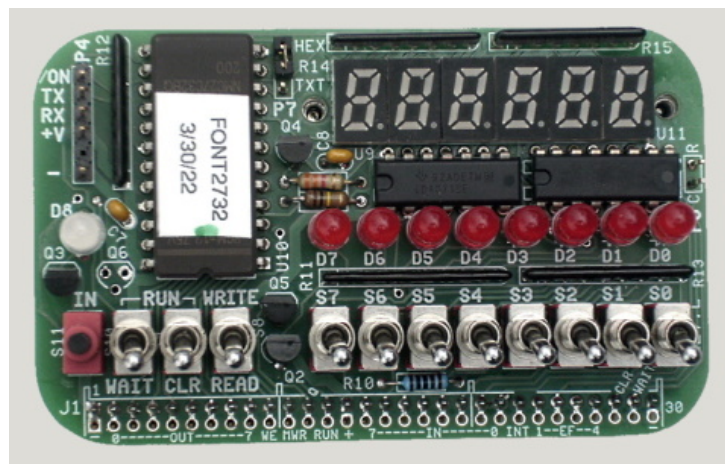
La computadora está compuesta por dos placas de circuito impreso, cada una del tamaño de una tarjeta de crédito. Una es la propia Membership Card (figura 2.6a), que tiene el microprocesador 1802, hasta 64k bytes de memoria, 22 bits de E/S, circuitos de reloj, reinicio, fuente de energía y además de un super-capacitor para mantener el contenido de la memoria sin energía. Se puede usar solo como un microcontrolador para proyectos como los microcontroladores Parallax BASIC Stamps o Arduino.

La segunda placa es el panel frontal (figura 2.6b). Proporciona los interruptores y los LED para un panel de control de “luces intermitentes”, al igual que las computadoras clásicas de antaño. Puede leer y escribir en la memoria y en los puertos de E/S manualmente, sin la ayuda de software o dispositivos externos. Los datos se muestran tanto en bits binarios individuales como en dos dígitos hexadecimales (o incluso mensajes ASCII de 6 caracteres con 4 dígitos adicionales opcionales y software apropiado).

Un adaptador serial/USB se conecta directamente al conector de 6 pines para proporcionar alimentación y E/S serial, que permite que una terminal externa o PC cargue, guarde y ejecute programas.



(a) PCB Membership Card



(b) PCB Membership Card Panel Frontal

Figura 2.6: Circuitos impresos Membership Card

2.3. CDP1802 y la arquitectura COSMAC

El RCA-CDP1802 es una unidad central de procesamiento (CPU) orientada a bytes que emplea la arquitectura COSMAC (Complementary Symmetry Monolithic Array Computer) y utiliza tecnología MOS (metal-oxide semiconductor) de simetría complementaria (CMOS). El microprocesador CDP1802 incluye todos los circuitos necesarios para obtener, interpretar y ejecutar instrucciones que se han almacenado en tipos estándar de memorias. También se proporcionan amplias funciones de control de entrada/salida (E/S) para facilitar el diseño del sistema.

A continuación se describen brevemente algunas características importantes del microprocesador CDP1802. La información fue obtenida del manual de usuario del mismo [8].

2.3.1. Características específicas

Las funciones avanzadas y características operativas del microprocesador RCA CDP1802 incluyen:

- Circuito CMOS estático, sin frecuencia de reloj mínima
- Rango completo de temperatura militar (-55 a +125 °C)
- Alta inmunidad al ruido y amplio rango de voltaje operativo
- Compatibilidad con TTL
- Reloj monofásico; oscilador opcional, en chip, controlado por cristal
- Control simple de reinicio, inicio y pausa
- Organización paralela de 8 bits con bus de datos bidireccional
- Facilidad de carga de programa incorporada
- Cualquier combinación de RAM y ROM estándar a través de una interfaz común
- Direccionamiento directo de memoria hasta 65.536 bytes
- Modo de E/S programable flexible
- Modo de interrupción del programa
- Recurso DMA (Direct Memory Access) en chip
- Cuatro entradas de bandera de E/S comprobables directamente por instrucción de salto
- Puertos de salida programable

- Formato de instrucción de uno a tres bytes con tiempo de ejecución de dos ciclos de máquina para cada instrucción¹
- 91 instrucciones de uso simple
- Compatibilidad de software con el CDP1801
- Matriz de registros generales de 16 X 16 para usar como múltiples contadores de programa, punteros de datos o registros de datos

2.3.2. Organización del sistema

La figura 2.7 ilustra un sistema informático típico que incorpora el microprocesador CDP1802. Las operaciones que se pueden realizar incluyen:

- a) Control de dispositivos de E/S
- b) Transferencia de datos o información de control entre dispositivos de E/S y memoria
- c) Movimiento de datos entre diferentes posiciones de memoria
- d) Interpretación o modificación de bytes guardados en memoria

La memoria puede comprender cualquier combinación de RAM y ROM hasta un máximo de 65 536 bytes. La ROM (memoria de solo lectura) se utiliza para el almacenamiento permanente de programas, tablas y otros tipos de datos fijos. Se requiere RAM (memoria de acceso aleatorio) para los sistemas informáticos de propósito general que requieren cambios frecuentes de programa. También se requiere RAM para el almacenamiento temporal de datos variables. El tipo de memoria y la capacidad de almacenamiento requerida están determinados por la aplicación específica del sistema.

Los datos se transfieren desde o hacia el microprocesador entre los dispositivos de E/S y las memorias por medio de un **bus de datos bidireccional común de ocho bits** (BUS0-BUS7).

Las instrucciones de entrada/salida generan un **código N de tres bits** (N0, N1 y N2). El uso del código N para especificar un dispositivo de E/S permite un control simple y económico de una pequeña cantidad de dispositivos de E/S.

El microprocesador cuenta con **cuatro entradas que actúan como banderas de E/S** ($\overline{EF1}$, $\overline{EF2}$, $\overline{EF3}$ y $\overline{EF4}$). Los dispositivos de E/S pueden controlar estas entradas en cualquier momento para señalar al CDP1802 que se requiere una transferencia de bytes, que se ha producido una condición de error, etc. Estos indicadores también se pueden utilizar como líneas de entrada binaria si se desea. Se pueden probar mediante instrucciones CDP1802 para determinar si están activos o no. El uso de las entradas de bandera debe coordinarse con los programas que las prueban.

También está disponible un **pin de salida (Q)**, cuyo valor es controlado por instrucciones. Esta línea Q, bajo el control del programa, puede activar o señalar dispositivos de E/S. También se puede

¹Excepto las instrucciones de saltos largos (long-branch instructions) y salteo largo (long-skip instructions) que requieren tres ciclos de máquina.

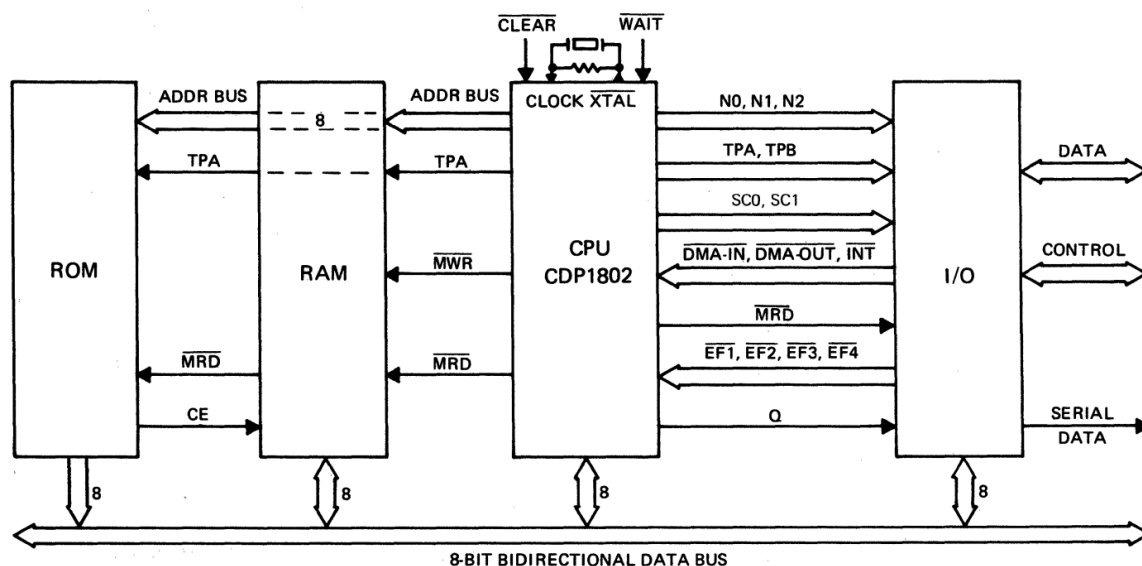


Figura 2.7: Diagrama de bloques de un sistema computacional típico usando el microprocesador CDP1802

usar en conjunto con una de las entradas de bandera ($\overline{EFx}$) para formar una interfaz de comunicación serial.

Los circuitos de E/S pueden activar una **línea de interrupción de programa** ($\overline{INT}$) en cualquier momento para obtener una respuesta inmediata del microprocesador. La interrupción hace que el CDP1802 suspenda su secuencia de programa actual y ejecute una secuencia predeterminada de operaciones diseñadas para responder a la condición de interrupción. Después de atender la interrupción, el CDP1802 reanuda la ejecución del programa interrumpido. Se puede hacer que el CDP1802 ignore la línea de interrupción reiniciando su **flip-flop de habilitación de interrupción** (IE).

Se proporcionan dos líneas de E/S adicionales para tipos especiales de transferencia de bytes entre la memoria y los dispositivos de E/S. Estas líneas se denominan **líneas de acceso directo a memoria** (Direct Memory Access - DMA). La activación de la línea $\overline{DMA-IN}$ hace que un byte de entrada se almacene inmediatamente en una ubicación de memoria sin la intervención del programa que se está ejecutando. La línea $\overline{DMA-OUT}$ hace que un byte se transfiera inmediatamente desde la memoria a los circuitos de salida solicitantes. Se utiliza un registro de puntero de memoria incorporado para indicar la ubicación de memoria para los ciclos de DMA. El programa establece inicialmente este puntero en una ubicación de memoria inicial. Cada transferencia de byte DMA incrementa automáticamente el puntero a la siguiente ubicación de memoria. La activación repetida de una línea DMA puede provocar la transferencia de cualquier número de bytes consecutivos hacia y desde la memoria, independientemente de la ejecución simultánea del programa.

Los circuitos de dispositivos de E/S pueden provocar la transferencia de datos al activar una línea de bandera ($\overline{EFx}$), la línea de interrupción ($\overline{INT}$) o una línea DMA ($\overline{DMA-IN}$ o $\overline{DMA-OUT}$). El programa

debe muestrear las líneas de bandera para determinar cuándo se activan y se utilizan para detectar señales relativamente lentas. La activación de la línea de interrupción provoca una respuesta inmediata, independientemente del programa que se esté ejecutando actualmente, suspendiendo la operación de ese programa y permitiendo una respuesta en tiempo real. El uso de DMA proporciona la respuesta más rápida con la menor perturbación del programa.

Se proporciona un **código de estado de dos bits** (SC0 y SC1) y **dos líneas de temporización** (TPA y TPB) para que los utilicen los circuitos de dispositivos de E/S. Estas cuatro señales permiten la sincronización de circuitos de E/S con ciclos operativos internos del CDP1802. El código de estado indica si el CDP1802 está respondiendo a una solicitud de DMA, respondiendo a una solicitud de interrupción, obteniendo una instrucción o ejecutando una instrucción; esto se encuentra resumido en la tabla 2.1. Los sistemas de memoria y de E/S utilizan las señales de temporización para señalar un nuevo código de estado del procesador, bloquear bits de dirección de memoria (para obtener direccionamiento de hasta 16 bits con solo 8 pines de dirección), tomar datos de memoria del bus y establecer o restablecer los flip-flops del controlador de E/S.

Tabla 2.1: Códigos de estado

State Type	State Code	
	SC1	SC0
S0 (Fetch)	L	L
S1 (Execute)	L	H
S2 (DMA)	H	L
S3 (Interrupt)	H	H

El CDP1802 proporciona ocho líneas de dirección de memoria (MA0-MA7). Estas ocho líneas proporcionan direcciones de memoria de 16 bits en forma de dos bytes de 8 bits sucesivos. El byte de dirección más significativo (de orden superior) aparece primero en las ocho líneas de dirección, seguido del byte de dirección menos significativo (de orden inferior). El número de bits de orden superior necesarios para seleccionar una ubicación de byte de memoria única depende del tamaño de la memoria. Por ejemplo, una memoria de 4096 bytes requeriría una dirección de 12 bits. Esta dirección de 12 bits se obtiene combinando 4 bits del byte de dirección de orden superior con los 8 bits del byte de dirección de orden inferior. Uno de los dos pulsos de temporización del CDP1802 se puede usar para introducir los bits de orden superior requeridos en un latch de dirección (registro paralelo de 8 bits) cuando aparecen en las ocho líneas de dirección.

Cuatro líneas adicionales completan la interfaz del sistema del microprocesador. Una entrada de reloj monofásica (CLOCK) determina la velocidad de funcionamiento. El reloj externo se puede detener y comenzar nuevamente, continuando con la operación del CDP1802 donde se había quedado antes de detenerse el reloj. La construcción del circuito de reloj se simplifica mediante el uso de la entrada $\overline{\text{XTAL}}$. Se conecta un cristal entre $\overline{\text{XTAL}}$ y la entrada de reloj CLOCK, por lo cual no se necesitan componentes activos. La línea de entrada $\overline{\text{CLEAR}}$ inicializa el microprocesador y su liberación inicia la ejecución de las instrucciones. La línea $\overline{\text{WAIT}}$ suspende la operación de la CPU limpiamente. La activación simultánea de $\overline{\text{CLEAR}}$ y $\overline{\text{WAIT}}$ pone a la CPU en un modo de carga de programa. La tabla 2.2 resume estos modos

de operación.

Tabla 2.2: Modos del CDP1802

$\overline{\text{CLEAR}}$	$\overline{\text{WAIT}}$	MODE
L	L	Load
L	H	Reset
H	L	Pause
H	H	Run

2.3.3. Notación de la arquitectura COSMAC

La figura 2.8 ilustra la estructura interna del microprocesador COSMAC CDP1802. Esta arquitectura se basa en una matriz de registros que comprende dieciséis registros tipo **scratch-pad** de 16 bits de propósito general. Cada registro (denominados **R**) se designa mediante un código binario de 4 bits. Aquí se utilizará la **notación hexadecimal (hex)** para referirse a códigos binarios de 4 bits; en la tabla 2.3 se encuentra el equivalente decimal y binario de todos los dígitos hexadecimales. Siguiendo la notación hexadecimal, podemos referirnos a los registros 3 y 12 como **R(3)** y **R(C)**, respectivamente. De esta manera nos referimos a los registros de 16 bits; sin embargo, podemos referirnos a los bytes que lo componen como **R(3).0** y **R(3).1**, que indican el byte menos significativo y más significativo de **R(3)**, respectivamente.

Tres registros de 4 bits denominados **N**, **P** y **X** contienen los códigos binarios de 4 bits (dígitos hexadecimales) que se utilizan para seleccionar registros de 16 bits individualmente. La notación **R(X)**, **R(N)** o **R(P)** se usa para referirse a un registro de 16 bits seleccionado por el código de 4 bits en **X**, **N** o **P**, respectivamente.

De esta manera, la siguiente notación denota que la parte baja de **R(N)** es transferida al registro de 8 bits **D**, que es similar al acumulador encontrado comúnmente en microprocesadores, y el bit **DF** es el Carry de **D**, y el mismo pasa a “1” si el resultado de una operación desborda **D**.

$$R(N).0 \rightarrow D$$

También se puede leer directamente de la memoria utilizando uno de los registros de trabajo de 16 bits para realizar el direccionamiento para la lectura, como se muestra en la siguiente notación.

$$M(R(X)) \rightarrow D$$

El flip-flop interno **Q** se puede configurar o restablecer mediante instrucciones, y también existen instrucciones de salto condicional en base al valor de **Q**. El estado de **Q** también está disponible como salida del microprocesador.

Tabla 2.3: Numeración decimal, binaria y hexadecimal

Decimal	Binario	Hexadecimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

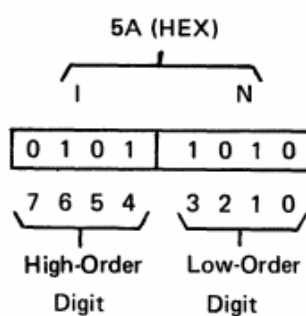


Figura 2.9: Formato de instrucción

se está utilizando actualmente como contador de programa. Las operaciones realizadas por el ciclo de búsqueda (Fetch Cycle) se representan de la siguiente manera.

$$M(R(P)) \rightarrow I, N ; R(P)+1 \rightarrow R(P)$$

Contadores de programa múltiples Un contador de programa es un registro que apunta a la siguiente instrucción a buscar y ejecutar. La arquitectura COSMAC brinda la capacidad única de especificar, en una sola instrucción (instrucción SEP), cualquiera de los 16 registros de trabajo como contador

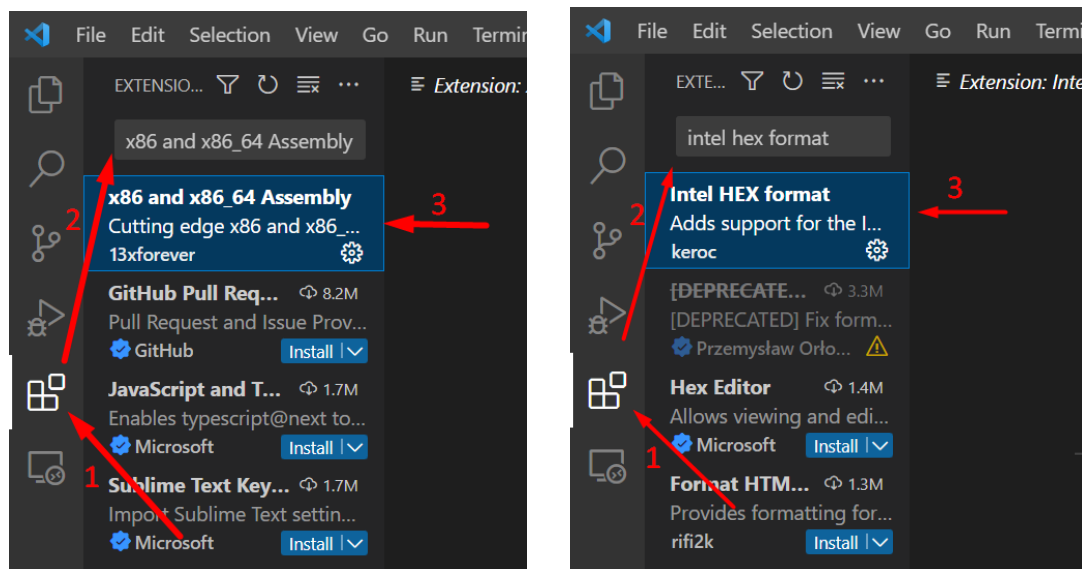
de programa. Esta característica hace posible mantener punteros a varios programas diferentes simultáneamente y transferir el control rápidamente de uno a otro. Un puntero a un programa que atiende una solicitud de interrupción, o la posibilidad de crear subrutinas que pueden operar sin RAM, son solo dos ejemplos de esta función.

Programas utilizados

3.1. Entorno de programación - Visual Studio Code

Existen muchos entornos de programación disponibles, el utilizado para el proyecto fue Visual Studio Code [9]. El mismo fue utilizado porque es gratuito y se encuentra disponible para Windows, Linux y Mac.

Visual Studio Code es un entorno de desarrollo flexible que cuenta con extensiones que pueden ser instaladas para ayudar con la escritura y visualización de código. En la figura 3.1 se encuentran capturas de pantalla del entorno de programación, indicando con flechas numeradas la secuencia para encontrar las dos extensiones utilizadas.



(a) Extensión para visualización en Assembler

(b) Extensión para visualización de formato Intel

Figura 3.1: Extensiones utilizadas en Visual Studio Code

La primera de ellas, “x86 and x86_64 Assembly”, es una extensión para el resaltado de sintaxis. En la figura 3.2 puede apreciarse un programa sencillo que hace parpadear el LED Q en un sistema con el CDP1802, donde lo más importante es la diferencia de color entre las instrucciones y los comentarios, lo cual facilita la lectura y análisis del código.

```

26
27 ; Start code segment
28 INICIO
29     REQ      ; reset Q
30 START
31     LDI 010H ; carga 10 hexadecimal en D
32     PHI R1   ; carga el valor de D a R1.1
33 LOOP
34     DEC R1   ; decrementa R1.1
35     GHI R1   ; carga R1.1 en D
36     BNZ LOOP ; salta si D es 0
37     BQ INICIO ; salta si Q = 1 (para que Q conmute)
38     SEQ      ; set Q
39     BR START ; salto incondicional
40     END
41

```

Figura 3.2: Extensión para resaltar sintaxis de código en assembler

La segunda extensión es “Intel HEX format”, dedicada a resaltar la composición de archivos en formato Intel, lo cual sería difícil de otro modo, o requeriría un programa específico para esta tarea. En la figura 3.3 se encuentra una captura de pantalla de un archivo Intel visualizado con esta extensión. En el anexo B (Formato Intel (Archivo .hex)) se encuentra descrito el formato Intel.

```

1 :0D0000007AF810B121913A0431007B3001F3
2 :00000D01F2
3

```

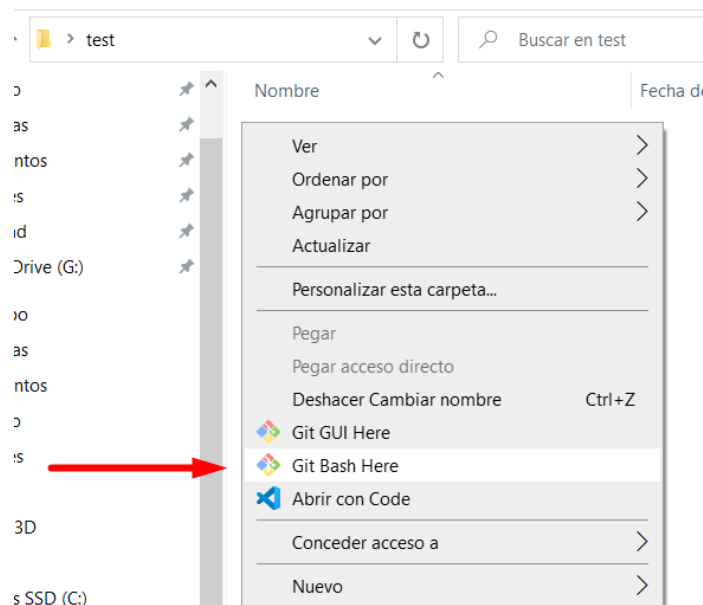
Figura 3.3: Extensión para resaltar la composición de archivos en formato Intel

3.2. Revisión de modificaciones con Git y Visual Studio Code

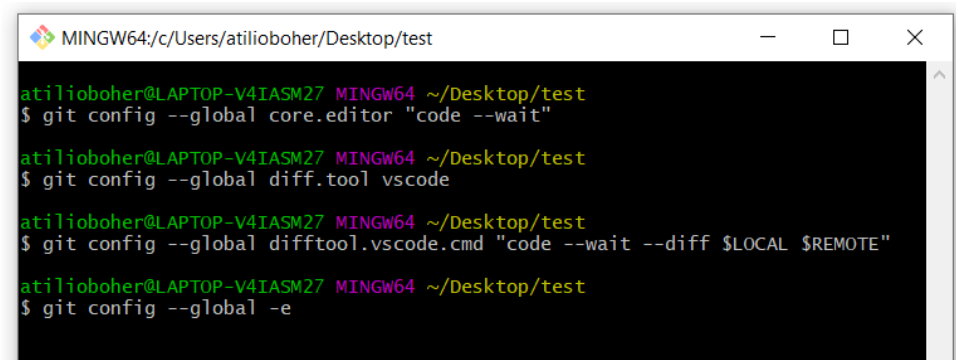
Cuando se tienen programas grandes y se sabe que se han realizado modificaciones, intentar encontrarlas y ver nada más estos cambios puede resultar una tarea engorrosa. Sin embargo, existen herramientas para poder visualizar cambios realizados en un código. Una de estas herramientas es Git, un programa para llevar el control de versión del desarrollo de código. Git es un programa con muchas funciones, pero la única que es de interés en este momento es la revisión de diferencias entre dos archivos.

Para utilizar esta herramienta primero debe descargarse Git desde [10], y una vez instalado se requiere una pequeña configuración, la cual puede verse en el video tutorial [11], pero de igual manera se describe a continuación.

Primero debe abrirse Git, para ello debe hacerse click derecho en una parte vacía de un directorio cualquiera, también puede ser el escritorio, y hacer click en “Git Bash Here”, como puede verse en la figura 3.4a. Luego, en la ventana del programa, escribir los comandos como se muestran en la figura 3.4b.



(a) Abrir Git



(b) Comandos para la configuración inicial

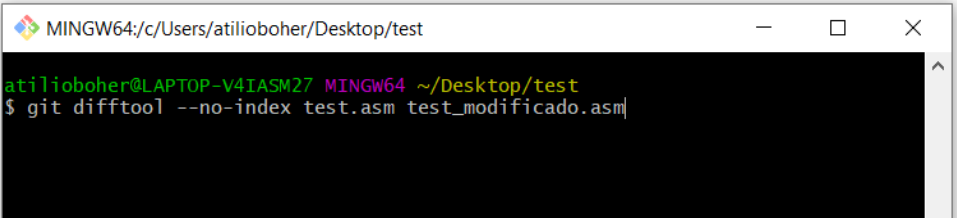
Figura 3.4: Configuración inicial de Git

Al ejecutar el último comando, Visual Studio Code se abrirá, y la configuración realizada debería verse como en la figura 3.5, de no ser así, debe rellenarse lo faltante, guardar el archivo y cerrar el programa.

```
[diff]
  tool = vscode
[difftool "vscode"]
  cmd = "code --wait --diff $LOCAL $REMOTE"
```

Figura 3.5: Configuración de Git relacionada a difftool

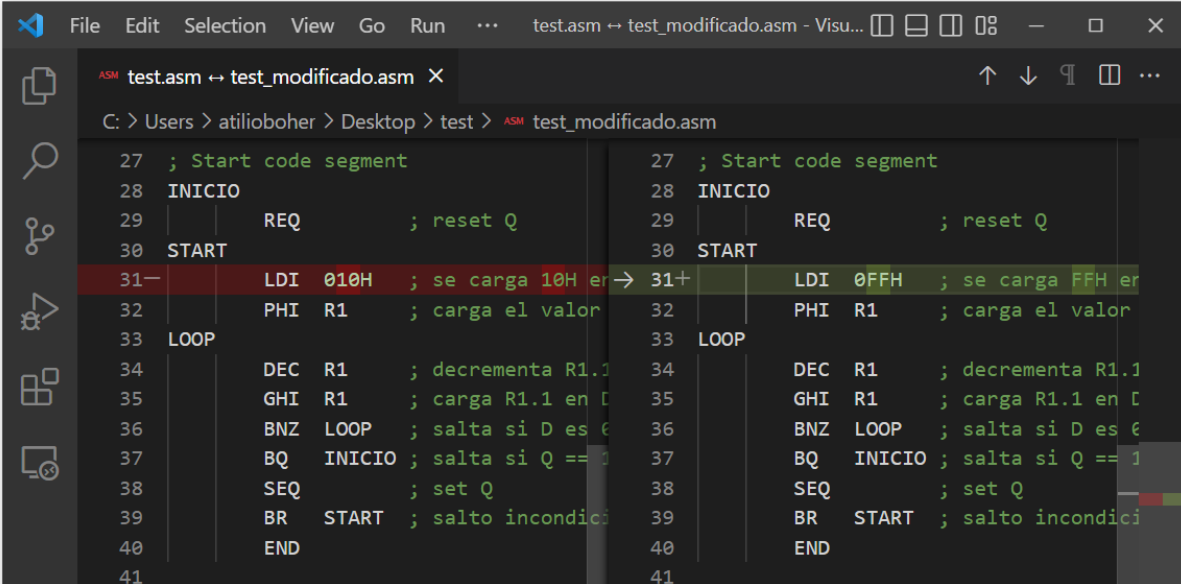
Una vez hecho todo esto, Git se encuentra configurado y se pueden comparar dos archivos simplemente abriendo Git, como ya se indicó en la figura 3.4a, y escribiendo el comando que se observa en la captura de la figura 3.6, donde los dos últimos parámetros son los nombres de los archivos a comparar (en este caso `test.asm` y `test_modificado.asm`).



```
MINGW64/c/Users/atilioboher/Desktop/test
atilioboher@LAPTOP-V4IASM27 MINGW64 ~/Desktop/test
$ git difftool --no-index test.asm test_modificado.asm
```

Figura 3.6: Comando para comparar dos archivos con Git

En la figura 3.7 puede apreciarse la diferencia entre los códigos, en este caso es una simple diferencia ilustrativa.



The screenshot shows a side-by-side comparison of two assembly files. The left pane shows the original file (`test.asm`) and the right pane shows the modified file (`test_modificado.asm`). The difference is highlighted in line 31: the original code loads `010H` into register `R1`, while the modified code loads `0FFH`. The rest of the code is identical.

Line	test.asm	test_modificado.asm
27	; Start code segment	; Start code segment
28	INICIO	INICIO
29	REQ ; reset Q	REQ ; reset Q
30	START	START
31	LDI 010H ; se carga 10H en R1	LDI 0FFH ; se carga FFH en R1
32	PHI R1 ; carga el valor en R1	PHI R1 ; carga el valor en R1
33	LOOP	LOOP
34	DEC R1 ; decrementa R1.1 en 1	DEC R1 ; decrementa R1.1 en 1
35	GHI R1 ; carga R1.1 en R1	GHI R1 ; carga R1.1 en R1
36	BNZ LOOP ; salta si D es 0	BNZ LOOP ; salta si D es 0
37	BQ INICIO ; salta si Q == 1	BQ INICIO ; salta si Q == 1
38	SEQ ; set Q	SEQ ; set Q
39	BR START ; salto incondicional	BR START ; salto incondicional
40	END	END
41		

Figura 3.7: Ejemplo de comparación

Lo útil de esta herramienta es que no solo indica la línea, sino también el lugar exacto del cambio o diferencia.

3.3. Ensamblador A18

El ensamblador utilizado se denomina A18 y fue encontrado en [12] donde se encuentra el enlace para poder descargarlo. Con el mismo es posible ensamblar código assembler del CDP1802 (con los mne-mónicos indicados en su hoja de datos [1]) y obtener los archivos en formato Intel (.hex) y el listing (.lst). Además cuenta con un manual donde se indica cómo utilizarlo y todos los tipos de errores que pueden encontrarse a la hora de realizar el ensamblado.

Para utilizarlo es necesario tener el archivo a18.exe en el mismo directorio que el código .asm que busca ensamblarse. Luego, manteniendo apretado Shift izquierdo, se debe hacer click en una parte vacía del directorio y luego seleccionar “Abrir la ventana de PowerShell aquí”, como se aprecia en la figura 3.8 (es posible que en vez de PowerShell diga Command Prompt, cualquiera sea el caso, se prosigue de la misma manera), esto abre una ventana de comandos en el directorio en cuestión.

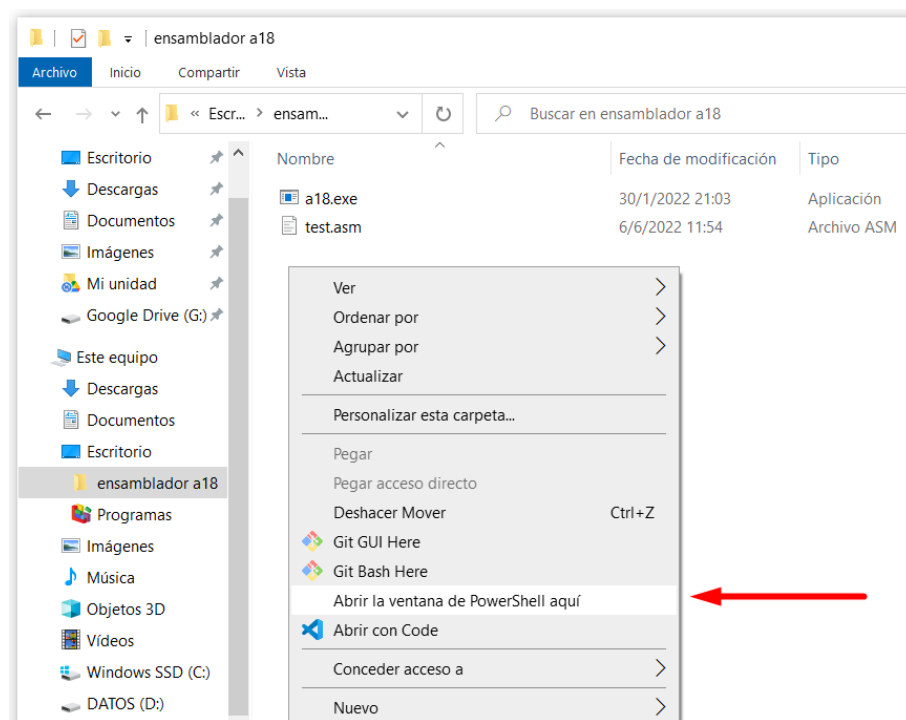
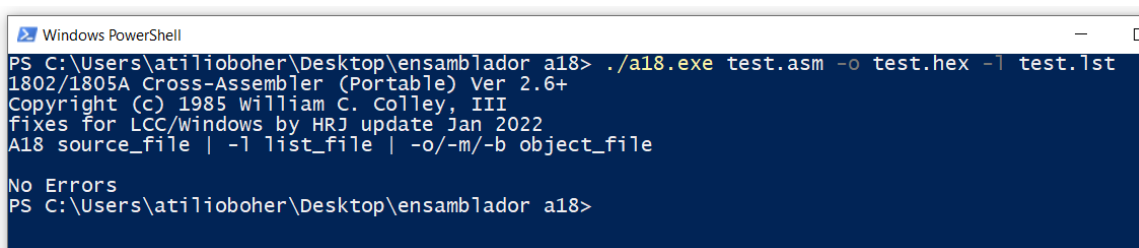


Figura 3.8: Apertura de ventana de comandos en directorio

En la ventana de comandos se deben reemplazar los nombres correspondientes; el primero es el nombre del programa a ensamblar, en formato .asm, el segundo es el nombre del archivo .hex a generar, y el último es el nombre del archivo listing, en formato .lst.

```
./a18.exe <nombre-de-archivo>.asm -o <nombre-de-archivo>.hex -l <nombre-de-archivo>.lst
```

En la figura 3.9 puede verse el resultado del ensamblado de un programa y si el mismo posee errores, es indicado al ejecutar el comando. En el archivo listing generado se encuentran indicadas las líneas que poseen errores, utilizando un código para identificar cada tipo de error, de manera que los detalles sobre los mismos pueden ser encontrados en el manual del ensamblador, que como ya se mencionó, puede encontrarse en la página [12].

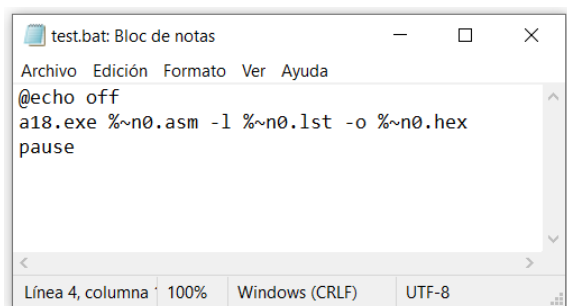


```
PS C:\Users\atilioboher\Desktop\ensamblador a18> ./a18.exe test.asm -o test.hex -l test.lst
1802/1805A Cross-Assembler (Portable) Ver 2.6+
Copyright (c) 1985 William C. Colley, III
fixes for LCC/Windows by HRJ update Jan 2022
A18 source_file | -l list_file | -o/-m/-b object_file

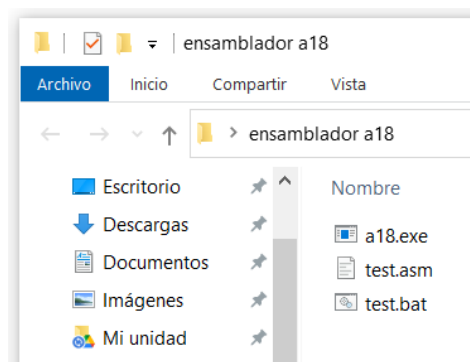
No Errors
PS C:\Users\atilioboher\Desktop\ensamblador a18>
```

Figura 3.9: Resultado del ensamblado en ventana de comandos

Escribir este comando puede resultar engorroso. Por lo cual se escribió un pequeño script Batch, cuyas instrucciones se encuentran en la figura 3.10a. Este script Batch se puede ejecutar con doble click y ensambla el código. Para ello el archivo Batch .bat debe tener el mismo nombre que el archivo .asm a ensamblar. En la figura 3.10b se encuentra una captura de pantalla de los archivos necesarios, al hacer doble click en test.bat, se abrirá una ventana de comandos y test.asm se ensamblará automáticamente, sin necesidad de escribir ningún comando.



(a) Instrucciones Batch para el ensamblado con doble click



(b) Uso del archivo .bat

Figura 3.10: Ensamblado con script Batch

3.4. Emulador Emma02

Existe un software llamado Emma02, el cual puede encontrarse en su página oficial [13]. El mismo permite la simulación de varios modelos de computadoras basadas en el microprocesador CDP1802. En la figura 3.11 puede apreciarse la variedad de computadoras disponibles, sin embargo, la más utilizada fue el Membership Card, seguida del Netronics Elf II.

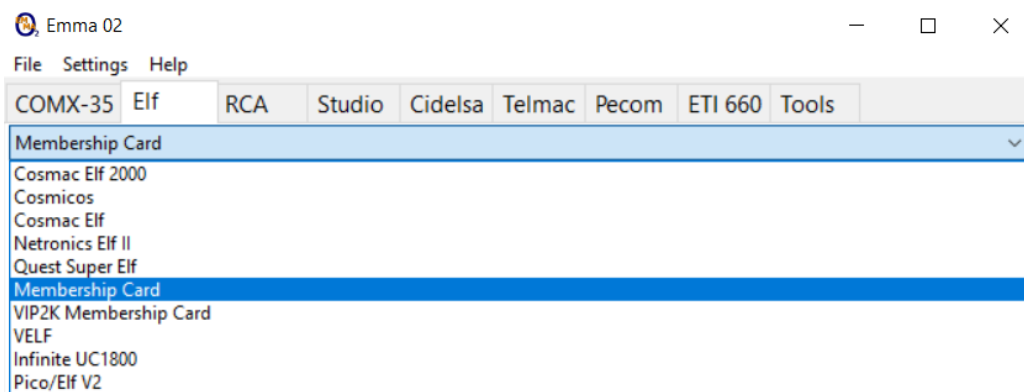


Figura 3.11: Computadoras disponibles que utilizan el CDP1802

El simulador cuenta con programas ya disponibles para cada computadora. Para utilizar uno, debemos seleccionarlo en la solapa que se indica en la figura 3.12; pero hay que tener en cuenta que quizás requiera cambiar alguna configuración de la computadora, como seleccionar la velocidad de transmisión correcta, el mapa de memoria, la dirección donde el microprocesador comenzará a ejecutar el programa, etc.

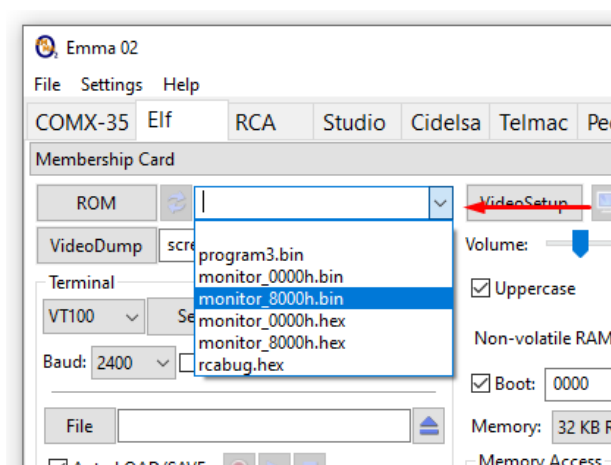


Figura 3.12: Seleccionar programa

Además, existe una forma de seleccionar programas ya disponibles en el simulador junto a la confi-

guración específica necesaria para su funcionamiento, para ello hay que ir a File->Configuration->Load, como se indica en la figura 3.13, para el caso de la computadora Netronics Elf II.

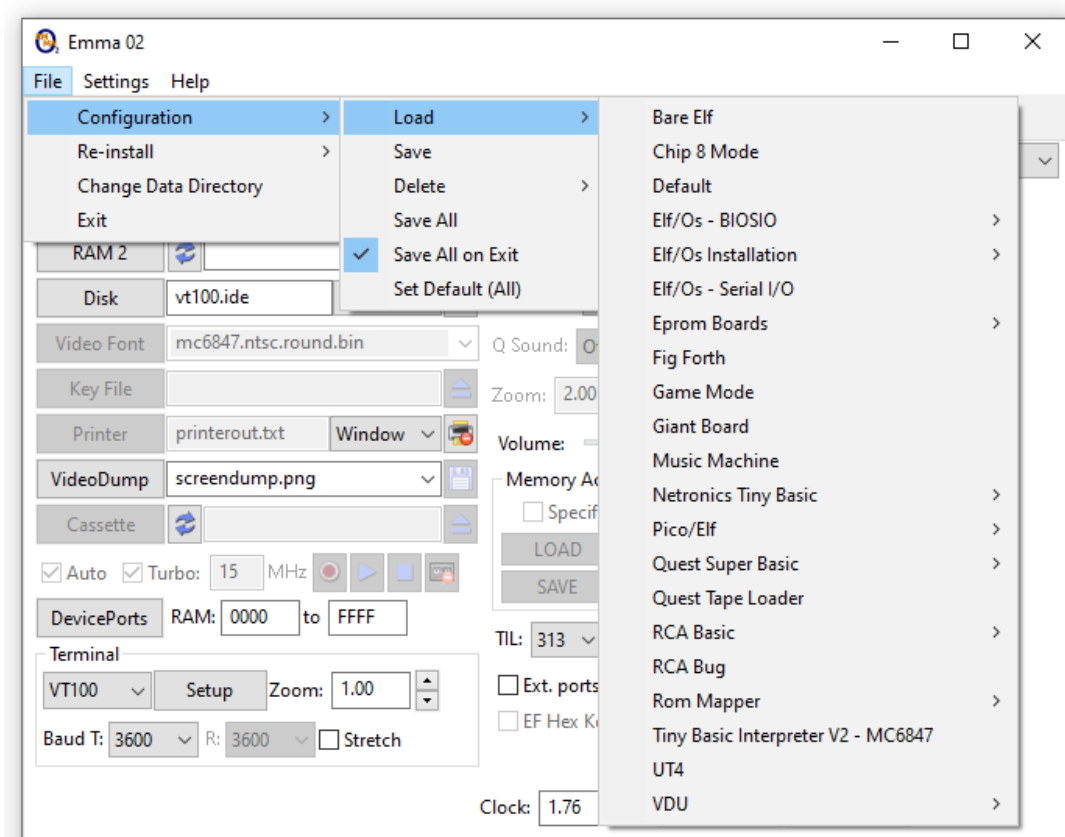


Figura 3.13: Seleccionar programa y configuración

Algunos de estos programas se encuentran en formato `.bin` y otros en formato `.hex`. Además, algunos también poseen el código fuente `.asm` en la carpeta contenedora. Para acceder a esta carpeta podemos hacer click en “ROM”, como se indica en la figura 3.14. Esto abrirá una ventana para seleccionar un archivo `.hex` o `.bin`, y en esta ventana puede verse el directorio del emulador donde se encuentran los programas disponibles.

En este directorio (hay uno por cada modelo de computadora) pueden agregarse programas obtenidos en línea, o escritos y ensamblados por uno mismo para ser probados.

El emulador también provee una herramienta denominada “Direct Assembler” (ver figura 3.15), la cual muestra el código del programa en lenguaje assembler; puede ensamblar y desensamblar. Sin embargo, en este proyecto, se utilizó únicamente como herramienta de depuración de código y para el ensamblado se utilizó el ensamblador A18, ya expuesto en la sección 3.3.

También existe una pestaña donde puede visualizarse los registros internos del microprocesador, y haciendo click en el botón “Trace” (figura 3.16), se registran todas las instrucciones ejecutadas, pudiendo

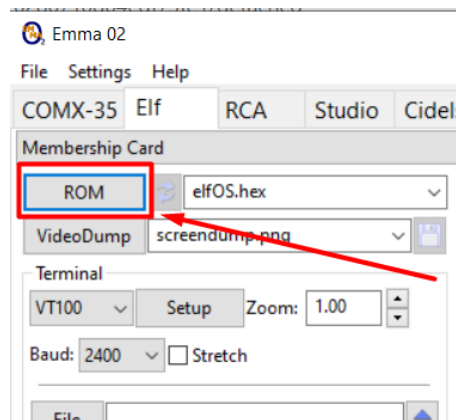


Figura 3.14: Seleccionar programa en Emma02

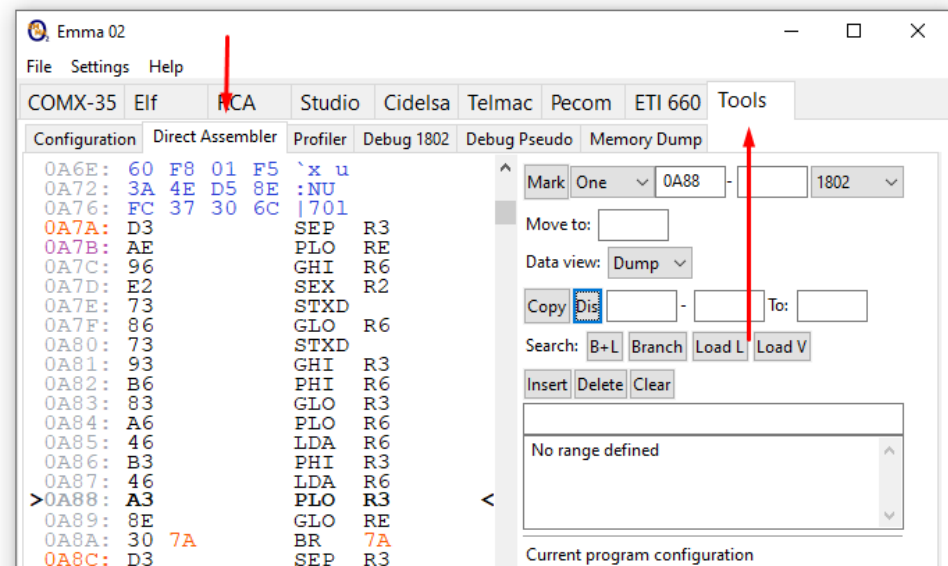


Figura 3.15: Direct Assembler

luego copiar y pegar esto en un documento de texto para analizar detenidamente la ejecución de una secuencia de código.

En la figura 3.17 se encuentra una captura de instrucciones registradas con la herramienta “Trace”, en rojo se marcó una instrucción de escritura en memoria como ejemplo para ver cómo puede visualizarse información de interés, en este caso, tanto la dirección de memoria que fue escrita como el dato escrito en ella.

Estas características y herramientas mencionadas fueron las más utilizadas, sin embargo, se encuentran descritos más detalles de estas herramientas y otras más en la sección de ayuda de la página del emulador [14].

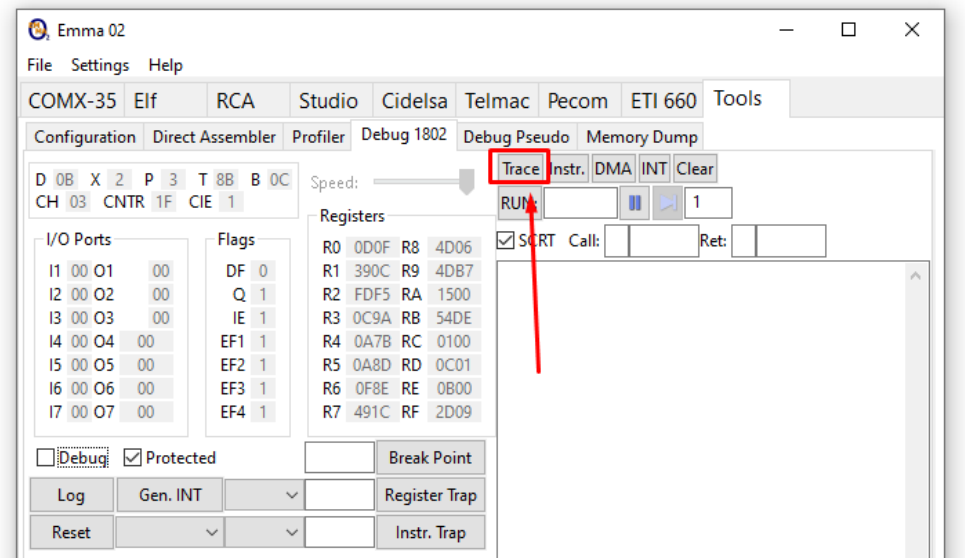


Figura 3.16: Registrar instrucciones ejecutadas y visualización de Registros internos

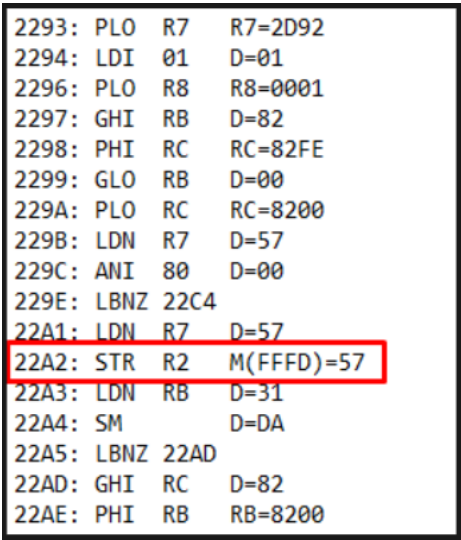


Figura 3.17: Captura de instrucciones ejecutadas

3.5. Terminal Tera Term

Para la comunicación con la computadora se utiliza una terminal externa, en este caso se utilizó el programa Tera Term®[15] ejecutándose en un ordenador personal. El mismo es sencillo de usar, sin embargo, se mencionan algunas configuraciones que se hicieron para volver más fácil su uso.

En la figura 3.18 se puede ver la configuración realizada para tener un tamaño de letra más grande, debido a que la configuración por defecto es muy chica. Esta ventana de configuración puede encontrarse en Setup->Font->Font...

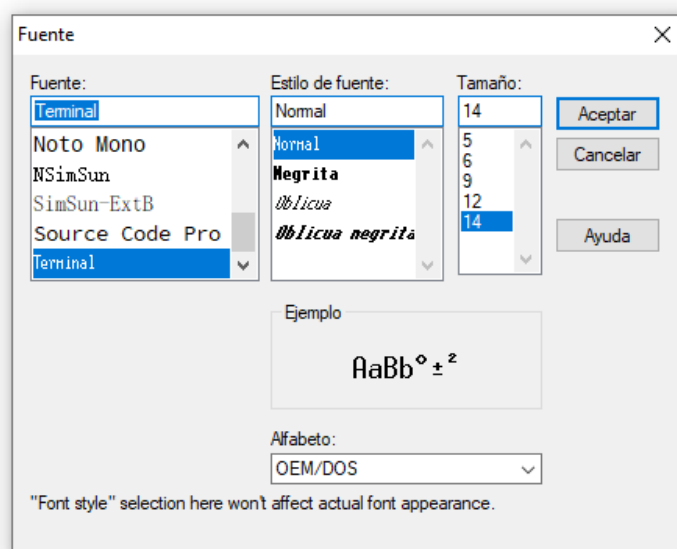


Figura 3.18: Tamaño y tipo de letra de la terminal

La configuración realizada se pierde cada vez que el programa es cerrado, por lo tanto, cada vez que se abre el mismo debería realizarse la misma configuración nuevamente. Para evitar esto es posible guardar la configuración actual, para ello es necesario ir a Setup->Save setup..., una ventana se abrirá y podrá guardarse un archivo llamado "TERATERM.INI", guardarlo reescribe la configuración anterior, por lo tanto, si no desea perderla, debería guardar el archivo de configuración ya existente antes de sobrescribirlo.

Algunos programas pueden ser cargados desde la terminal. Esto es usado, por ejemplo, para cargar programas escritos en Basic, al intérprete de Basic corriendo en el microprocesador, el cual es discutido en el capítulo 6: Sistema medio, sección 6.2: Programas del sistema medio.

Para realizar esta tarea se tiene que ir a File->Send file... y seleccionar el archivo que contiene el programa a ser enviado a través de la terminal (ver figura 3.19a). Como el microprocesador requiere de un período de tiempo mínimo para poder procesar un carácter recibido, no es posible enviarlos todos de seguido sin establecer un retardo entre cada uno de ellos; por esta razón Tera Term posee una configuración para esto en Setup->Serial port..., en la ventana de configuración pueden configurarse los retardos entre caracteres y líneas; en la figura 3.19b se indica dónde colocar estos valores.

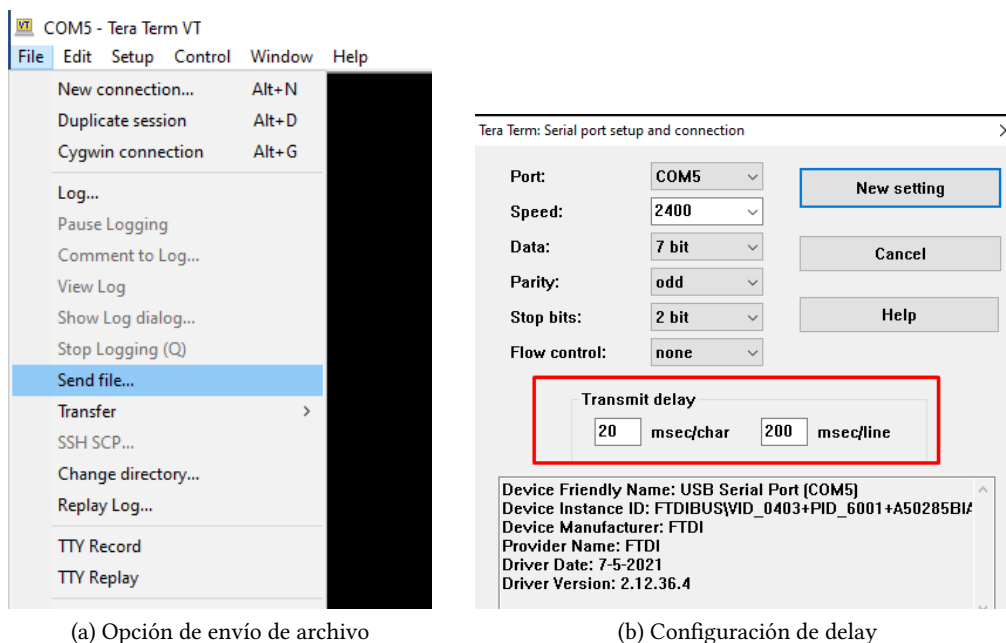


Figura 3.19: Cargado de programas a través de la terminal

Para que este proceso funcione correctamente debe respetarse el carácter ASCII utilizado para representar un salto de línea. Dado que distintos formatos de texto utilizan diferentes caracteres para esto (algunos utilizan retorno de carro CR, del inglés *Carriage Return*, o salto de línea LF, del inglés *Line Feed*). Estos no necesariamente son los que el programa ejecutándose en el microprocesador interpreta como salto de línea, a veces es necesario modificarlos. Para ello se utilizó el programa discutido en la siguiente sección, Multi Commander.

3.6. Multi Commander

Multi Commander [16] es un programa para el manejo de archivos y se utiliza como alternativa al explorador de Windows. Sin embargo, es de interés para este proyecto únicamente la herramienta para cambiar los caracteres utilizados para representar saltos de línea en archivos de texto. En la figura 3.20 se puede observar dónde encontrar esta herramienta dentro del programa.

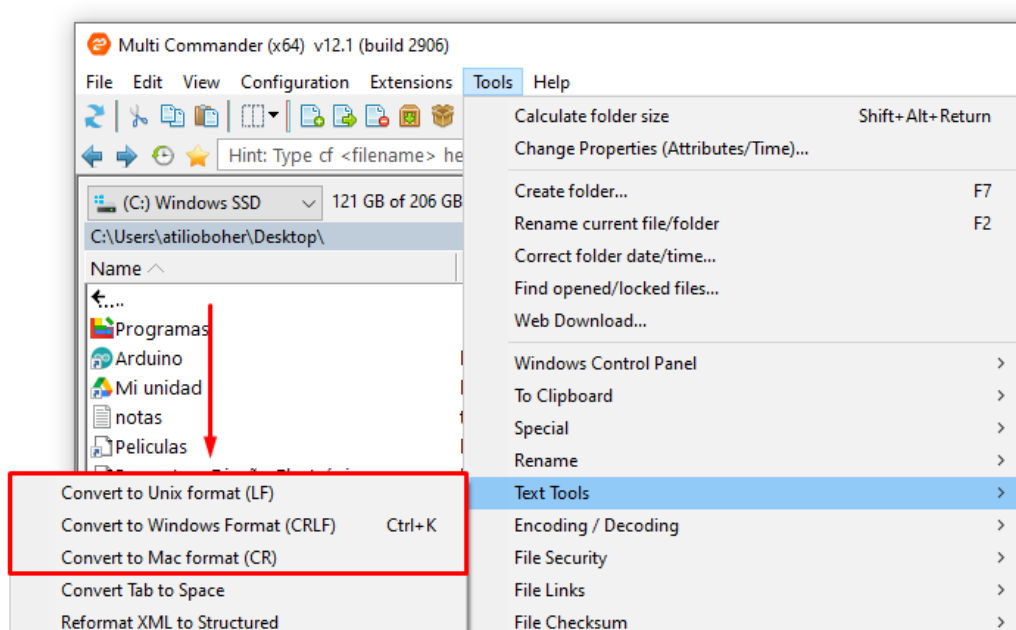


Figura 3.20: Herramienta para cambio de carácter de salto de línea en Multi Commander

3.7. Programador universal XGecu TL866-3G

Para el grabado de la memoria se utilizó el programador universal XGecu TL866-3G (figura 3.21). Para ello fue necesaria la descarga e instalación del software correspondiente al mismo, el cual puede ser encontrado en la página oficial del fabricante [17]. Se recomienda descargar la opción 2 “DownLoad From Local Server”.

En la figura 3.22 puede verse el entorno gráfico del programa. Con la flecha 1 se encuentra indicado donde seleccionar la memoria a ser grabada o leída. En este proyecto fueron utilizadas las memorias AT28C64[18] y X28C256[19], ambas disponibles en la librería del programador. Con la flecha 2 se indica una función para el testeo de chips lógicos digitales, herramienta que fue de utilidad en varias ocasiones.



Figura 3.21: Programador universal XGecu TL866-3G

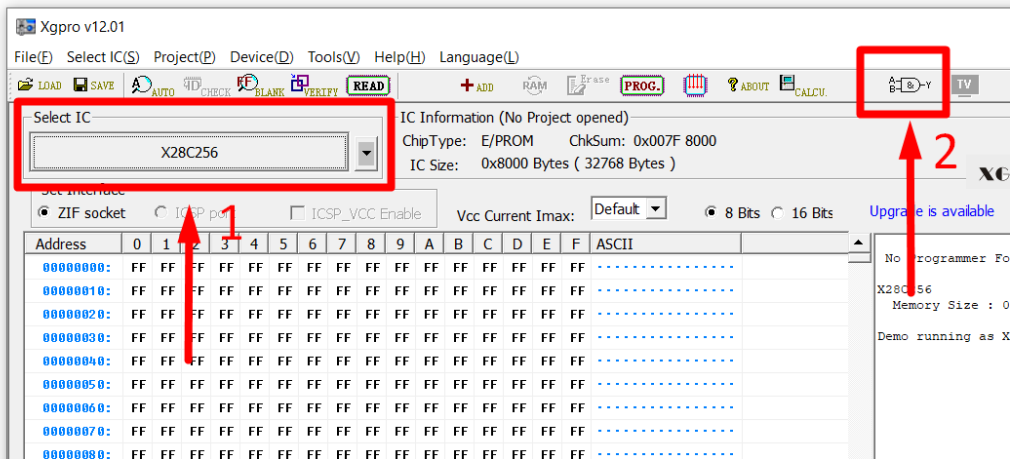


Figura 3.22: Entorno gráfico del programa del programador universal XGecu TL866-3G

3.8. KiCad EDA

Para la construcción de los circuitos electrónicos del presente proyecto se utilizó el software KiCad, un software libre para el diseño electrónico automatizado (del inglés: Electronic Design Automation EDA) de circuitos electrónicos. Entre las aplicaciones más destacadas del software se encuentran: el editor de esquemas, el editor de circuitos impresos y el selector de huellas impresas para los componentes utilizados, entre otras. KiCad es un software de código abierto y gratuito, el cual puede ser descargado de su página oficial [20]. En línea pueden encontrarse video-tutoriales muy completos para el uso adecuado del mismo. La versatilidad de la herramienta mencionada ha proporcionado simpleza al momento de confeccionar los esquemáticos y los diagramas de circuito impreso para las placas del proyecto. A modo de ejemplo en las figuras 3.23 y 3.24 se encuentran el editor de esquemas y el editor de circuitos impresos del software KiCad.

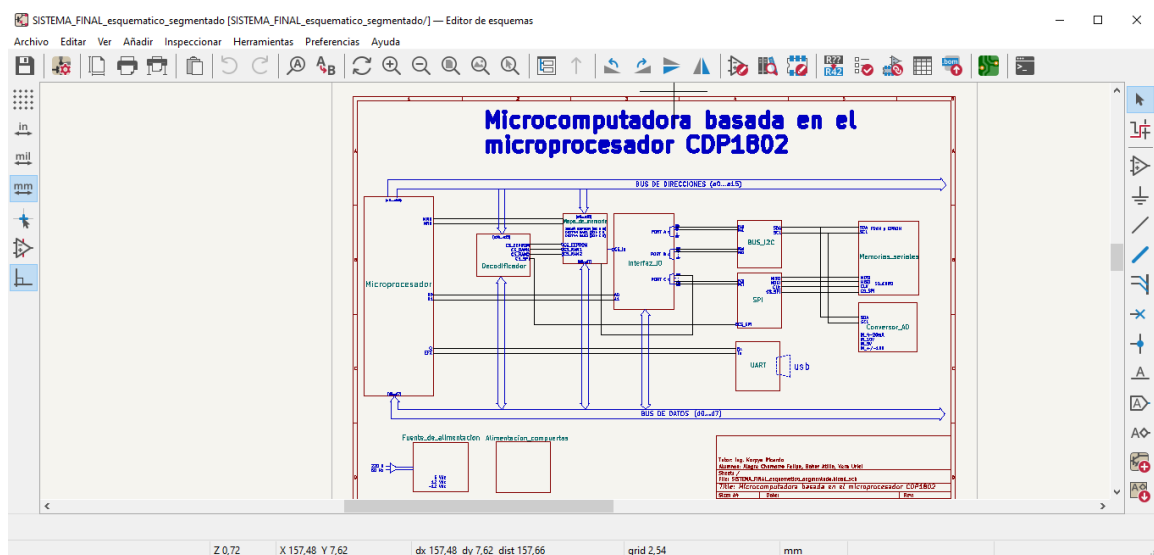


Figura 3.23: Entorno gráfico del editor de esquemas del software KiCad

Una herramienta destacada para el diseño de los circuitos impresos es el editor de huellas y de símbolos, lo cual es útil a la hora de confeccionar un esquemático o un PCB cuando se posee un dispositivo cuyo símbolo o huella no está incluida dentro de las librerías del programa. KiCad cuenta con un entorno adecuado para el dibujo y desarrollo de los elementos mencionados, permitiendo agregar a las librerías los mismos. Esta solución realza la aplicabilidad del programa y ha sido de gran utilidad para el desarrollo del trabajo.

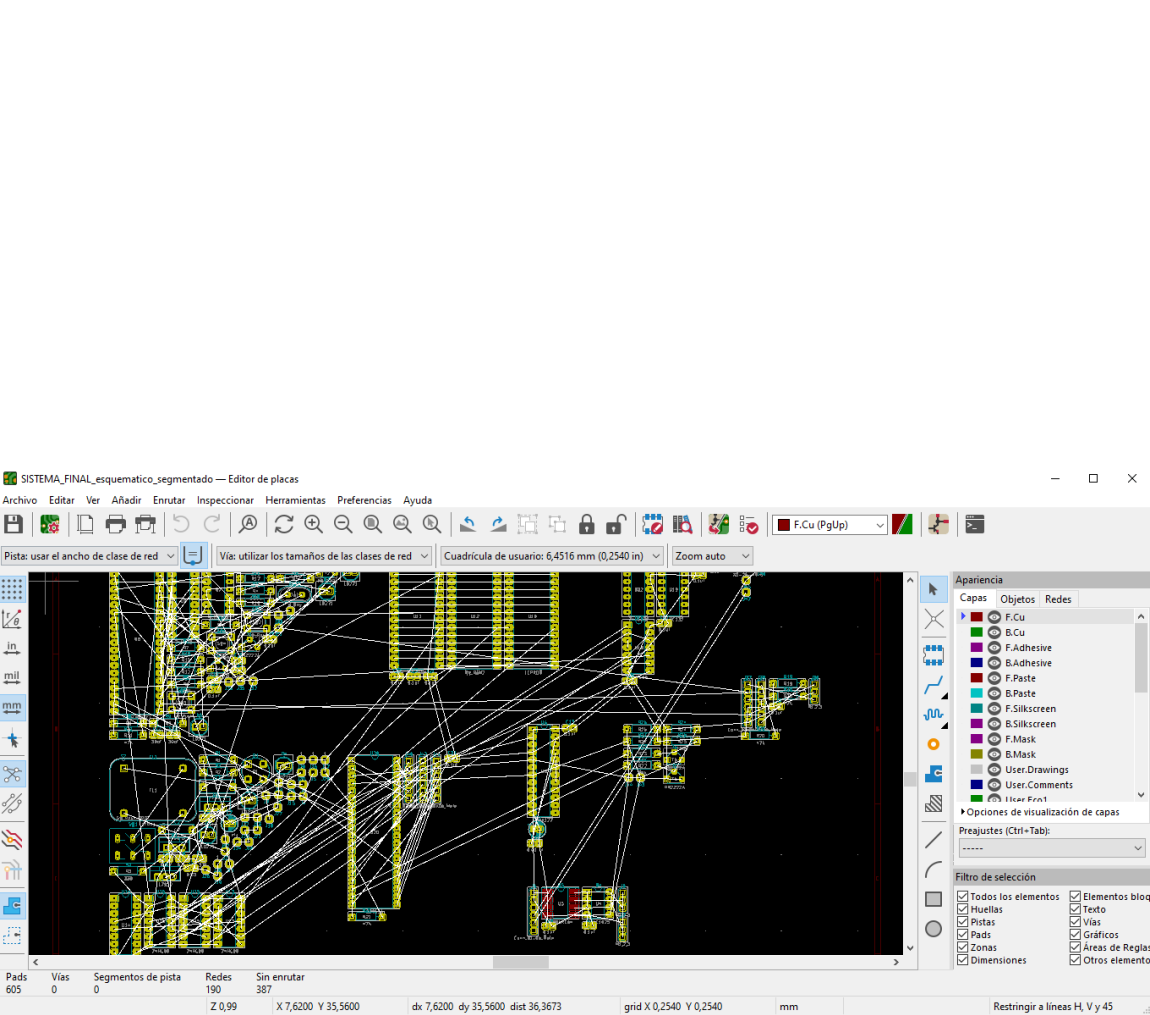


Figura 3.24: Entorno gráfico del editor de circuito impreso del software KiCad

Pruebas en el COSMAC ELF II

En las instalaciones de la facultad de ingeniería se cuenta con el COSMAC ELF II, el cual fue comentado en la sección 2.2.2. A modo de introducción, antes de comenzar a desarrollar los sistemas del proyecto, se realizaron pruebas sobre el COSMAC ELF II, para probar su funcionamiento y comenzar a comprender la operación del microprocesador.



Fotografía 4.1: COSMAC ELF II, propiedad de la facultad de ingeniería

En la figura 4.1 se encuentra una fotografía del COSMAC ELF II que se encuentra en la facultad. Para utilizarlo fue necesario realizar reparaciones sobre el mismo. Un capacitor de entrada en la etapa de alimentación (en paralelo al pin de entrada del regulador LM7805) se encontraba dañado y fue necesario reemplazar el mismo, como así también soldar un cable a la placa para reemplazar una pista que se encontraba dañada. Luego se soldaron dos cables a los pines de alimentación de la placa de la computadora, para así poder conectarla a una fuente de alimentación.

Una vez realizadas estas reparaciones, se conectó el COSMAC ELF II a una fuente de alimentación configurada a 8 V y se encendió. Para comprobar su funcionamiento correcto, se grabaron dos programas encontrados en línea [21], junto con la explicación detallada del procedimiento para realizar la programación.

Se puede cargar un programa en el COSMAC ELF II poniendo el interruptor LOAD en la posición ON, con RUN y MP (Memory Protect) en OFF, y luego ingresar un byte con el teclado para finalmente presionar el botón INPUT. Esto graba un byte en la primera dirección de memoria. Para grabar otro byte en la siguiente dirección de memoria simplemente se debe ingresar el mismo en el teclado y volver a presionar INPUT. Una vez que se ingresaron todos los bytes del programa, MP debe colocarse en ON para proteger la memoria.

Con el programa ya cargado, para correrlo, se debe colocar LOAD en ON, esto resetea el microprocesador, luego se coloca RUN en ON, con lo cual el programa comienza a funcionar.

Siguiendo este procedimiento, se comprobó el funcionamiento del COSMAC ELF II con dos programas, uno que hace parpadear el LED Q, y otro que enciende el mismo cuando el botón INPUT es presionado.

Sistema mínimo

El primer circuito diseñado para el desarrollo del trabajo fue el **Sistema Mínimo**, un sistema simplificado que consta de una arquitectura simple, con el objetivo de analizar el funcionamiento del microprocesador y el uso del mapa de memoria. Este sistema consta de dos grandes bloques como se observa en la figura 5.1: el microprocesador y la memoria.

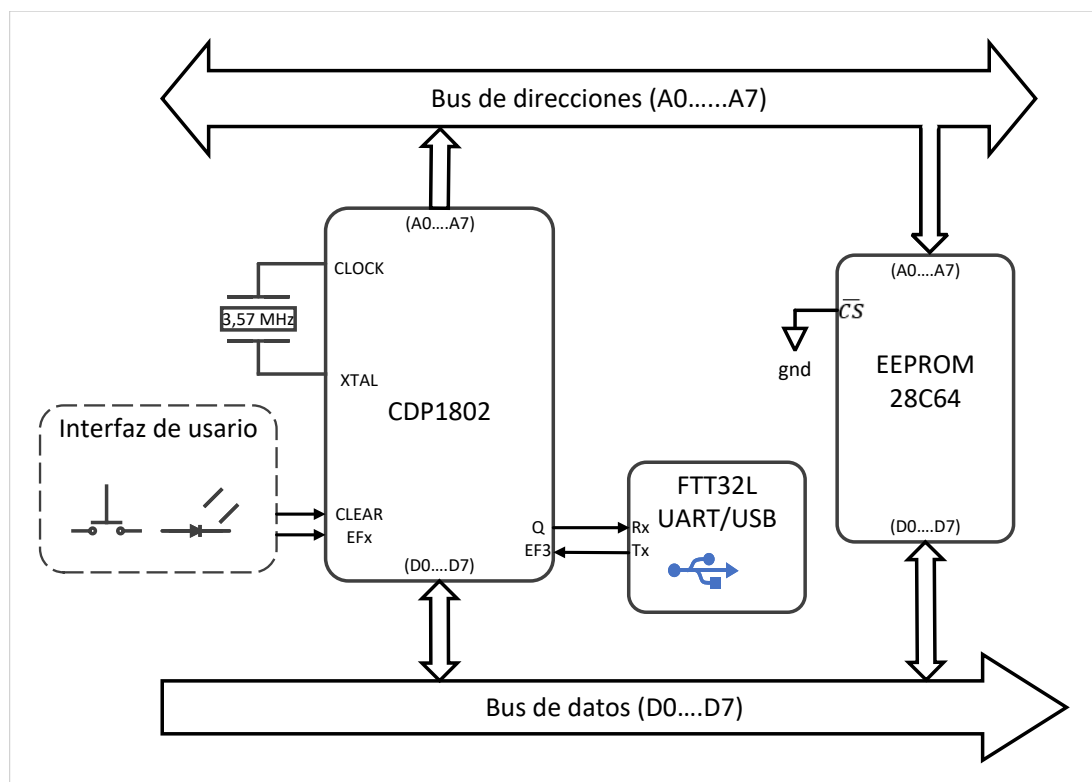


Figura 5.1: Diagrama de bloques del sistema mínimo

El microprocesador es el elemento central del circuito diseñado, que es donde se realiza todo el procesamiento de datos. La incorporación del mismo permite realizar pruebas de funcionamiento, facilitando el entendimiento del lenguaje de programación y la secuencia de las instrucciones y ejecución dentro del microprocesador. Por otro lado, la memoria dentro de este sistema cumple la función de almacenar las instrucciones del programa a ejecutar y no se utiliza la misma para guardar datos generados en ejecución. Debido a la simplicidad del sistema presentado y la arquitectura del microprocesador, solo se utilizó un direccionamiento de 8 bits, lo que limita el tamaño del programa principal a un máximo de 256 bytes. Esto se debe a que el direccionamiento de 16 bits requiere el agregado de más elementos al circuito lo cual se explica en el desarrollo del sistema medio en la sección 6. Con esta arquitectura los programas desarrollados debían ser simples con el único objetivo de determinar y comprobar el correcto funcionamiento de las conexiones y lograr un entendimiento básico del lenguaje de programación y del microprocesador. Para este fin se utilizan los programas desarrollados y presentados en la sección 5.2.

Debido a las limitaciones en el direccionamiento, se dispuso un diseño que permite establecer el estado de los pines de dirección A8 y A9 de la memoria de programa (28C64), directamente sobre el chip. Esto permite utilizar cuatro sectores de la memoria, aunque no simultáneamente. El microprocesador sigue utilizando únicamente 8 bits para el direccionamiento de memoria. Para la modificación se utilizan puentes tipo jumper que permiten conectar a masa o a VCC tanto A8 como A9 individualmente. De esta forma se establece un mapa de memoria como el presentado en la figura 5.2. De esta manera al realizar el grabado de la memoria EEPROM se pueden incluir los cuatro programas presentados en la sección 5.2 y pueden ser probados sin necesidad de regrabar la memoria.

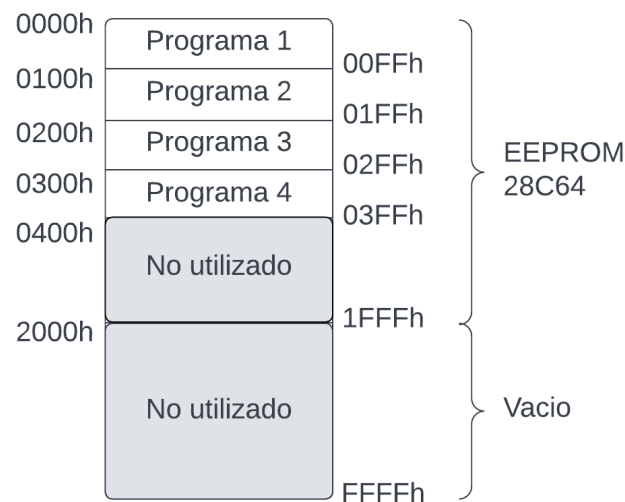


Figura 5.2: Mapa de memoria Sistema Mínimo

Para elegir cada uno de los programas debe conectarse adecuadamente un jumper en los conectores de J10 y otro jumper en los conectores de J11, como se describe a continuación.

Programa 1: Conectar un jumper en los pines 2 y 3 de J10, y otro jumper en los pines 2 y 3 de J11.

Programa 2: Conectar un jumper en los pines 1 y 2 de J10, y otro jumper en los pines 2 y 3 de J11.

Programa 3: Conectar un jumper en los pines 2 y 3 de J10, y otro jumper en los pines 1 y 2 de J11.

Programa 4: Conectar un jumper en los pines 1 y 2 de J10, y otro jumper en los pines 1 y 2 de J11.

Para que el circuito funcione correctamente se implementó un sistema de alimentación basado en un regulador de tensión de 5 V, específicamente el UTC7805. La incorporación del mismo proporciona mayor estabilidad en la alimentación protegiendo a los circuitos integrados del sistema. También se incorporó una interfaz de usuario simple que consta de pulsadores como sistema de entrada y LED indicadores de funcionamiento.

Para la realización del esquemático y el diseño de la placa PCB se utilizó el programa KiCad. El esquema obtenido se presenta en la figura 5.3, además se lo presenta en el apéndice donde se encuentra en tamaño original. La placa PCB diseñada es una simple faz, la pista generada se presenta en la figura 5.4a, mientras que la máscara de componentes de la misma se presenta en la figura 5.4b.

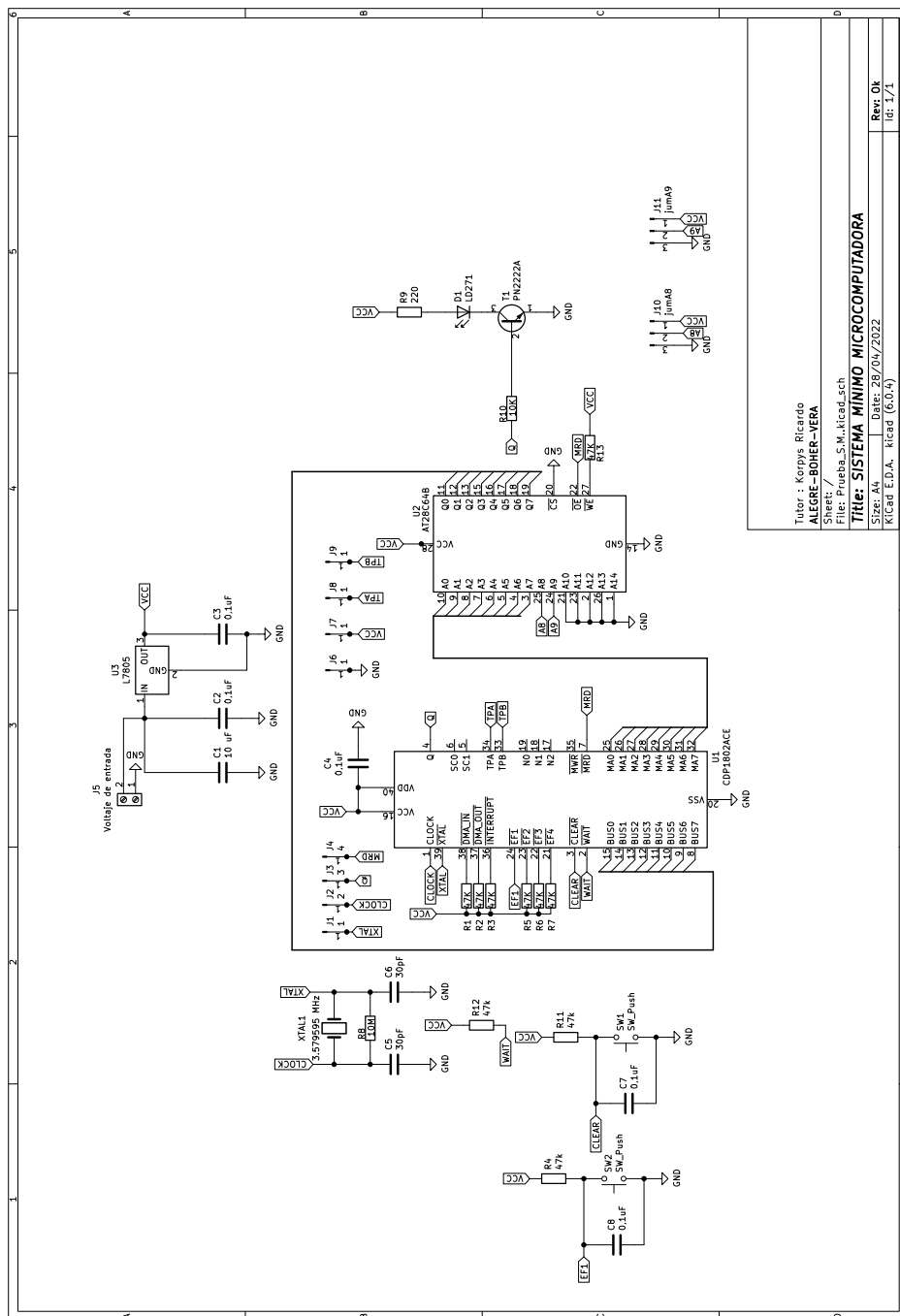
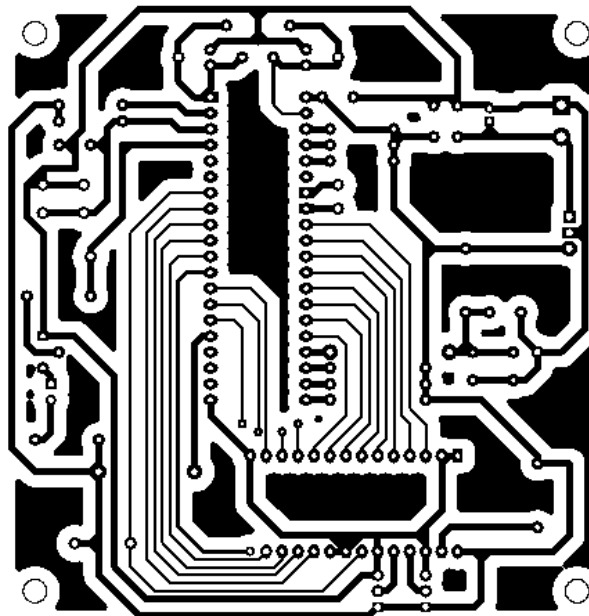
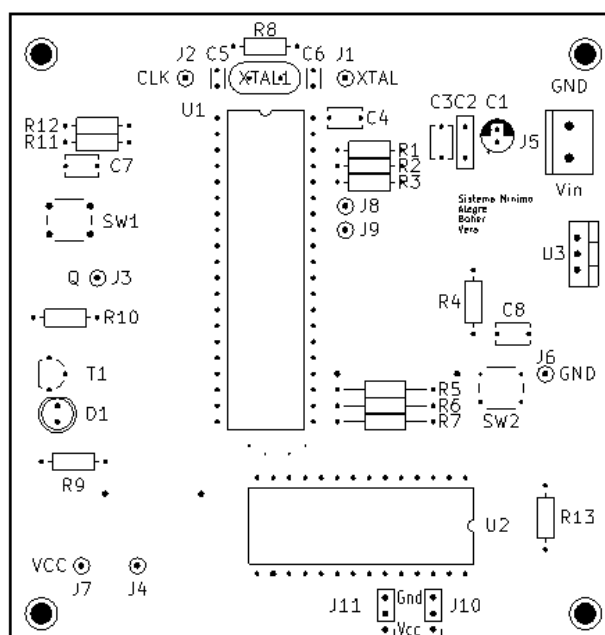


Figura 5.3: Esquema de sistema mínimo



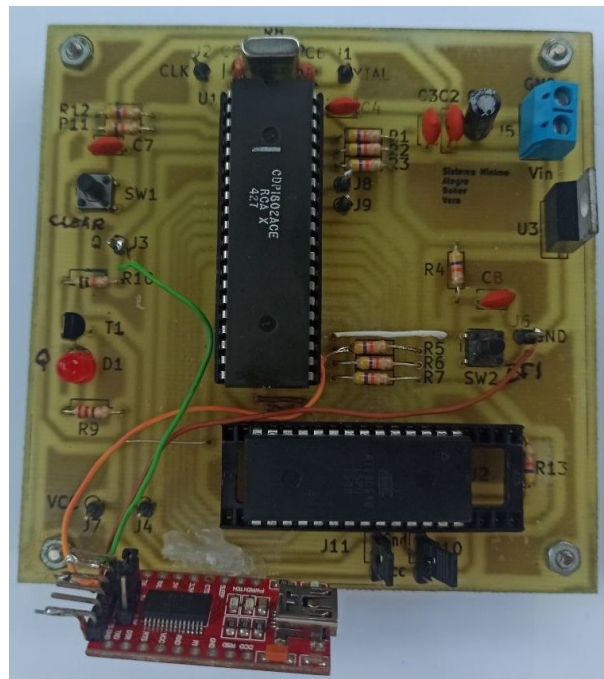
(a) Pista diseñada para sistema mínimo



(b) Máscara de componentes sistema mínimo

Figura 5.4: Placa PCB diseñada para sistema mínimo

A partir de los diagramas presentados se procedió a la construcción del circuito. El resultado obtenido se presenta en la fotografía 5.1. El mismo funcionó correctamente sin necesidad de realizar modificaciones a la arquitectura propuesta. Sin embargo, para la prueba de transmisión de datos mediante UART, con el programa que se presentará en la sección 5.2.4, se utilizó una interfaz externa para la conversión del sistema UART a USB, esto permite la recepción de la información en una terminal. La interfaz utilizada es el circuito FT232RL[22], la cual fue contemplada para los diseños posteriores al sistema mínimo.



Fotografía 5.1: Sistema mínimo construido

5.1. Lista de componentes

En la tabla 5.1 se encuentra la lista de materiales para la construcción del sistema mínimo.

Tabla 5.1: Lista de componentes del sistema mínimo

Cantidad	Etiqueta Identificador	Descripción	Fabricante	Número de parte
1	U1	Microprocesador	RCA	CDP1802ACE
1	U2	Memoria EEPROM 8k x 8	ATMEL	AT28C64B
1	U3	Regulador de tensión	YOUW ANG	UTC7805
1	T1	Transistor BJT NPN	ST Microelectro-nics	PN2222A
1	XTAL1	Cristal 3,579 545 MHz	CQ Electronics	Varios
2	SW1, SW2	Pulsador 6mm x 4,3mm	Varios	Varios
10	R1, R2, R3, R4, R5, R6, R7, R11, R12, R13	Resistor 47 k Ω 1/8 W	Varios	Varios
1	R8	Resistor 10 M Ω 1/8 W	Varios	Varios
1	R9	Resistor 220 Ω 1/4 W	Varios	Varios
1	R10	Resistor 10 k Ω 1/8 W	Varios	Varios
1	D1	LED Rojo 5mm	Varios	Varios
1	C1	Capacitor electrolítico 10 μ F 25 V	Varios	Varios
5	C2, C3, C4, C7, C8	Capacitor cerámico 0,10 μ F 50 V	Varios	Varios
2	C5, C6	Capacitor cerámico 30 pF 50 V	Varios	Varios
8	J1, J2, J3, J4, J6, J7, J8, J9	Pin conector macho (1 <i>Male Header Pin</i>)	Varios	Varios
1	J5	Bornera 2 pines 5,08 mm	Varios	Varios
2	J10, J11	1x3 Pines Conectores Macho 2,54 mm	Varios	Varios

Algo no contemplado en esta lista de componentes es el módulo FT232RL [22], dado que fue soldado luego de armado el circuito, como puede apreciarse en la figura 5.1. El módulo FT232RL tiene soldado con cables sus pines Rx, Tx, Vcc y GND, a los pines Q, EF2, Vcc y GND del microprocesador, respectivamente.

5.2. Programas del sistema mínimo

Para probar el sistema mínimo, se confeccionaron cuatro programas sencillos de no más de 256 bytes, debido a que el sistema mínimo no puede direccionar más que esta cantidad de direcciones de memoria. Los programas se presentan a continuación, haciendo uso de diagramas de flujo. Sin embargo, en el apéndice C se encuentra el código fuente de los cuatro programas.

5.2.1. Programa 1: Parpadeo LED Q

El primer programa hace parpadear el LED conectado a la salida Q del microprocesador. Para ello, como puede apreciarse en el diagrama de flujo de la figura 5.5, el programa comienza apagando el LED ($Q = 0$), luego realiza un retardo por software, que consiste en la asignación de un valor específico a un registro, para luego decrementarlo hasta que llegue a cero. Una vez finalizado el retardo, el LED togglea; si está encendido, se apaga, y si está apagado, se enciende. Esta secuencia se repite constantemente, haciendo parpadear el LED conectado a la salida Q. En el apéndice C.1, se encuentra el código fuente del programa 1.

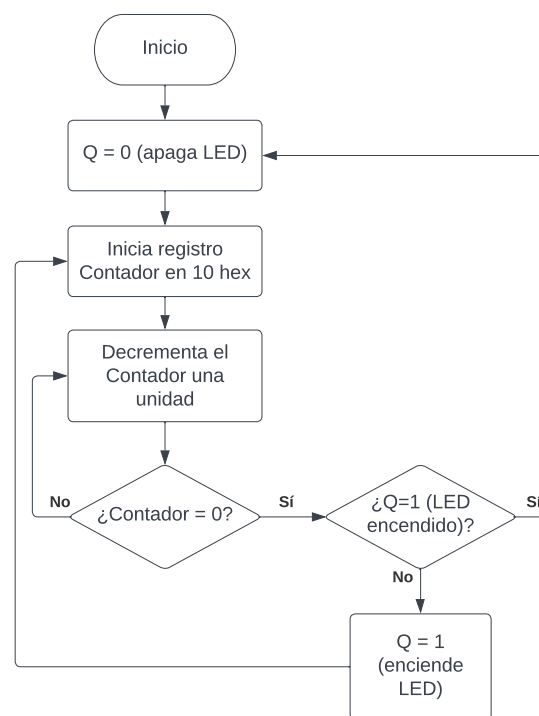


Figura 5.5: Programa 1

5.2.2. Programa 2: LED Q se enciende al presionar pulsador

El programa dos permite al usuario interactuar con el sistema, de manera que se prueba el funcionamiento de las entradas E_{Fx} (en el caso del sistema mínimo, EF1, dado que allí se encuentra conectado el pulsador). En el diagrama de flujo de la figura 5.6 se puede observar cómo opera el programa. Cuando el botón se encuentra presionado, el programa entra a un bucle infinito que ejecuta el comando para encender el LED, en caso contrario, el programa entra a un bucle similar, pero donde la instrucción que se ejecuta continuamente es la de apagado del LED. En el apéndice C.2, se encuentra el código fuente del programa 2.

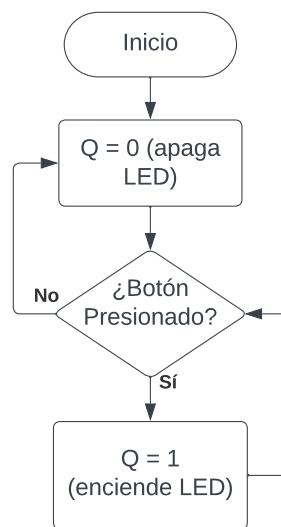


Figura 5.6: Programa 2

5.2.3. Programa 3: LED Q parpadea al presionar pulsador

El programa 3 es una combinación de los dos programas previos, el mismo hace parpadear el LED conectado a la salida Q del microprocesador, pero lo hace solo si el usuario presiona el pulsador.

En la figura 5.7 se encuentra el diagrama de flujo del programa, el mismo consiste de una rutina de retardo con un contador que disminuye hasta cero (como en el programa 1), y luego, si el pulsador se encuentra presionado, el LED comienza a parpadear, pero si el pulsador no se encuentra presionado, el LED permanece apagado. En el apéndice C.3, se encuentra el código fuente del programa 3.

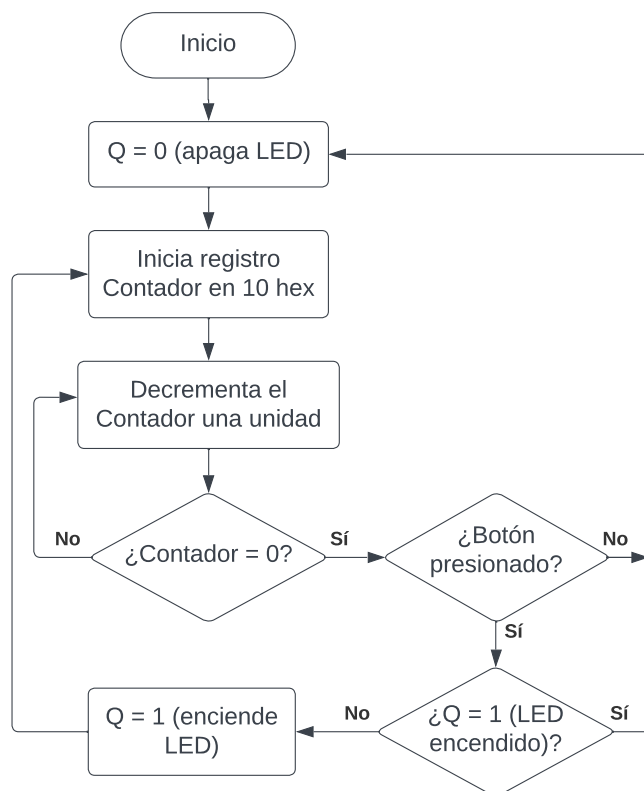


Figura 5.7: Programa 3

5.2.4. Programa 4: Transmisión serial de caracteres

El programa 4 realiza la transmisión serial por protocolo UART a través del pin Q de los caracteres “ABCDEFGH”, seguido de un retorno de carro y un salto de línea. Este es el más complejo de los programas confeccionados para el sistema mínimo, por lo tanto, se encuentra descrito en dos diagramas de flujo separados, uno para la rutina principal, figura 5.8a y otro para la rutina de transmisión de datos, figura 5.8b. Si es de interés para el lector, en el apéndice C.4 se encuentra el código fuente.

En la rutina principal, figura 5.8a, inicialmente se inicializa el registro Contador (en 7 decimal), que se utiliza para realizar la misma rutina 7 veces, y el registro Carácter (en 65 decimal), el cual representa el carácter A del código ASCII. Luego se ejecuta un bucle 7 veces seguidas, utilizando para ello el registro Contador. El mismo se decrementa una unidad en cada repetición del bucle y al llegar a cero en la última repetición, el programa sigue con su rutina. Dentro del bucle, Carácter es asignado a otro registro denominado Dato¹, en este registro debe encontrarse el dato a ser transmitido por la rutina de transmisión, seguidamente se llama a esa rutina, enviando el primer carácter (letra A). Seguidamente se llama a una rutina de retardo, se incrementa Carácter (pasando al siguiente carácter ASCII) y se vuelve a transmitir. Luego de ejecutarse 7 veces este bucle, se han transmitido los caracteres “ABCDEFGH”.

A continuación, se sale del bucle y se envían los caracteres retorno de carro (D hexadecimal en codificación ASCII) y salto de línea (A hexadecimal en codificación ASCII). Esto concluye la rutina principal, la cual, al finalizar, vuelve a ejecutarse desde el comienzo.

En cuanto a la rutina de transmisión, figura 5.8b, la misma envía los datos de forma serial por protocolo UART utilizando para ello el pin Q del microprocesador como salida de transmisión. En el apéndice A (Protocolos de Comunicación Serial) se encuentran explicados los detalles relacionados con este protocolo.

La rutina de transmisión comienza pasando el pin Q de su estado de reposo ($Q = 1$) a estado bajo ($Q = 0$) por un periodo de 1 bit-time, este es el bit de inicio de la trama UART. Luego se inicializa un contador en 8 decimal, para realizar un bucle que se repite 8 veces, una por cada bit del dato de 8 bits. En este bucle el dato a enviar se corre a la derecha, cargando el LSB en el bit DF interno del microprocesador, en base al valor de este bit, se decide si poner la línea en estado alto ($Q = 1$) o bajo ($Q = 0$). Esto se repite 8 veces, de manera tal que se envían todos los bits del dato, del LSB al MSB. Finalmente se envían dos bits de parada ($Q = 1$) para terminar la transmisión del dato.

¹Esta asignación parece a primera vista redundante, pero la razón de ello es que la rutina de transmisión modifica el registro Dato, por lo tanto, se requiere que el dato a transmitir permanezca guardado en otro lado, en este caso en el registro Carácter.

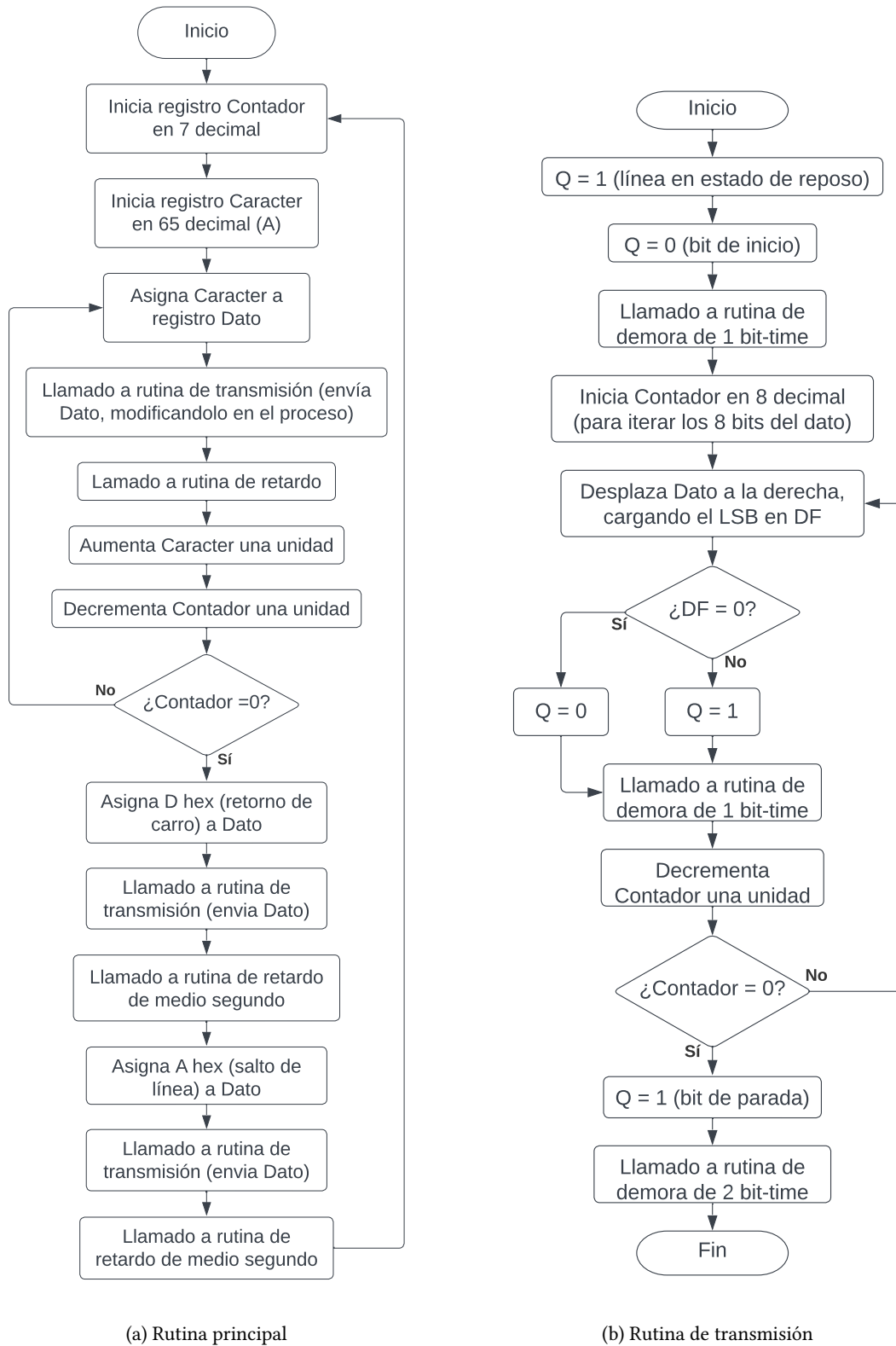


Figura 5.8: Programa 4

El programa 4 cuenta con muchas subrutinas, todas ellas fueron escritas con el método denominado “SEP technique”, el cual se encuentra descrito, junto a otros métodos, en la página 130 del libro Design Ideas Book for the CDP1802 COSMAC Microprocessor[23]. El método SEP para las subrutinas fue utilizado por no requerir RAM para su operación, algo necesario, dado que el sistema mínimo no posee RAM.

En el listado 5.1, se encuentra la rutina de retardo de un bit-time. En cada línea, como comentario, está indicado, para cada instrucción, el número de bytes, el número de ciclos máquina y la cantidad de veces que se ejecuta cada instrucción al llamarse la rutina. Esta información nos permite determinar el retardo ocasionado por la misma, necesario para poder calcular el valor que debe cargarse en R1 (línea 5 y 6) y así poder obtener el retardo necesario.

Listado 5.1: Rutina de retardo de 1 bit-time

1	;-----subrutina de ratardo de tranmision-----					
2	ORG	0005FH	; Num de bytes/ Nnum Ciclos Maq/Nnum de ejecuciones			
3	BACK	SEP TX	; 1-Byte	/ 2-Cliclos Maq	/	1 vez
4	DELAY		;			
5		LDI CONT1BT	; 2-Byte	/ 2-Cliclos Maq	/	1 vez
6		PLO R1	; 1-Byte	/ 2-Cliclos Maq	/	1 vez
7	TIMER		;			
8		DEC R1	; 1-Byte	/ 2-Cliclos Maq	/	CONT1BT veces
9		GLO R1	; 1-Byte	/ 2-Cliclos Maq	/	CONT1BT veces
10		BNZ TIMER	; 2-Byte	/ 2-Cliclos Maq	/	CONT1BT veces
11		BR BACK	; 2-Byte	/ 2-Cliclos Maq	/	1 vez

En base a esta información, en (5.1) se tiene la expresión para el retardo; teniendo en cuenta que: todas las instrucciones ejecutadas en la rutina tienen una duración de dos ciclos máquina y que un ciclo máquina equivale a ocho ciclos de reloj.

$$Retardo = 2 \text{ ciclosMaq} (4 + 3 \text{ CONT1BT}) \frac{8 \text{ ciclosReloj}}{1 \text{ cicloMaq}} T_{reloj} \quad (5.1)$$

Simplificando (5.1), se obtiene (5.2).

$$Retardo = 16 (4 + 3 \text{ CONT1BT}) T_{reloj} \quad (5.2)$$

Finalmente, a partir de (5.2), se despeja CONT1BT, obteniéndose (5.3).

$$\text{CONT1BT} = \frac{Retardo}{48 T_{reloj}} - \frac{4}{3} \quad (5.3)$$

En base a (5.3), y teniendo en cuenta que el sistema mínimo utiliza un cristal de 3,579 545 MHz ($T_{reloj} = (3,579\,545 \text{ MHz})^{-1} = 279,3651 \text{ ns}$), podemos calcular CONT1BT. Lo único faltante es la duración del retardo, el cual se obtiene simplemente haciendo la inversa del baud rate. Tomando un baudrate de 600 baudios, se obtiene un retardo de 0,001 666 667 s. Con estos datos, y la ecuación (5.3), es posible calcular el valor del contador para la rutina de retardo de 1 bit-time, como se muestra en (5.4); redondeando el resultado, debido a que se trata de un valor que debe cargarse a un registro de 8 bits.

$$CONT1BT = \frac{Retardo}{48 T_{reloj}} - \frac{4}{3} = \frac{0,001\,666\,667\,s}{48 \cdot 279,3651\,ns} - \frac{4}{3} = 122,96 \approx 123 \quad (5.4)$$

5.3. Simulaciones y resultados experimentales

Todos los programas fueron probados en el simulador Emma02 antes de ser implementados en el sistema mínimo, los resultados obtenidos fueron los deseados. Para el testeo se utilizó la computadora Membership Card, y la configuración exacta utilizada puede visualizarse en la figura 5.9.

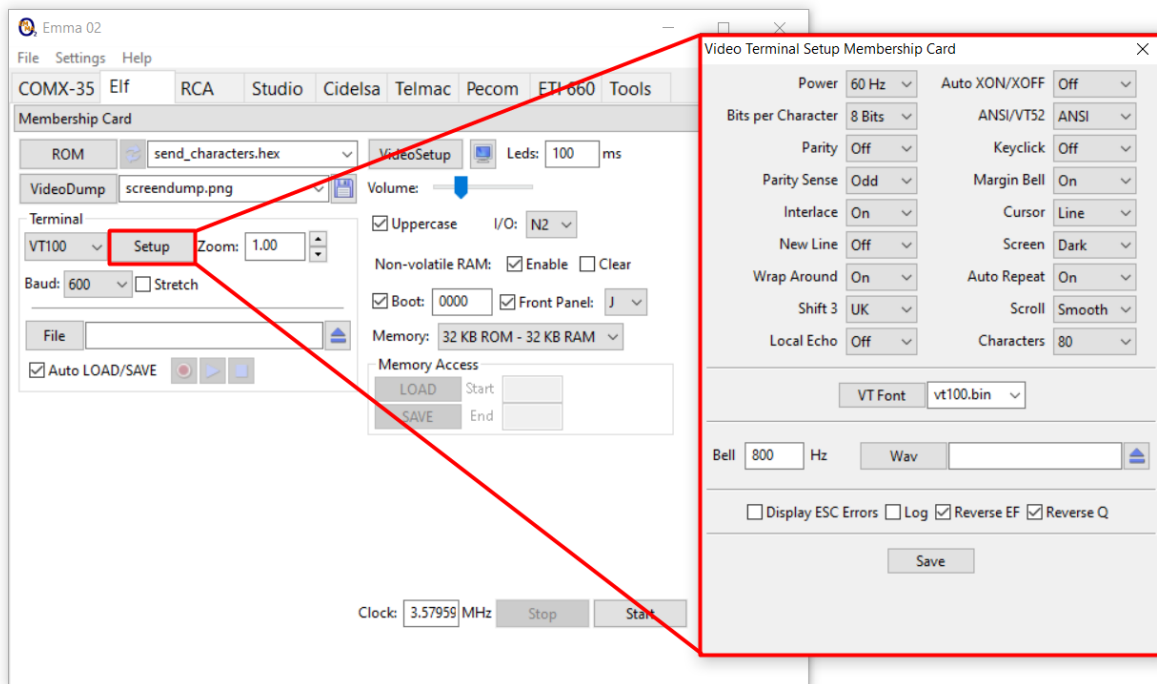


Figura 5.9: Configuración utilizada en el simulador Emma02

En la figura 5.10 puede observarse el resultado obtenido tanto en simulación como en el sistema mínimo construido. Los resultados obtenidos en ambos casos fueron acordes a lo esperado. La configuración en Tera Term fue de 600 baudios, 8 bits para tamaño de datos, sin bit de paridad y con dos bits de parada.

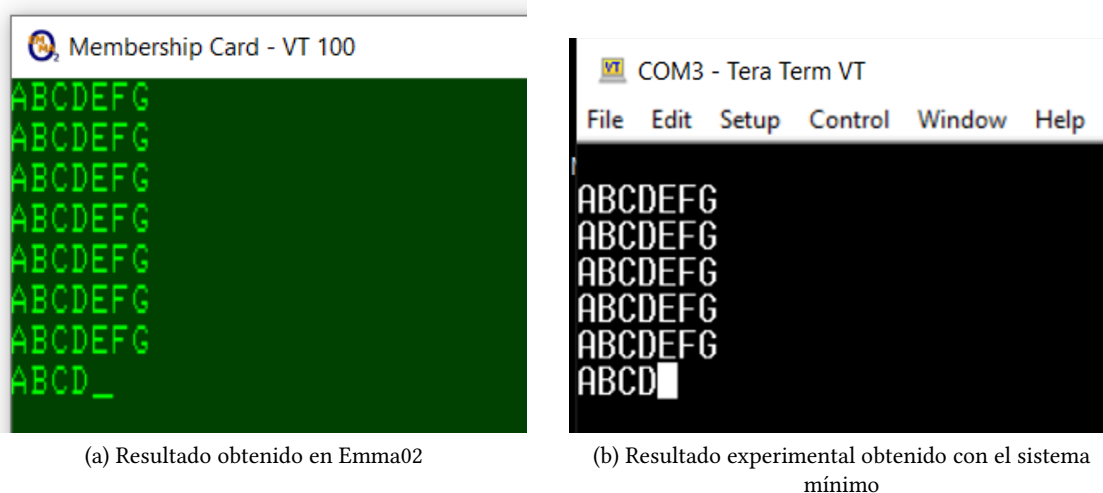


Figura 5.10: Resultados de la ejecución del programa 4

5.4. Programador

Para insertar los programas diseñados en la memoria EEPROM fue requerido el diseño de un circuito programador. El diseño adoptado se basa en un Arduino Nano. El software utilizado para el mismo no es de nuestra autoría, se encuentra disponible en la red [24]; aun así, se tuvieron que realizar modificaciones, con ayuda de programación en Python, para obtener un resultado satisfactorio, ya que luego de compilar un programa se disponía del mismo en formato Intel y este debía ser interpretado antes de pasarlo al Arduino que se encargaría de grabar la memoria.

Finalmente, el resultado obtenido con el programador sirvió para realizar unos ensayos y grabar memoria, sin embargo, debido a su dificultad de operación se terminó descartando y se reemplazó por el uso del programador presentado en la sección 3.7. Aun así, por haberse construido, se presenta el esquemático del circuito en la figura 5.11 y una fotografía del resultado obtenido en la figura 5.12.

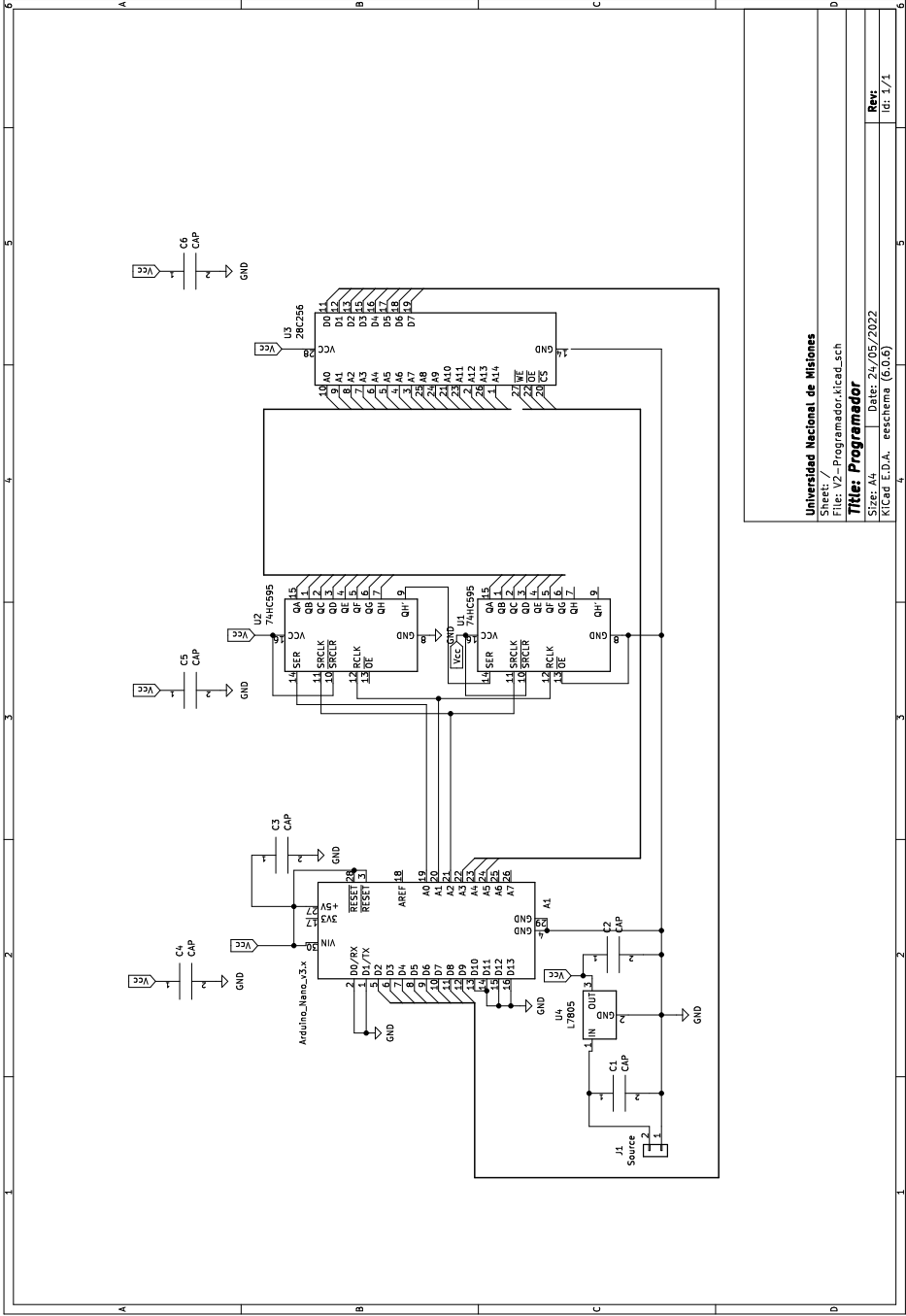
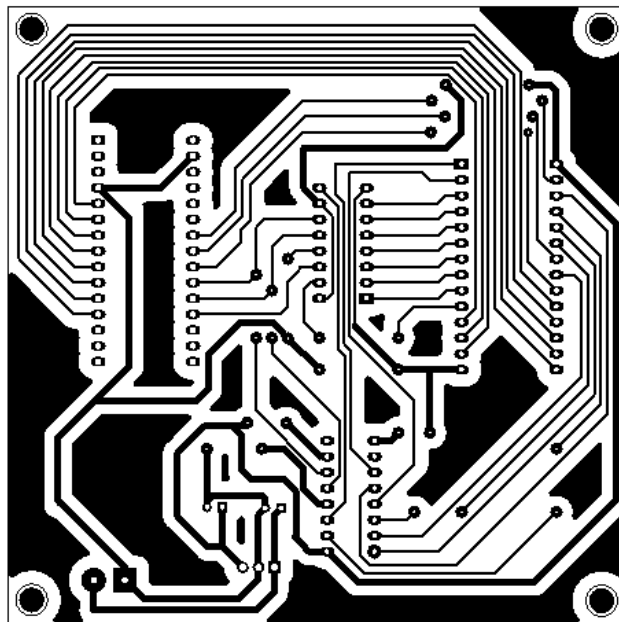
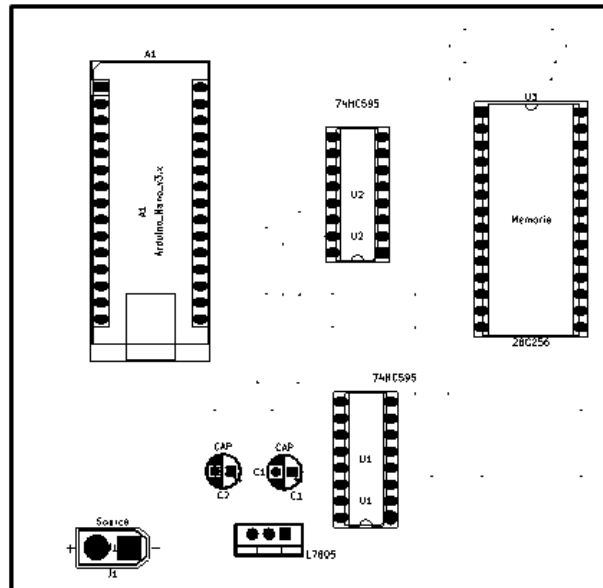


Figura 5.11: Esquema de programador



(a) Pista diseñada para Programador



(b) Máscara de componentes de Programador

Figura 5.12: Placa PCB diseñada para el programador de memoria

Sistema medio

El sistema medio se realizó con el objetivo de implementar una arquitectura más compleja que el sistema mínimo, permitiendo desarrollar programas del tipo dinámico en el sistema. Principalmente, se buscaba implementar programas que interactuaban con la memoria del sistema de forma dinámica, es decir, que utilizaban una memoria del tipo RAM, la cual se utiliza en la arquitectura del sistema medio. La diferencia principal con respecto al sistema mínimo es el uso de toda la memoria disponible en el mapa de memoria, permitiendo implementar códigos que requieren de mayor disponibilidad de memoria y por ende son más complejos.

El microprocesador CDP1802 cuenta con un direccionamiento de 16 bits, no obstante, el mismo utiliza solamente 8 líneas para el bus de direcciones. La solución presentada por el microprocesador para el direccionamiento es enviar primero los 8 bits más significativos de la dirección, luego los 8 bits menos significativos. Para evitar la pérdida de los 8 primeros bits se debe integrar un latch (74HC373) que almacene los primeros 8 bits del bus de direcciones, y el mismo microprocesador sincroniza al latch con una señal para el registro de los datos.

Es de esta manera que el sistema medio puede direccionar una mayor cantidad de memoria que en el sistema mínimo (256 x 8), utilizando un latch de 8 bits 74HC373[25] y aprovechando el mecanismo propuesto por el CDP1802 para aumentar la cantidad de líneas de dirección hasta 16 bits, logrando direccionar 64k x 8. Esto se observa en el diagrama de bloques de la Figura 6.1.

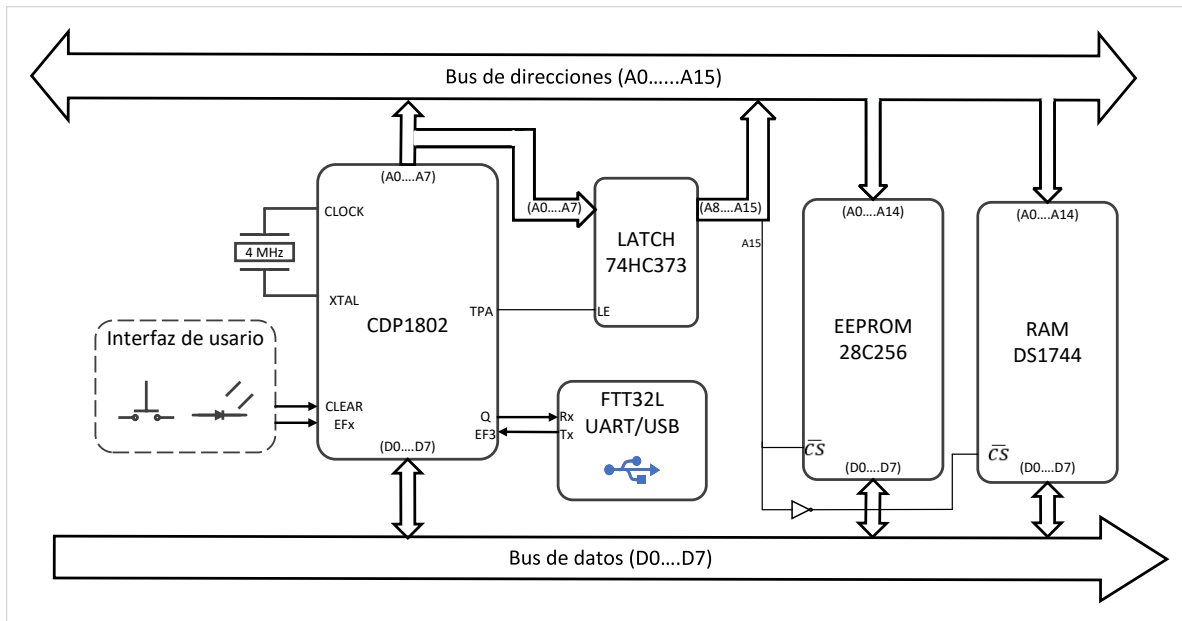


Figura 6.1: Diagrama de bloques del sistema medio

Esta incorporación de la memoria RAM es la principal diferencia con el sistema mínimo, lo cual se muestra en el mapa de memoria de la Figura 6.2, donde se tiene una memoria de programa EEPROM (32k x 8) y una memoria de datos RAM (32k x 8); que en total utilizan toda la capacidad de direccionamiento (64k x 8).

Para realizar la interacción con la terminal externa, se incluyó en el esquema del sistema medio un conversor de UART/USB que dota al sistema de la capacidad de interactuar con una computadora externa, lo cual se explicará mejor en la sección de programas. Los demás dispositivos utilizados son los mismos que en el sistema mínimo y son los requeridos para asegurar el funcionamiento del sistema de forma integral. El esquemático del sistema medio se encuentra en la Figura 6.3, el cual fue realizado con el software KiCad.

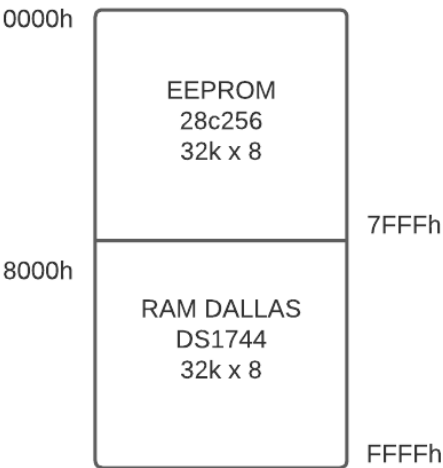


Figura 6.2: Mapa de memoria del sistema medio

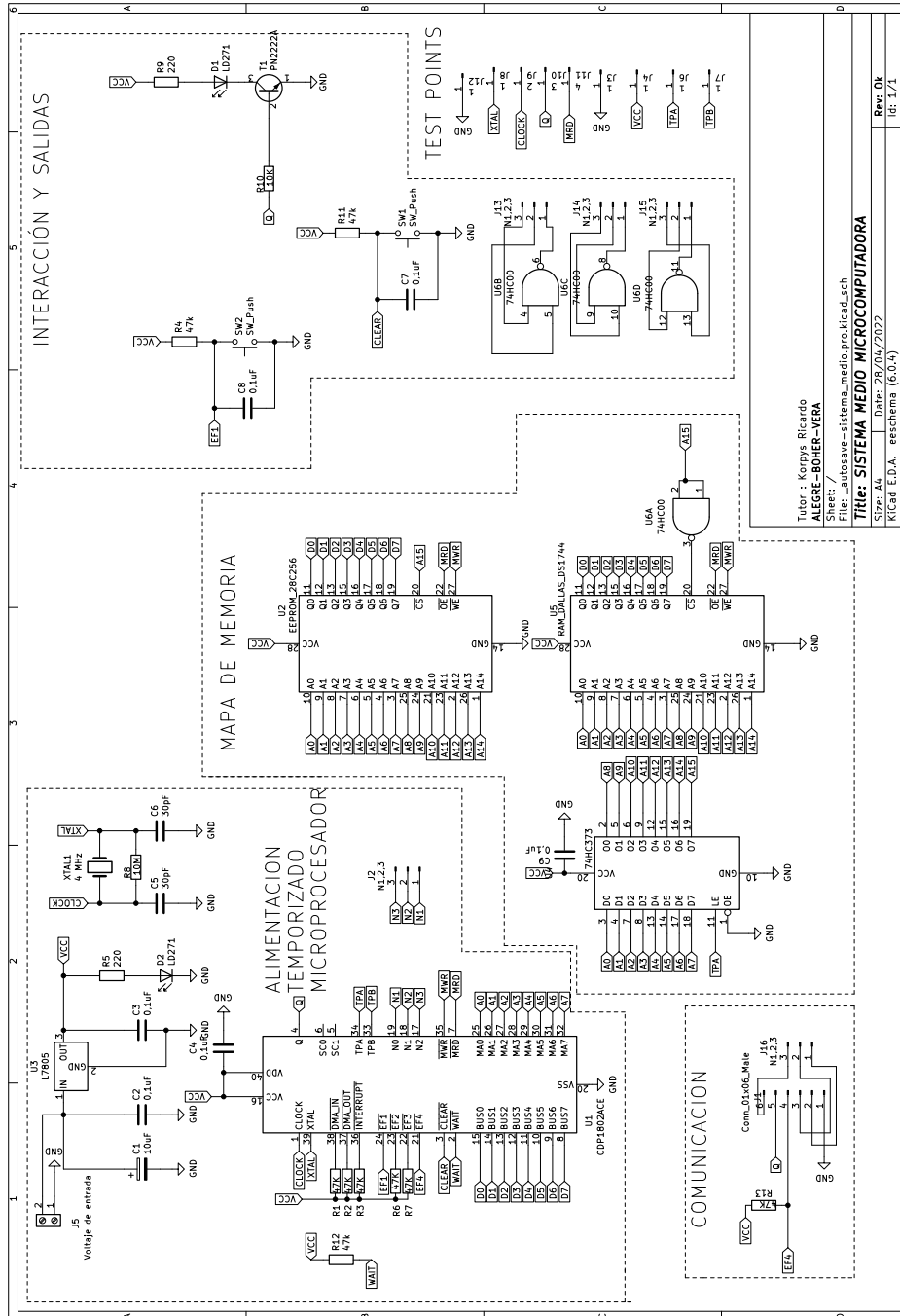
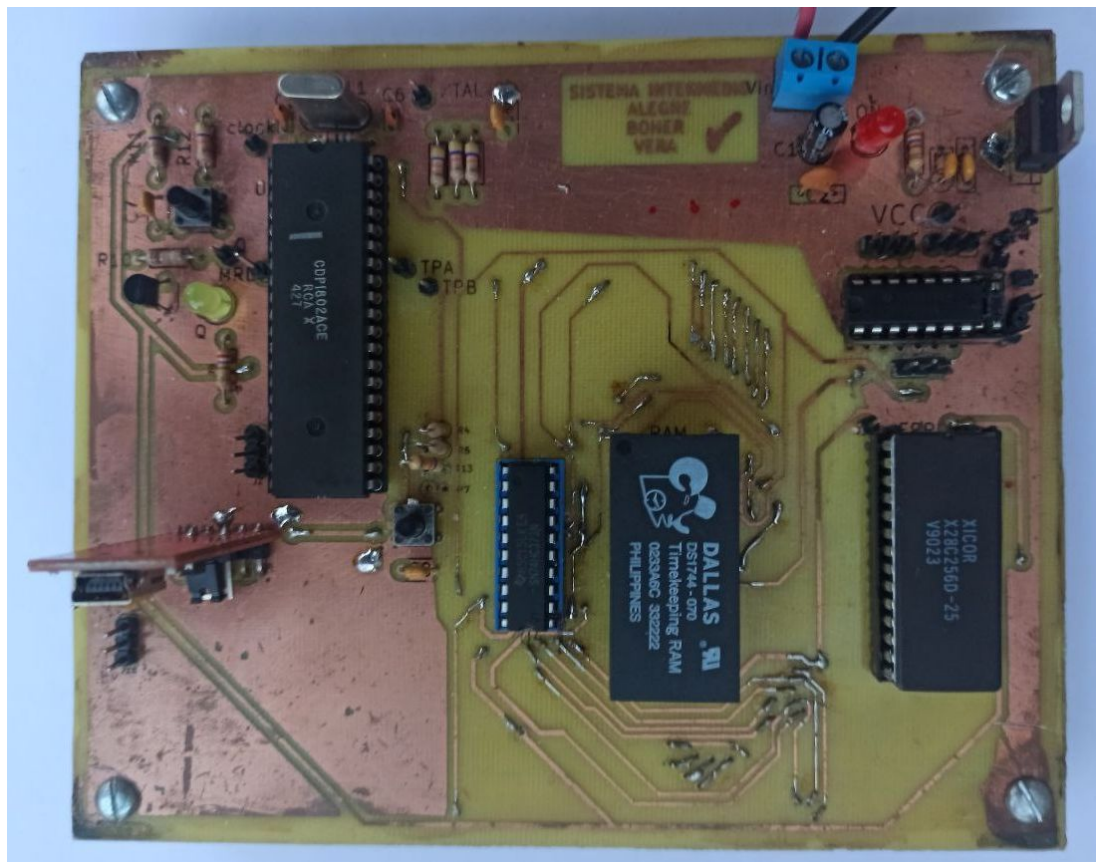


Figura 6.3: Esquemático del sistema medio



Fotografia 6.1: Sistema medio construido

6.1. Lista de componentes

En la tabla 6.1 se encuentra la lista de materiales para la construcción del sistema medio.

Tabla 6.1: Lista de componentes del sistema medio

Cantidad	Etiqueta Identificador	Descripción	Fabricante	Número de parte
1	U1	Microprocesador	RCA	CDP1802ACE
1	U2	Memoria EEPROM 32k x 8	XICOR	X28C256D-25
1	U3	Regulador de tensión 5 V	ST Microelectronics	L7805CV
1	U4	Latch de 8 bits	Texas Instruments	SN74HC373N
1	U5	Time keeping RAM 32K x8	Dallas	DS1744-070
1	U6	Quadruple 2-Input NAND Gates	Texas Instruments	SN74HC00N
1	T1	Transistor BJT NPN	ST Microelectronics	PN2222A
1	XTAL1	Cristal 4 MHz	CQ Electronics	Varios
2	SW1, SW2	Pulsador 6mm x 4,3mm	Varios	Varios
9	R1, R2, R3, R4, R6, R7, R11, R12, R13	Resistor 47 k Ω 1/8 W	Varios	Varios
2	R5, R9	Resistor 220 Ω 1/4 W	Varios	Varios
1	R8	Resistor 10 M Ω 1/8 W	Varios	Varios
1	R10	Resistor 10 k Ω 1/8 W	Varios	Varios
1	D1	LED Rojo 5mm	Varios	Varios
1	D2	LED Verde 5mm	Varios	Varios
1	C1	Capacitor electrolítico 10 μ F 25 V	Varios	Varios
6	C2, C3, C4, C7, C8, C9	Capacitor cerámico 0,10 μ F 50 V	Varios	Varios
2	C5, C6	Capacitor cerámico 30 pF 50 V	Varios	Varios
1	J1	1x6 Pines Conectores Macho 2,54 mm	Varios	Varios
5	J2, J13, J14, J15, J16	1x3 Pines Conectores Macho 2,54 mm	Varios	Varios
9	J3, J4, J6, J7, J8, J9, J10, J11, J12	Pin conector macho (1 <i>Male Header Pin</i>)	Varios	Varios
1	J5	Bornera 2 pines 5,08 mm	Varios	Varios
1	No aplica	Módulo conversor USB-Serial	Future Technology Devices International	FT232RL

6.2. Programas del sistema medio

Los programas realizados para el sistema mínimo, expuestos en la sección 5.2, son compatibles con el sistema medio, salvo el programa número 4, que requiere un ajuste en el contador utilizado para la rutina de retardo, debido a que el sistema medio utiliza un cristal de 4 MHz ($T_{reloj} = 250$ ns), distinto al utilizado en el sistema mínimo. Para su cálculo, utilizamos la ecuación (5.3), expuesta en la sección 5.2.4, y considerando, al igual que en el sistema mínimo, 600 baudios (Retardo = 0,001 666 667 s).

$$CONT1BT = \frac{Retardo}{48 T_{reloj}} - \frac{4}{3} = \frac{0,001\ 666\ 667\ s}{48 \cdot 250\ ns} - \frac{4}{3} = 137,56 \approx 138 \quad (6.1)$$

A continuación, se exponen otros programas implementados en el sistema medio.

6.2.1. Intérprete Tiny Basic

En la página oficial del Membership Card[7] puede encontrarse un intérprete Basic bajo el nombre de Tiny BASIC. El mismo tiene documentación y el código fuente, lo cual resulta de gran ayuda. Sin embargo, para su correcto funcionamiento en el sistema medio, es necesario realizar una serie de modificaciones en el código fuente.

La primera de estas modificaciones es en el archivo 1802reg.asm, el cual contiene definido el nombre de los registros de trabajo. Sin embargo, faltaban algunos registros y fue necesario agregarlos. Para que el código pueda ser ensamblado sin errores, es necesario que este archivo tenga todas las definiciones que se encuentran en el código del listado 6.1.

Listado 6.1: Archivo 1802reg.asm con definición de etiquetas para los registros de trabajo

```

1 ;
2 ; $Id: 1802reg.asm,v 1.4 2018/02/22 20:56:33 loren Exp $
3 ;
4 ;HRJ A18 wants register names defined...
5 ;
6 R0      EQU 0
7 R1      EQU 1
8 R2      EQU 2
9 R3      EQU 3
10 R4     EQU 4
11 R5     EQU 5
12 R6     EQU 6
13 R7     EQU 7
14 R8     EQU 8
15 R9     EQU 9
16 R10    EQU 10
17 R11    EQU 11
18 R12    EQU 12
19 R13    EQU 13
20 R14    EQU 14
21 R15    EQU 15
22
23 r0      EQU 0
24 r1      EQU 1
25 r2      EQU 2
26 r3      EQU 3
27 r4      EQU 4
28 r5      EQU 5
29 r6      EQU 6
30 r7      EQU 7
31 r8      EQU 8
32 r9      EQU 9
33 ra      EQU 10
34 rb      EQU 11
35 rc      EQU 12
36 rd      EQU 13
37 re      EQU 14
38 rf      EQU 15
39
40 RA      EQU 10
41 RB      EQU 11
42 RC      EQU 12
43 RD      EQU 13
44 RE      EQU 14
45 RF      EQU 15

```

Las siguientes modificaciones se realizaron sobre el código fuente del intérprete Tiny BASIC, más específicamente, el archivo `tb0_test.asm`. Este código es muy extenso para ser incluido en este informe, sin embargo, se incluyen segmentos pequeños del código, donde pueden visualizarse las modificaciones

realizadas.

La segunda modificación fue realizada en la línea 25, que puede verse en el listado 6.2, donde la etiqueta QHI debe ser definida como 0, esto es necesario para que la línea de transmisión serial a través del pin Q, cuando esté en reposo, sin transmitir datos, permanezca en estado alto, como lo establece el protocolo UART. Y la última modificación realizada, esta vez en la línea 28 del listado 6.2, fue definir la etiqueta AUTORUN como 0; si esta etiqueta permanece definida como 1, al iniciar el intérprete, se ejecuta un código predefinido, lo cual en ocasiones resulta molesto, por lo que se decidió desactivar esta opción.

Listado 6.2: Segmento del código fuente `tb0_test.asm` con modificación en línea 25 y 28

```

1 ;
2 ; $Id: tb0.asm,v 1.9.2.4.2.7 2018/04/19 23:16:50 loren Exp $
3 ;
4 ; TB-MON.ASM                                PAGE 1
5 ; INCL "1802reg.asm" ;R0-RF defined HRJ
6 ;
7 ;
8 ; AVOCET XASM18 MACRO-ASSEMBLER SYNONYMS
9 ;
10 ; CALL ADDRESS IS A SYNONYM FOR SEP R4 (D4)
11 ; DW ADDR
12 ; EXIT IS A SYNONYM FOR SEP R5 (D5)
13 ; POP IS A SYNONYM FOR LDXA (72)
14 ; PUSH IS A SYNONYM FOR STXD (73)
15 ;
16 ; I had to change these pseudo-ops to actual code HRJ
17 ;
18 ; L. Christensen 22 Feb 2018, (Membership Card Rev. J)
19 ; sense of EF3 and Q for TTL is +5V idle on P4 pins 4 and 5;
20 ;
21 ; Rx - Pin 4, /EF3, MARK High, (no inversion, 1K resistor R2),
22 ; Tx - Pin 5, Q, MARK Low, (inverted by Q7).
23 ;
24 EFHI EQU 0 ; 0=active low EF for serial, (0 = +5V MARK, 0V SPACE).
25 QHI EQU 0 ; 0=active low Q for serial, (1 = 0V MARK, +5V SPACE).
26 ;
27 ; L. Christensen 10 April 2018,
28 AUTORUN EQU 0 ; Define a Tiny BASIC Autorun program at $0D00.
29 ;
30 ; Herb Johnson July 20 2018, EFHI=0 and QHI=1 for Rev J and USB/TTL FTDI dongle.
31 ; Autorun program starts on reset after delay, ends after short loop.
32 ; 1200 baud fixed baud rate at 2Mhz, 2400 baud at 4Mhz, no start CR needed.

```

Estas son todas las modificaciones realizadas. Sin embargo, existen partes de interés en el código que vale la pena mencionar, a pesar de que no fueron modificadas. La primera de ellas se encuentra en la línea 91 del código, listado 6.3, donde se encuentra definida la etiqueta PAGE, que define la posición de la memoria RAM en el mapa de memoria, empezando en este caso a partir de 8000 hexadecimal. Para el funcionamiento en el sistema medio no es necesario modificar esta etiqueta, sin embargo, en otras arquitecturas sí puede ser necesario.

Listado 6.3: Segmento del código fuente `tb0_test.asm`, modificación de posición de memoria RAM en el mapa de memoria

```

89 ; SCRATCHPAD MEMORY ALLOCATION - OFFSET ADDED TO PAGE ADDRESS
90 ;
91 PAGE: EQU 8000H ; WORKSPACE BEGINNING
92 KEYV: EQU PAGE+06H ; KEY-INPUT ROUTINE VECTOR
93 TYPEV: EQU PAGE+09H ; TYPE-OUTPUT ROUTINE VECTOR
94 BREAKV: EQU PAGE+0CH ; BREAK-DETECTION ROUTINE VECTOR
95 ; EQU +0FH ; UNUSED
96 BS: EQU +13H ; BACKSPACE CHARACTER

```

Finalmente, en la línea 2194 del código, listado 6.4, puede modificarse la velocidad de transmisión (baudrate). Por defecto es de 2400 baudios para el caso del sistema medio, que utiliza un cristal de 4 MHz.

Listado 6.4: Segmento del código fuente `tb0_test.asm`, definición del baudrate utilizado con cristal de 4 MHz

```

2187 ; SET DEFAULT BAUD RATE AND HALF/FULL DUPLEX. VALUES SHOWN
2188 ; ARE FOR A 2 MHZ CLOCK. NUMBERS SHOWN FOR FULL DUPLEX:
2189 ; ADD 1 FOR HALF DUPLEX.
2190 ; 75 BAUD = 0A2H
2191 ; 110 BAUD = 06CH
2192 ; LDI 26H ; 300 BAUD = 026H, Loren: original default.
2193 ; 600 BAUD = 012H
2194 LDI 08H ; 1200 BAUD = 008H
2195 PHI RE
2196 GHI R0 ; INITIALIZE DELAY SUBROUTINE PC
2197 PHI RC
2198 LDI LOW DELAY1
2199 PLO RC
2200 LDI 0DH ; SEND <CR> TO BASIC
2201 SEP R5 ; < TO EXECUTE "RUN" COMMAND

```

6.2.2. Sistema operativo ElfOS sin disco

En la página oficial del Membership Card[7] puede encontrarse una versión del sistema operativo ElfOS sin disco (en la página el término en inglés es “Diskless Elf/OS ROM”). Desafortunadamente, el código fuente de este programa no se encuentra disponible, solo un archivo `.hex`. Esto significa que no es posible modificar el programa, pero puede ser grabado en una memoria y utilizado en el sistema medio.

Este sistema operativo inicia presentando un menú donde puede seleccionarse una serie de programas, en la figura 6.4 pueden visualizarse todos los programas disponibles.



Figura 6.4: Menú del sistema operativo ElfOS sin disco

6.3. Simulación Emma02

6.3.1. Intérprete Tiny Basic

El intérprete Tiny Basic fue ejecutado correctamente en el simulador Emma02, utilizando el Membership Card, en la figura 6.5 puede apreciarse la configuración utilizada. Algo a ser destacado es que para la transmisión serial de datos, la configuración utilizada en el simulador es de 8 bits para el dato y sin bit de paridad, sin embargo, utilizando el sistema medio construido, se encontró que la configuración debe ser distinta; esta configuración se encuentra detallada en la sección 6.4.1.

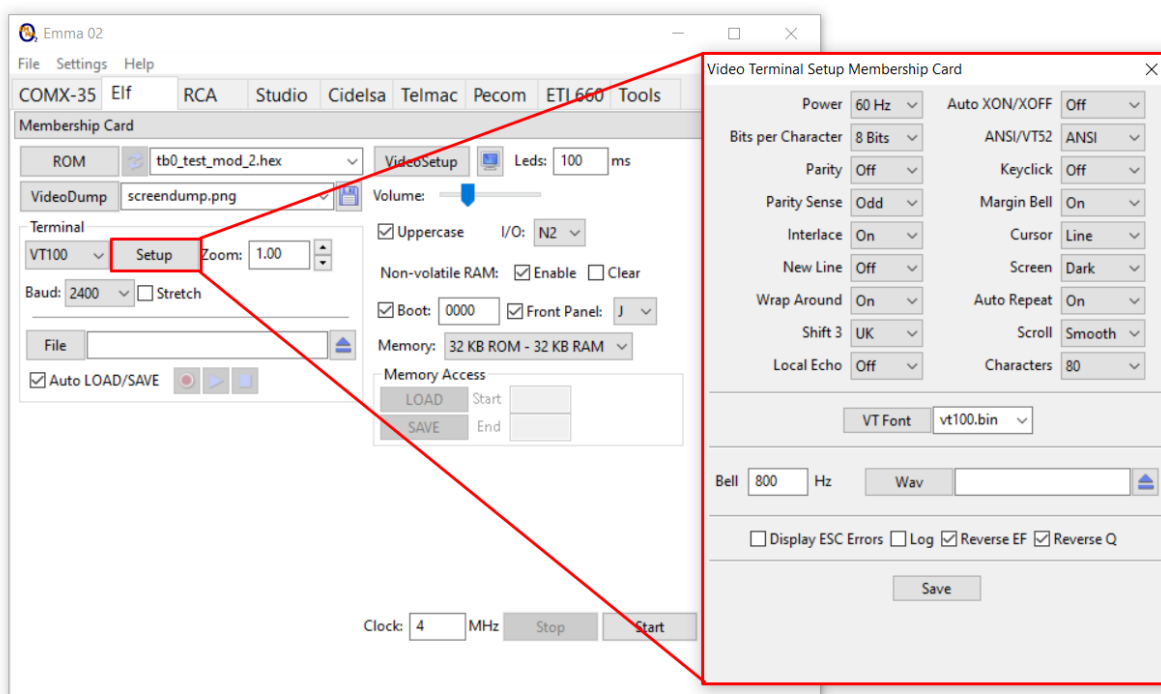


Figura 6.5: Configuración utilizada en el simulador Emma02 para ejecutar el intérprete Tiny Basic y el ElfOS sin disco

6.3.2. Sistema operativo ElfOS sin disco

El sistema operativo ElfOS sin disco también fue ejecutado correctamente en el simulador Emma02, de nuevo utilizando el Membership Card con la misma configuración expuesta en la figura 6.5.

6.4. Resultados experimentales

6.4.1. Intérprete Tiny Basic

El intérprete Basic fue probado en el sistema medio, habiendo obtenido los mismos resultados que con el simulador Emma02. Sin embargo, para poder funcionar correctamente, fue necesario utilizar una configuración distinta para la comunicación serial. A continuación, se describe la misma.

Para la comunicación serial se utilizó una velocidad de 2400 baudios, 7 bits para los datos, bit de paridad impar y dos bits de stop, Flow Control en none, y transmit delay nulo.

6.4.2. Sistema operativo ElfOS sin disco

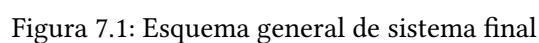
El sistema operativo ElfOS sin disco pudo ejecutarse correctamente en el sistema medio. La configuración para la comunicación serial fue la siguiente: Velocidad de 2400 baudios, 8 bits para los datos, sin bit de paridad y dos bits de stop, Flow Control en none, y transmit delay nulo.

Sistema Final

El sistema final es la culminación de los sistemas desarrollados en este proyecto, el cual parte del planteado para el sistema medio agregando funcionalidades extras al sistema que se detallarán más adelante. El mismo se encuentra planteado de manera que presente retrocompatibilidad, es decir, que todo software diseñado tanto para el sistema mínimo como para el sistema medio, debe funcionar en el sistema final. En comparación al sistema medio, se presentan la integración de la fuente de alimentación al sistema, el uso de memorias seriales, sistema de conversión AD, una interfaz de entrada salida expandida y un sistema de comunicación SPI. El circuito del sistema se encuentra diseñado, se presenta el esquemático, lista de componentes y diseño de placa PCB. No obstante, al momento de la escritura de este informe, el mismo no ha sido construido.

7.1. Esquemáticos

A modo de simplificar el análisis y presentación del sistema, el esquemático del mismo se presenta de manera modular, comenzando con el esquema generalizado (Ver figura 7.1) en el cual se pueden ver todas las partes que componen al sistema, sin adentrarse en un funcionamiento específico, para luego en las siguientes secciones pasar a especificar su funcionamiento. Principalmente en este esquema podemos observar la interconexión entre el microprocesador y los dispositivos o periféricos asociados.



7.1.1. Fuente de alimentación

La fuente de alimentación utilizada es lineal, basada en un transformador de tensión con punto medio. Dispone de un filtro de línea lo que reduce la distorsión armónica introducida en la red. Luego del proceso de rectificación y filtrado, se colocan etapas de regulación de tensión, para así poder alimentar los diferentes elementos del circuito que lo requieran.

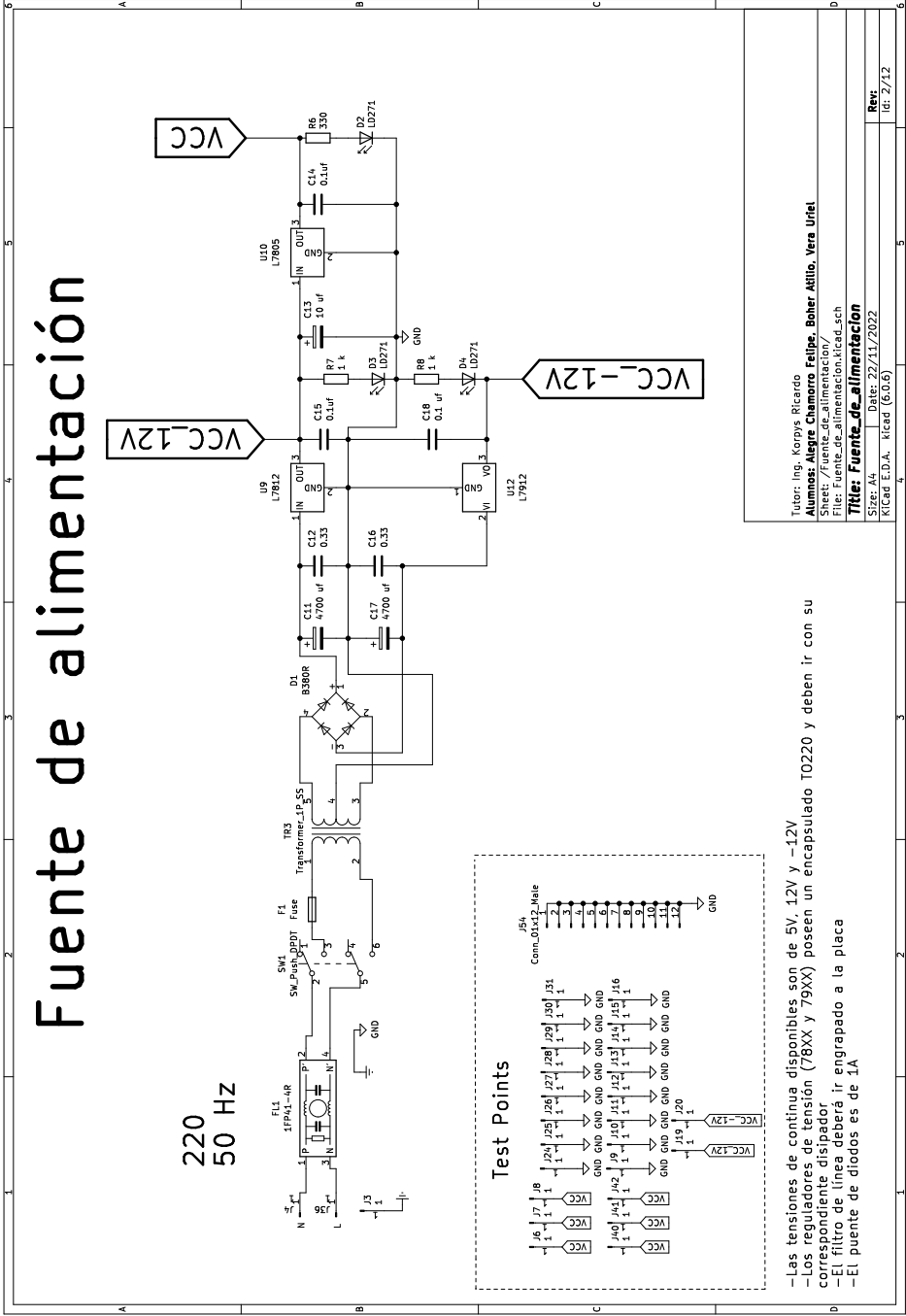


Figura 7.2: Esquema de fuente de alimentación de sistema final

7.1.2. Mapa de memoria

El planteo establecido para el mapa de memoria consta de dos modos de operación, un modo de inicio y un modo de ejecución o RUN. El sistema fue pensado de esta manera para tener disponible más memoria RAM en ejecución.

El modo inicio consiste en la memoria EEPROM 28C256 en la primera mitad del mapa de memoria (0000h a 7FFFh) y en la segunda mitad una memoria RAM en dos tramos de 8000h a F7FFh y luego continúa en FC00h hasta FFF8h, se interrumpe de esta manera la RAM para simplificar el circuito de codificador el cual se presenta más adelante (Sección 7.1.6). La sección del mapa de memoria de F800h hasta FBFFh es destinada a la comunicación SPI. Y finalmente, en la zona superior del mapa de memoria (FFF8h a FFFFh), la memoria RAM DALLAS 1744 posee un apartado para un reloj de tiempo real.

El modo de ejecución es un modo en el cual se cambia la primera mitad del mapa de memoria (0000h a 7FFFh). El proceso consiste en cambiar la memoria EEPROM del modo de inicio por una memoria RAM. Este proceso debe ser ejecutado por software y para ello en el modo de inicio se debe copiar el contenido de la memoria EEPROM en la RAM1, luego el sistema seguirá funcionando con la memoria de programa en la RAM1 y en la primera mitad del mapa de memoria se tendrá disponible la memoria RAM2.

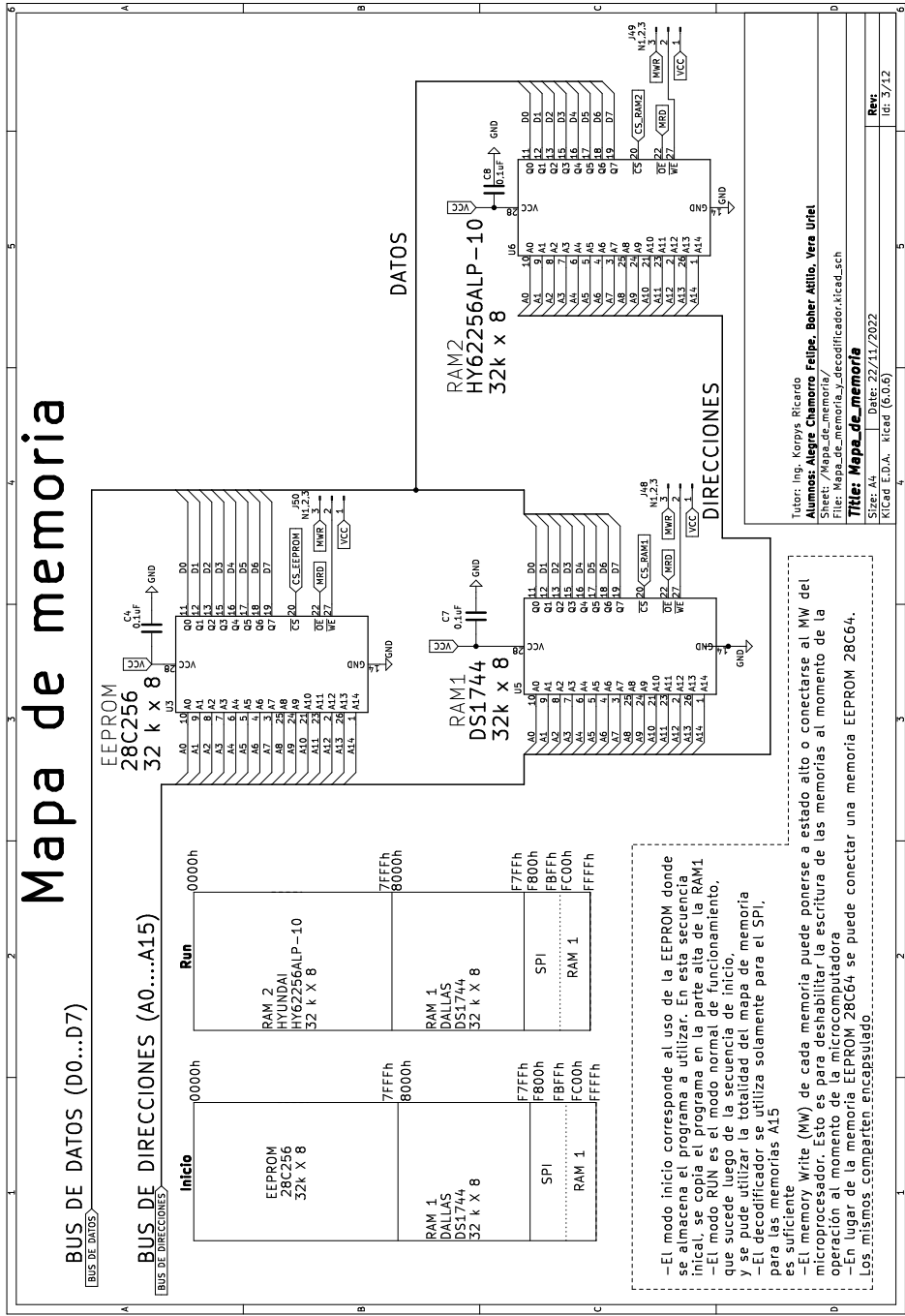
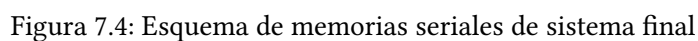


Figura 7.3: Esquema de Mapa de memoria de sistema final

7.1.3. Memorias seriales

El trabajo con memorias seriales fue planteado para diferentes tecnologías de memorias, específicamente del tipo EEPROM, FRAM y SDcard. Para la comunicación I2C se prevé el uso de un sistema de comunicación por software utilizando la interfaz de entrada salida presentada en la sección 7.1.10 e interactuando con el acondicionamiento presentado en 7.1.11, mientras que para la comunicación SPI, se diseñó un sistema por hardware presentado en la sección 7.1.8. En el esquema de la figura 7.4 se observan las etiquetas de interconexión con el resto del circuito.



7.1.4. Conversor analógico digital

El sistema de conversión analógico-digital se basa en el MAX127 [26]. Este conversor de 12 bits es configurable, lo que permite seleccionar el rango de tensiones a medir por cada canal y se realiza mediante comunicación I2C. Los rangos de medición pueden ser:

- 0 V a 5 V
- -5 V a 5 V
- 0 V a 10 V
- -10 V a 10 V

Para el acondicionamiento de señal se utilizan amplificadores operacionales OPA4277 [27]. Los amplificadores operacionales se configuraron como conversores de corriente a tensión, de 4-20 mA a 0-10 V. Se concibió el diseño de esta manera, ya que en caso de que se requiera medir tensiones, el circuito puede ser modificado para que los amplificadores operacionales funcionen como seguidores de tensión. Basta con modificar el rango de medición por software para que la lectura refleje correctamente el valor a la entrada.

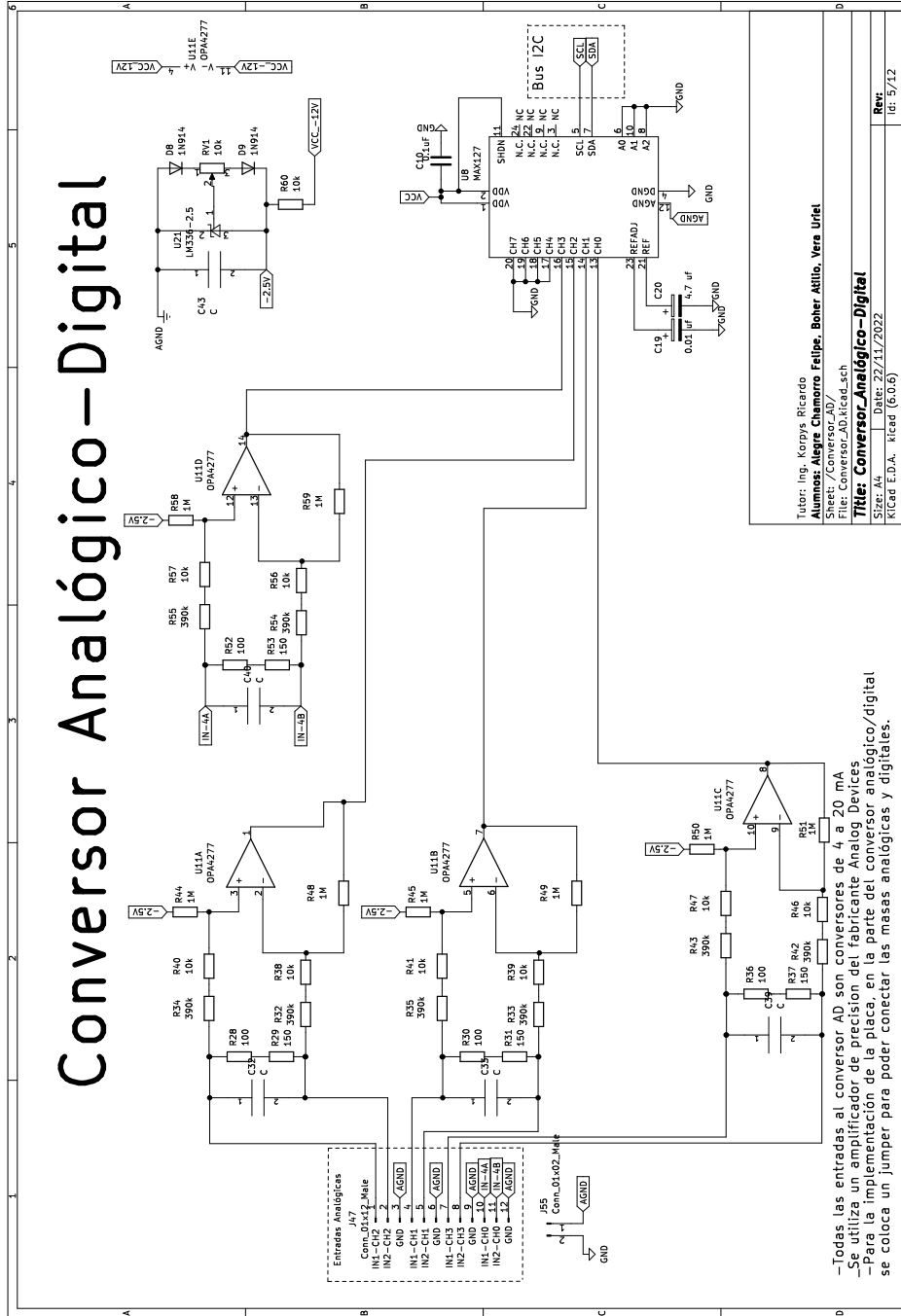


Figura 7.5: Esquema de conversor analógico digital de sistema final

7.1.5. Microprocesador

En el esquema del microprocesador se observa su interacción con el resto del circuito. Es interesante destacar el latch 74HC573 [28] el cual se encarga de transmitir los bits más significativos del bus de direcciones (A8-A15), esta configuración de microprocesador/latch para el direccionamiento es propia de la arquitectura propuesta por el CDP1802. Sin embargo, se utiliza un circuito integrado distinto al del sistema medio ya que su distribución de pines permite simplificar el diseño de la placa PCB. En el esquema también se puede observar una interfaz de usuario compuesta por pulsadores para señal de entrada y LEDs para señal de salida. El circuito oscilador que se presenta en el esquema está construido a partir de un cristal de 4 MHz. Es importante destacar que en la hoja de datos del microprocesador la frecuencia máxima para las condiciones de trabajo que establecemos es de 3,2 MHz, sin embargo, en la práctica es utilizado a mayor frecuencia y existen muchos casos prácticos que corroboran el funcionamiento con overclocking, como en la computadora Membership Card mencionada anteriormente [7].

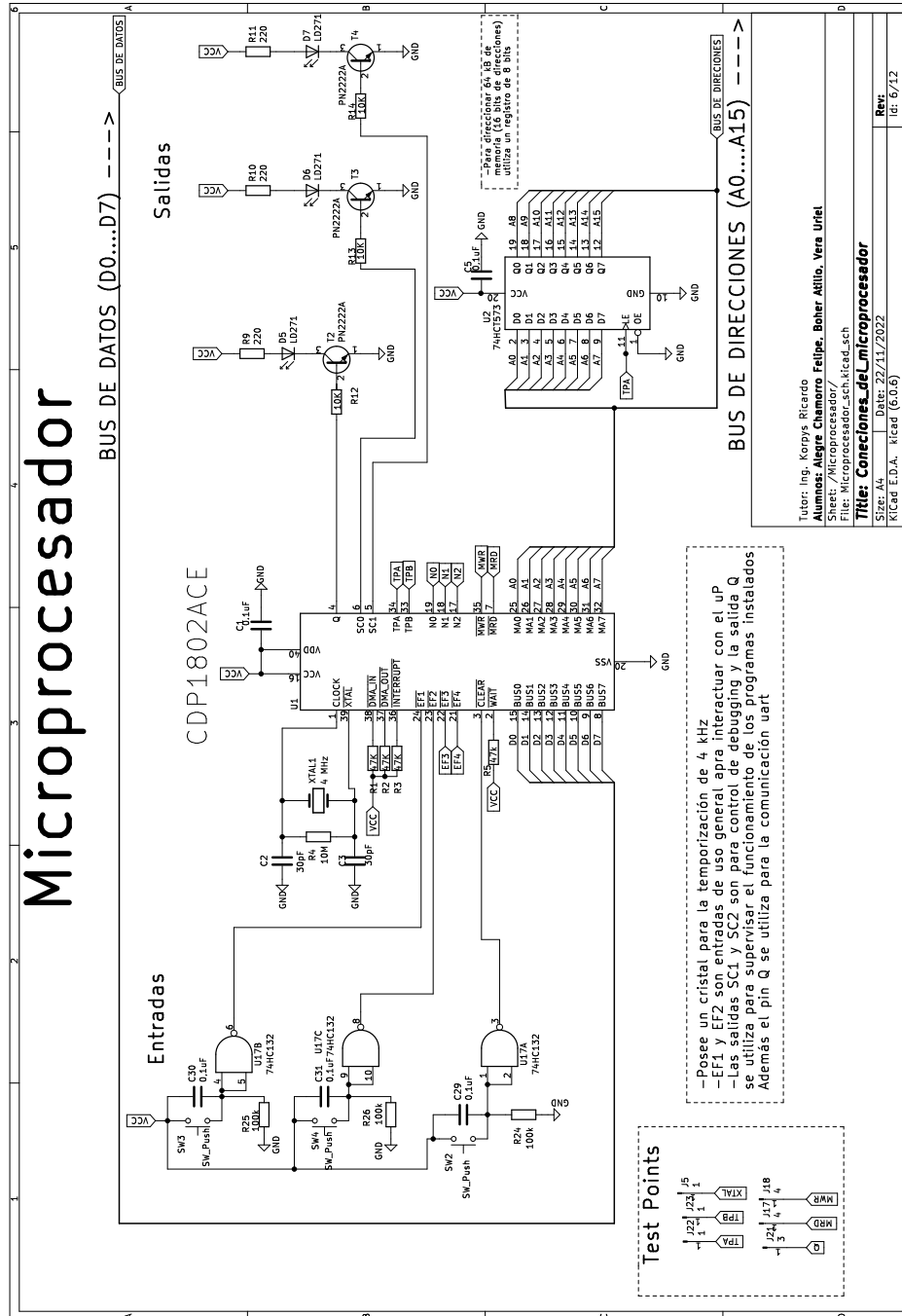
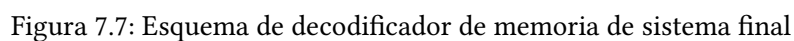


Figura 7.6: Esquema de conexión de microprocesador de sistema final

7.1.6. Decodificador

El circuito decodificador se encarga de activar la selección de los dispositivos según lo presente en el bus de direcciones. Para ello se utiliza un circuito lógico con compuertas y un decodificador de 3 líneas a 8 líneas 74HC138 [29].



7.1.7. Comunicación UART

La comunicación UART se basa principalmente en el software que la controla y se utiliza el pin Q del microprocesador para la transmisión de datos. Para lograr establecer comunicación con una terminal en una computadora personal se requiere la conexión por medio de USB, por lo que se conecta una interfaz FT232RL [22] que realiza la interpretación y retransmisión de los protocolos.

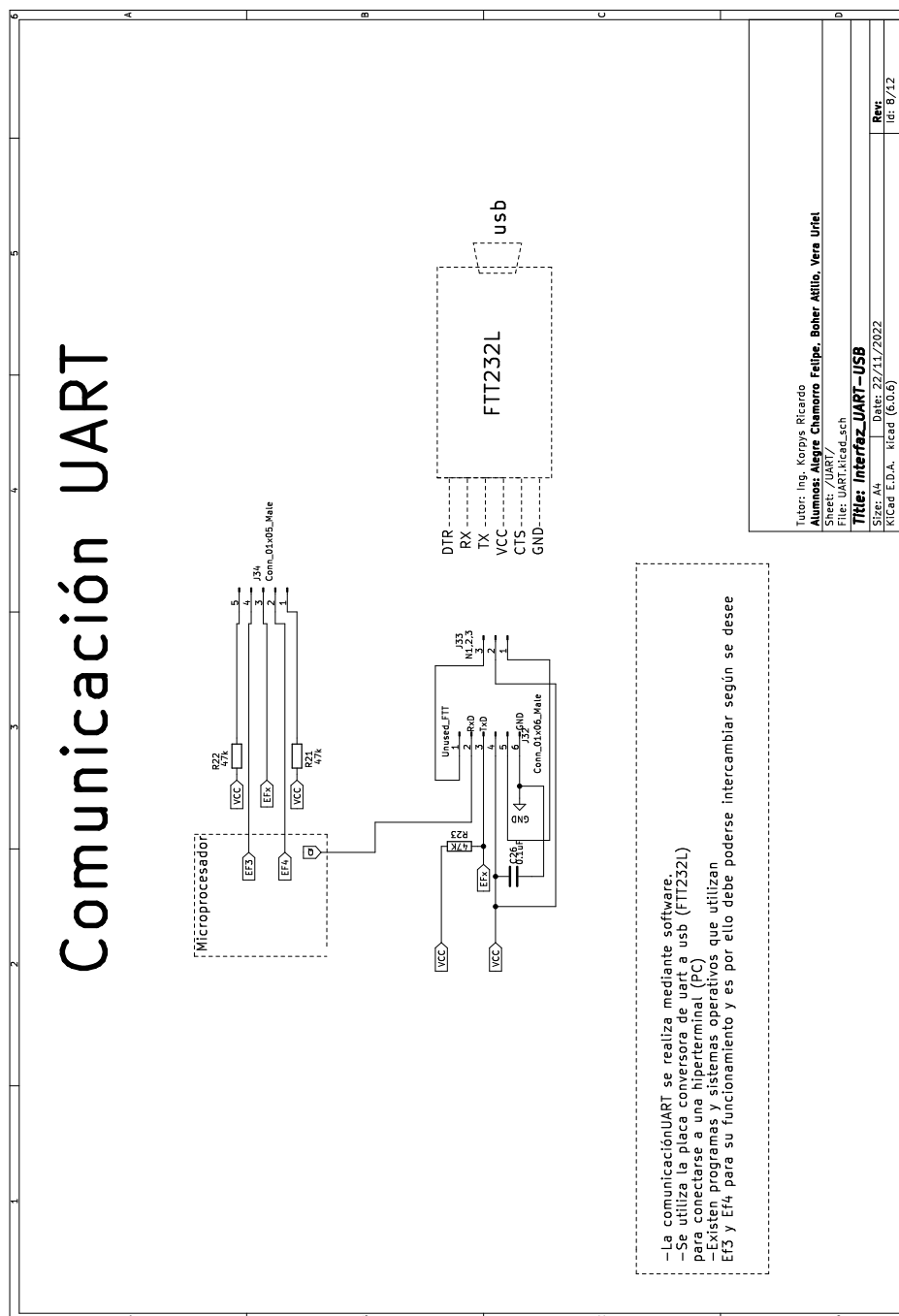


Figura 7.8: Esquema de comunicación UART de sistema final

7.1.8. Comunicación SPI

En el esquemático de la figura 7.9 se observa un circuito diseñado para realizar una comunicación basada en el protocolo SPI. Fue planteado con el objetivo de simplificar el procesamiento de datos en el microprocesador, realizándose un gran número de iteraciones en el circuito de manera transparente para el procesador.

En principio se debe considerar que el sistema planteado cuenta con una señal de reloj proveniente de un cristal oscilador compatible con CMOS. Este cristal, al igual que el conectado al microprocesador, es de 4 MHz. Considerando que las instrucciones en el procesador duran 8 ciclos de reloj, se tendrá una velocidad de procesamiento mayor en el hardware diseñado. Esto se representa más adelante en los diagramas temporales.

Básicamente el sistema presentado consiste en hacer trabajar 2 registros de desplazamiento (shift-register), uno para enviar datos y el otro para recibirlos.

Para el envío de datos se utiliza un registro de desplazamiento de entrada paralela y salida serie, el MC14014 [30] (también se encuentra como CD4014). Este dispositivo cuenta con un registro interno el cual guarda el dato disponible en el bus al recibir una señal en estado alto en el pin PL. La lectura de los puertos de entrada se realiza con la señal de clock, por lo que para el guardado de los datos en el bus se debe detectar un flanco ascendente en el pin CP. Una vez guardado el dato en el registro, se encuentra listo para transmitirlo, este proceso se acciona con una señal de estado bajo en el pin PL y se transmite de manera serial con las transiciones positivas de la señal de reloj.

Para la recepción de datos se utiliza un registro de desplazamiento de entrada serial y salida paralelo, el 74HC595 [31]. Este dispositivo cuenta con una entrada serial, por la cual toma el dato a almacenar, realizando una lectura en dicha entrada por cada flanco de reloj detectado. El dato presente en el registro de desplazamiento varía por cada ciclo de reloj, por lo que al transcurrir 8 ciclos se encuentra completo el registro, para guardar este dato y que no se vea alterado por el transcurso de los posteriores ciclos, se utiliza el pin RCLK que permite guardar el dato en un registro paralelo. Para que este dato sea presentado en la salida paralelo, se pone en estado bajo el pin de output enable $\overline{OE}$.

La señal de clock utilizada para los integrados mencionados anteriormente se genera a partir del circuito establecido entre el cristal oscilador, el flip flop D 74HC74, el contador 74HC161 y el flip flop JK 74HC73 [32], [33], [34]. La topología presentada en el esquema detecta el inicio del ciclo al recibir la señal proveniente de PC7, poniendo en estado alto el pin 4 del 74HC74, la señal generada por Q inicializa el contador. En el contador se toma la señal Q0 como señal de reloj, la cual tendrá una frecuencia de 2 MHz y luego cuando el contador se desborda la salida TC se pone en alto, esta señal es resguardada por el flip flop 74HC73 y utilizada como señal de reinicio del flip flop 74HC74. De esta manera se genera una señal oscilante de 2 MHz a la salida pero que solo tiene 8 pulsos, lo necesario para realizar el envío de un byte.

Para el control del sistema se desarrolló un algoritmo simple el cual se presenta en el listado 7.1. El funcionamiento general de este sistema se puede desglosar en tres partes. El proceso en el cual se guarda el dato a transmitir, presentado en la figura 7.10; la transmisión, presentada en la figura 7.11, y la lectura del dato recibido, presentada en la figura 7.12. El código assembler presentado en los gráficos se corresponde con el algoritmo presentado en el listado 7.1.

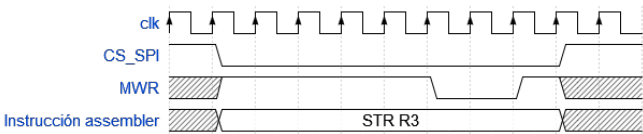


Figura 7.10: Proceso de guardado de dato a transmitir

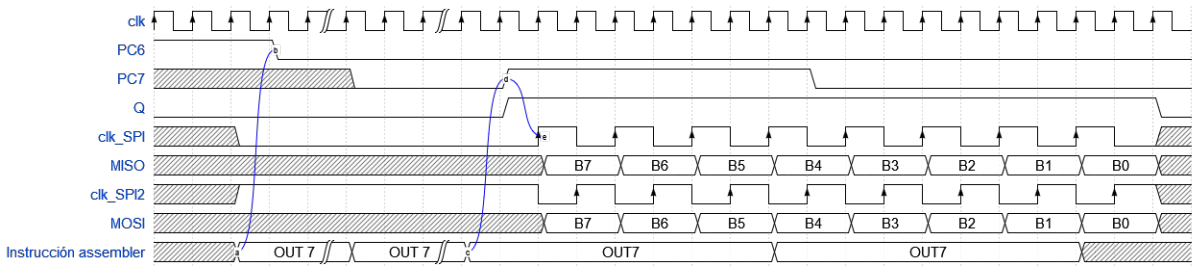


Figura 7.11: Proceso de transmisión SPI

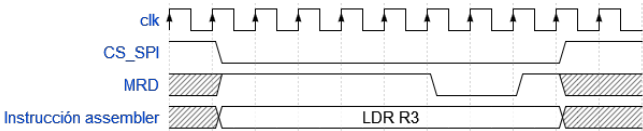


Figura 7.12: Proceso de lectura

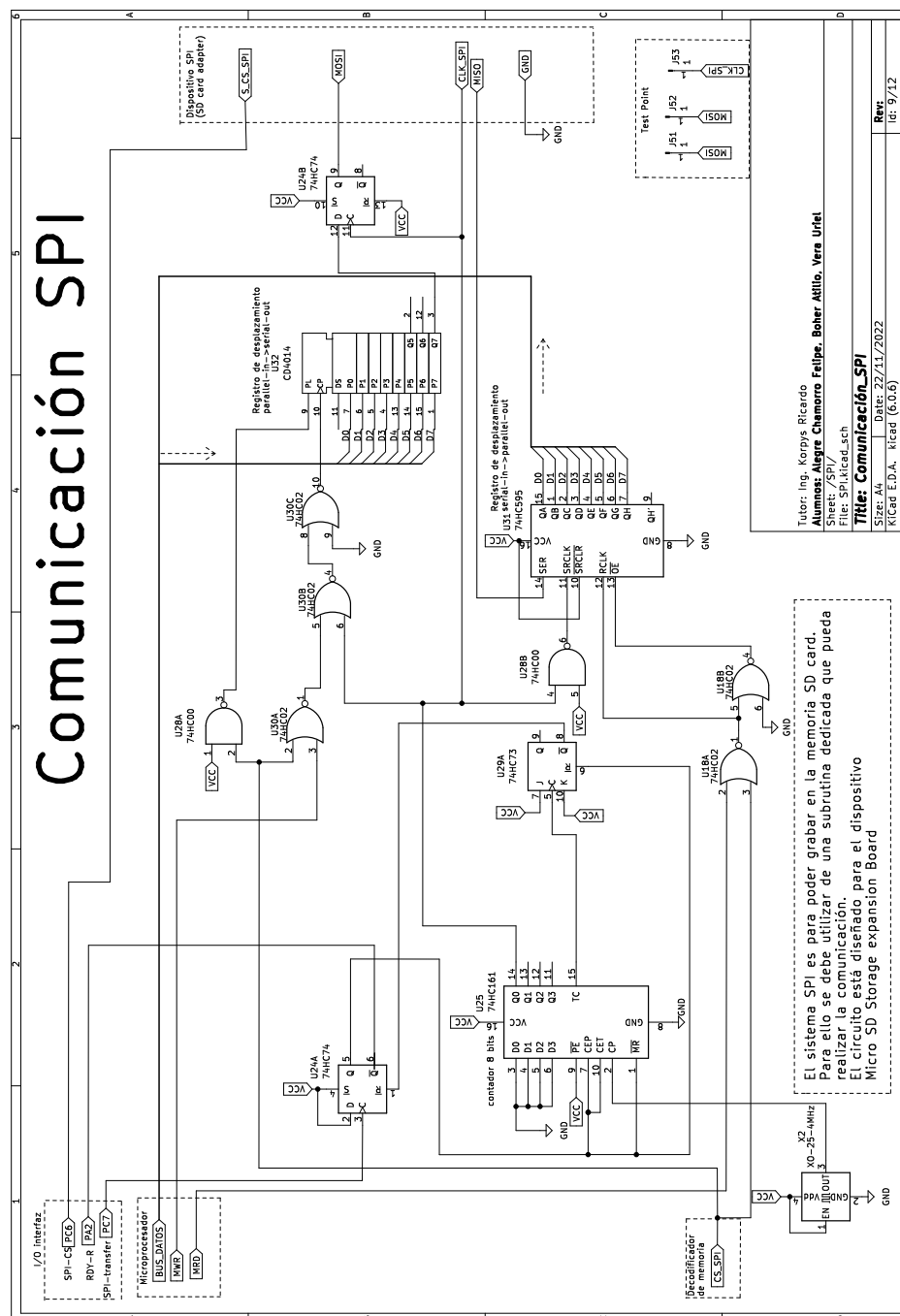


Figura 7.9: Esquema de comunicación SPI de sistema final

Listado 7.1: Rutina propuesta para la transmisión utilizando el circuito de comunicación SPI

```

1 ;-----DEFINICIONES PREVIAS-----
2 ; Se usan registros y valores especificos para que se entienda mejor, pero se
   pueden usar otros registros y otros valores
3 ; Para controlar el 82C55, debe estar en la memoria lo siguiente (Esas
   posiciones de memoria no deben ser utilizadas por otra rutina, de otro modo,
   elegir otras direcciones; pero tienen que ser consecutivas y apuntar a la
   RAM. Notese que así como estan, en el sistema medio y final no funcionaria,
   debido a que la RAM empieza en 8000 hex, se comenta esto para que quede
   claro que estos valores son meramente ilustrativos, y que a la hora de
   implementar la rutina deben ser elegidos correctamente y escritos en la RAM
   en una rutina previa, preferentemente la rutina que configura los puertos
   del 82C55, la cual debe se ejecutada antes de intentar utilizar el 82C55 o
   esta rutina)
4 ; los bits de datos son las instrucciones que se escriben en la palabra de
   control (control Word) del 82C55 para controlar sus salidas, para entender
   de donde salieron ver la seccion de control del puerto C a traves de la
   palabra de control en la hoja de datos del 82C55
5
6 ; Direccion | Dato | Accion sobre el 82C55
7 ; 7F00 | 0XXX1100 | para PC6 -> 0
8 ; 7F01 | 0XXX1110 | para PC7 -> 0
9 ; 7F02 | 0XXX1111 | para PC7 -> 1
10 ; 7F03 | 0XXX1110 | para PC7 -> 0
11 ; 7F04 | XXXXXXXX | EL MICRO VA A ESCRIBIR AQUI, SE UTILIZA PARA LEER EL
    PUERTO A
12 ; 7F05 | 0XXX1101 | para PC6 -> 1
13
14 ;-----REGISTROS UTILIZADOS-----
15
16 ; R3 -> F800 R3 apunta a la primera direccion del SPI
17 ; R4 -> 7F00 R4 apunta a la direccion donde estan las instrucciones para
    hacer Set y Reset de PC6 y PC7 del 82C55, tambien para la lectura del puerto
    A
18 ; Estos registros deben ser cargados en una rutina previamente, y no deben
    modificarse en el transcurso de otras rutinas no relacionadas a la del
    circuito para la comunicacion SPI de manera que siempre esten listas para
    ser utilizados
19
20 ;-----RUTINA DE ESCRITURA Y LECTURA JUNTAS-----
21 ; Se parte de que el dato que se quiere escribir esta en el acumulador D
    , si no se quiere transmitir y solo recibir, entonces no importa que
    tiene D
22 STR R3 ; D -> M(R3) se escribe el dato a enviar al CD4014
23 SEX R4 ; 4 -> X equivale a decir que ahora R(X) = R4
24 OUT 7 ; M(R(X)) -> BUS; R(X) + 1 -> R(X) Esto hace PC6 -> 0
25 OUT 7 ; M(R(X)) -> BUS; R(X) + 1 -> R(X) Esto hace PC7 -> 0
26 OUT 7 ; M(R(X)) -> BUS; R(X) + 1 -> R(X) Esto hace PC7 -> 1
27 OUT 7 ; M(R(X)) -> BUS; R(X) + 1 -> R(X) Esto hace PC7 -> 0
28 CHECK: INP 4 ; BUS -> M(R(X)); BUS -> D se lee el puerto A del 82C55
29 LDX ; M(R(X)) -> D

```

```
30      SHR      ; SHIFT D RIGHT, LSB(D) -> DF, 0 -> MSB(D)
31      SHR
32      SHR      ; Ahora DF = PA2
33      BNF CHECK
34      INC R4
35      OUT 7     ; M(R(X)) -> BUS; R(X) + 1 -> R(X) Esto hace PC6 -> 1
36      DEC R4
37      DEC R4
38      DEC R4
39      DEC R4
40      DEC R4
41      DEC R4 ; decremento R4 seis veces para ponerlo en su valor inicial (la
              ; instruccion OUT lo modifiko, lo quiero dejar como estaba)
42      LDN R3    ; M(R(N)) -> D; R(N) + 1 -> R(N) Hay que tener cuidado porque
              ; aumenta R(N)
43      DEC R3    ; Decrementa R3 porque la instruccion anterior lo aumento
```

7.1.9. Alimentación de compuertas

La alimentación de compuertas es un esquemático en el cual se representan los puntos de alimentación de los circuitos integrados y los elementos no usados de los dispositivos electrónicos. Principalmente sirve para tener una vista más limpia en los demás esquemas.

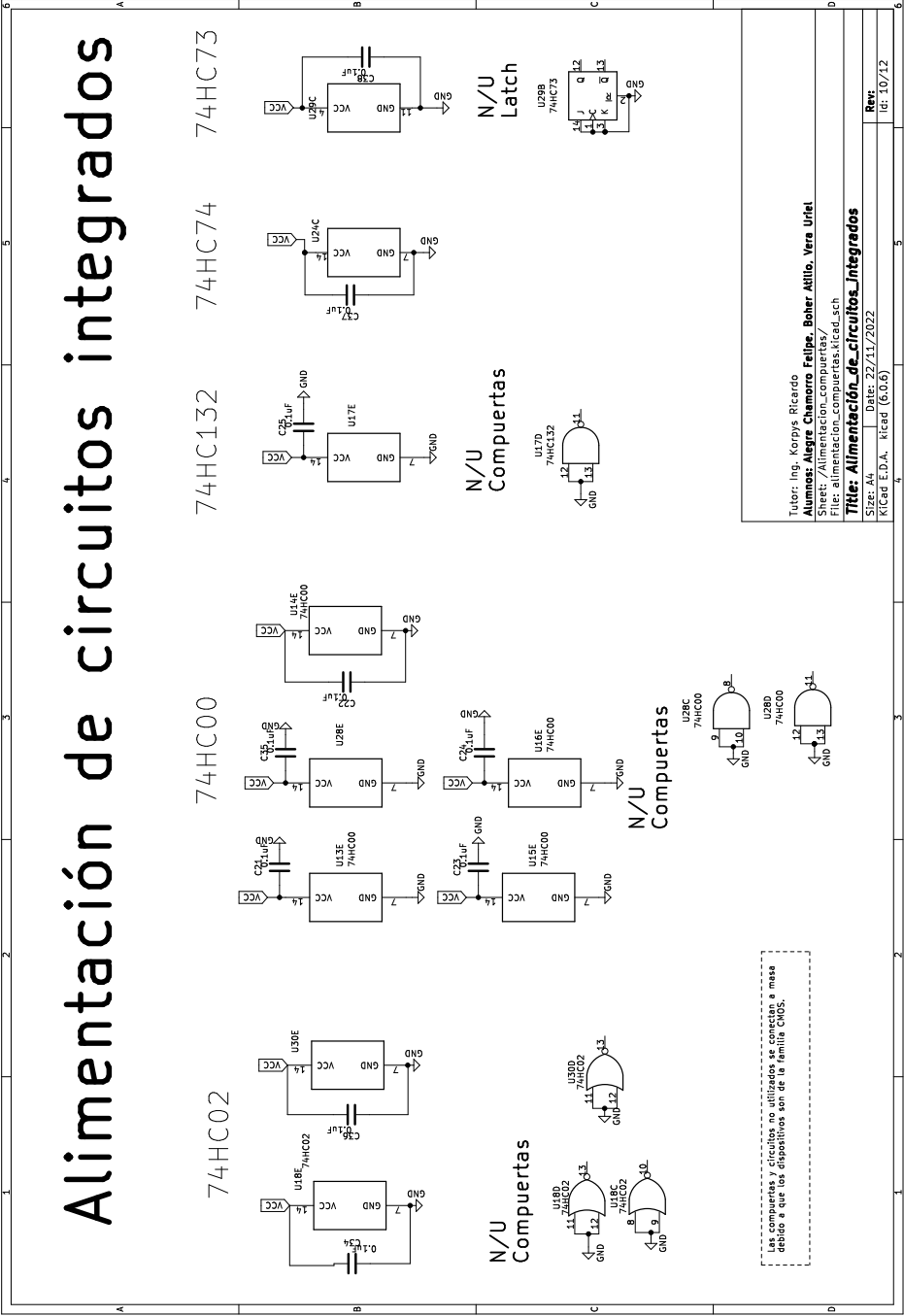


Figura 7.13: Esquema de alimentación de compuertas de sistema final

7.1.10. Interfaz I/O

El microprocesador CDP1802 posee una arquitectura poco común, la cual cuenta con un mapa de memoria separado para los dispositivos de entrada y salida. Esto le da la capacidad al microprocesador de poder acceder a la memoria y a los dispositivos de entrada y salida al mismo tiempo, es decir, con la ejecución de una sola instrucción.

En el set de instrucciones [1] puede verse la descripción de las instrucciones de entrada y salida. Por un lado, la instrucción “INP”, para leer datos de un dispositivo de entrada y salida:

$$\text{BUS} \rightarrow \text{M(R(X))}; \text{BUS} \rightarrow \text{D}$$

Es decir, el microprocesador actúa sobre el dispositivo de entrada para que este coloque el dato a leer en el BUS de datos, luego lo almacena en la dirección de memoria apuntada por **R(X)**.

Algo similar sucede con la instrucción para enviar datos a un dispositivo, denominada “OUT”:

$$\text{M(R(X))} \rightarrow \text{BUS}; \text{R(X)+1} \rightarrow \text{R(X)}$$

Es decir, el microprocesador coloca en el bus de datos el dato guardado en la posición de memoria apuntada por **R(X)**, y luego actúa sobre el dispositivo de salida para que este lea el dato.

Lo que significa esto es que el microprocesador no puede usar directamente sus pines *Memory Write* (**MWR**) o *Memory Read* (**MRD**) en un dispositivo de entrada y salida, dado que los mismos se utilizan para acceder a la memoria. Para solventar eso, el microprocesador utiliza los pines TPA, TPB y **MRD**, en conjunto con un circuito lógico para actuar sobre el dispositivo de entrada y salida.

Este circuito lógico se obtuvo de la figura 98 de la página 77 del manual del CDP1802 [8]; en conjunto con la figura 1 de la página 4 de la hoja de datos del CDP1853 [35], resultando en el circuito de la figura 7.14.

Con este circuito es posible controlar las salidas y leer las entradas del 82C55 utilizando las instrucciones de entrada y salida del CDP1802 (INP y OUT). Todas las instrucciones utilizables se encuentran en las tablas 7.1 y 7.2, junto a una descripción de la acción realizada por las mismas.

Tabla 7.1: Instrucciones INP utilizables con el 82C55

Instrucción	Acción
INP 4	Guarda en M(R(X)) y en D el dato presente en el puerto A del 82C55
INP 5	Guarda en M(R(X)) y en D el dato presente en el puerto B del 82C55
INP 6	Guarda en M(R(X)) y en D el dato presente en el puerto C del 82C55
INP 7	Guarda en M(R(X)) y en D el dato presente en el registro de control del 82C55

Tabla 7.2: Instrucciones OUT utilizables con el 82C55

Instrucción	Acción
OUT 4	Escribe el dato $M(R(X))$ en el puerto A del 82C55
OUT 5	Escribe el dato $M(R(X))$ en el puerto B del 82C55
OUT 6	Escribe el dato $M(R(X))$ en el puerto C del 82C55
OUT 7	Escribe el dato $M(R(X))$ en el registro de control del 82C55

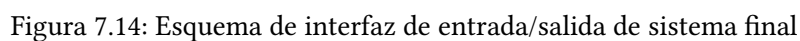
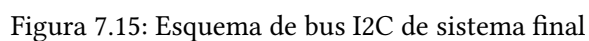


Figura 7.14: Esquema de interfaz de entrada/salida de sistema final

7.1.11. BUS I2C

El esquema presentado para el bus I2C presenta una interfaz construida a partir de transistores NPN. La razón de esta interfaz radica en la característica de los dispositivos I2C, que se conectan al bus mediante una interfaz de drenador abierto. Esto podría generar cortocircuitos en un accionamiento simultáneo del bus, por lo que se requiere de esta interfaz.



7.2. Lista de componentes

En la tabla 7.3 se encuentra la lista de materiales para la construcción del sistema final.

Tabla 7.3: Lista de componentes del sistema final

Cantidad	Etiqueta Identificador	Descripción	Fabricante	Número de parte
1	U1	Microprocesador	RCA	CDP1802ACE
1	U2	Latch de 8 bits	Philips	74HC573N
1	U3	Memoria EEPROM 32k x 8	XICOR	X28C256D-25
1	U4	Memoria EEPROM Serial I2C 128k x 8	Microchip	24LC1025
1	U5	Time keeping RAM 32K x8	Dallas	DS1744-070
1	U6	Memoria RAM 32k x 8	HYUNDAI	HY62256ALP-10
1	U7	Memoria FRAM Serial I2C 8k x 8	Fujitsu Semiconductor	MB85RC64A
1	U8	Conversor analógico-digital con interfaz I2C	Maxim Integrated	MAX127
1	U9	Regulador de tensión 12 V	Motorola	7812CT
1	U10	Regulador de tensión 5 V	ST Microelectronics	L7805CV
1	U11	High Precision Operational Amplifier	Burr-Brown	OPA4277PA
1	U12	Regulador de tensión -12 V	ST Microelectronics	L7912CV
5	U13, U14, U15, U16, U28	Cuádruple compuerta NAND de dos entradas	Texas Instruments	SN74HC00N
1	U17	Cuádruple compuerta NAND de dos entradas con Schmitt-Trigger	Texas Instruments	SN74HC132N
2	U18,U30	Cuádruple compuerta NOR de dos entradas	Texas Instruments	SN74HC02N
1	U19	Decodificador Multiplexador 3 a 8 líneas	Texas Instruments	SN74HC138N
1	U20	CMOS Programmable Peripheral Interface	Intersil	CP82C55A-5Z
1	U21	Regulador programable shunt	Fairchild	LM336Z25
1	U24	Doble flip-flop tipo D con Set y Reset	Texas Instruments	SN74HC74N
1	U25	Contador binario de 4 bits	Fairchild Semiconductor	MM74HC161N
1	U29	Doble flip-flop JK con Set y Reset	National Semiconductor	MM74HC73N
1	U31	Registro de desplazamiento de 8 bits	Texas Instruments	SN74HC595N
1	U32	Registro de desplazamiento de 8 estados	Motorola	MC14014BCP
1	X2	XO-22BE-4MHz	Varios	Varios
1	XTAL1	Cristal 4 MHz	CQ Electronics	Varios
1	SW1	Llave DPDT 6 pines	Varios	Varios

Cantidad	Etiqueta Identificador	Descripción	Fabricante	Número de parte
3	SW2, SW3, SW4	Pulsador 6mm x 4,3mm	Varios	Varios
5	T2, T3, T4, T5, T6	Transistor BJT NPN	Motorola	P2N2222A
1	RV1	Preset 10 k Ω	Varios	Varios
1	D1	Puente rectificador de onda completa B380R	Varios	Varios
6	D2, D3, D4, D5, D6, D7	LED 5 mm	Varios	Varios
2	D8, D9	1N914	Varios	Varios
1	F1	Fusible 10 mm 1 A	Varios	Varios
8	R1, R2, R3, R5, R21, R22, R23, R27	Resistor 47 k Ω 1/8 W	Varios	Varios
1	R4	Resistor 10 M Ω 1/8 W	Varios	Varios
1	R6	Resistor 330 Ω 1/4 W	Varios	Varios
2	R7, R8	Resistor 1 k Ω 1/8 W	Varios	Varios
3	R9, R10, R11	Resistor 220 Ω 1/4 W	Varios	Varios
16	R12, R13, R14, R15, R16, R17, R18, R38, R39, R40, R41, R46, R47, R56, R57, R60	Resistor 10 k Ω 1/8 W	Varios	Varios
5	R19, R20, R24, R25, R26	Resistor 100 k Ω 1/8 W	Varios	Varios
4	R28, R30, R36, R52	Resistor 100 Ω 1/4 W	Varios	Varios
4	R29, R31, R37, R53	Resistor 150 Ω 1/4 W	Varios	Varios
8	R32, R33, R34, R35, R42, R43, R54, R55	Resistor 390 k Ω 1/8 W	Varios	Varios
8	R44, R45, R48, R49, R50, R51, R58, R59	Resistor 1 M Ω 1/8 W	Varios	Varios
2	C2, C3	Capacitor cerámico 30 pF 50 V	Varios	Varios
14	C5, C7, C8, C29, C30, C31, C14, C15, C18, C32, C33, C39, C40, C43	Capacitor cerámico 0,10 μ F 50 V	Varios	Varios
2	C11, C17	Capacitor electrolítico 4700 μ F 25 V	Varios	Varios
2	C12, C16	Capacitor cerámico 0,33 μ F 50 V	Varios	Varios
1	C13	Capacitor electrolítico 10 μ F 25 V	Varios	Varios
1	C19	Capacitor electrolítico 0,01 μ F 25 V	Varios	Varios

Cantidad	Etiqueta Identificador	Descripción	Fabricante	Número de parte
1	C20	Capacitor electrolítico 4,7 μ F 25 V	Varios	Varios
6	J1, J2, J33, J48, J49, J50	1x3 Pines Conectores Macho 2,54 mm	Varios	Varios
38	J3, J4, J5, J6, J7, J8, J9, J10, J11, J12, J13, J14, J15, J16, J17, J18, J19, J20, J21, J22, J23, J24, J25, J26, J27, J28, J29, J30, J31, J36, J40, J41, J42, J45, J46, J51, J52, J53	Pin conector macho (1 <i>Male Header Pin</i>)	Varios	Varios
3	J32, J35, J38	1x6 Pines Conectores Macho 2,54 mm	Varios	Varios
3	J34, J37, J39	1x5 Pines Conectores Macho 2,54 mm	Varios	Varios
2	J43, J44	Bornera 3 pines 5,08 mm	Varios	Varios
2	J47, J54	1x12 Pines Conectores Macho 2,54 mm	Varios	Varios
1	J55	1x2 Pines Conectores Macho 2,54 mm	Varios	Varios
1	No aplica	Módulo conversor USB-Serial	Future Technology Devices International	FT232RL
1	No aplica	Módulo adaptador tarjeta micro SD	Varios	Varios

7.3. Placa PCB

La placa PCB fue desarrollada en KiCad, al igual que los anteriormente mencionados. En la misma se incluyen todas las partes presentadas en los esquemáticos de la sección 7.1. Para ello se requirió un tamaño de 30 cm de largo por 30 cm de ancho, en una placa doble faz. Ambos lados y la máscara de componentes se presentan en las figuras 7.16, 7.17 y 7.18. Sin embargo, se adjuntarán en tamaño original en el apéndice. Por otro lado, se presenta una visión en tres dimensiones creada por el mismo software de diseño en la figura 7.19.

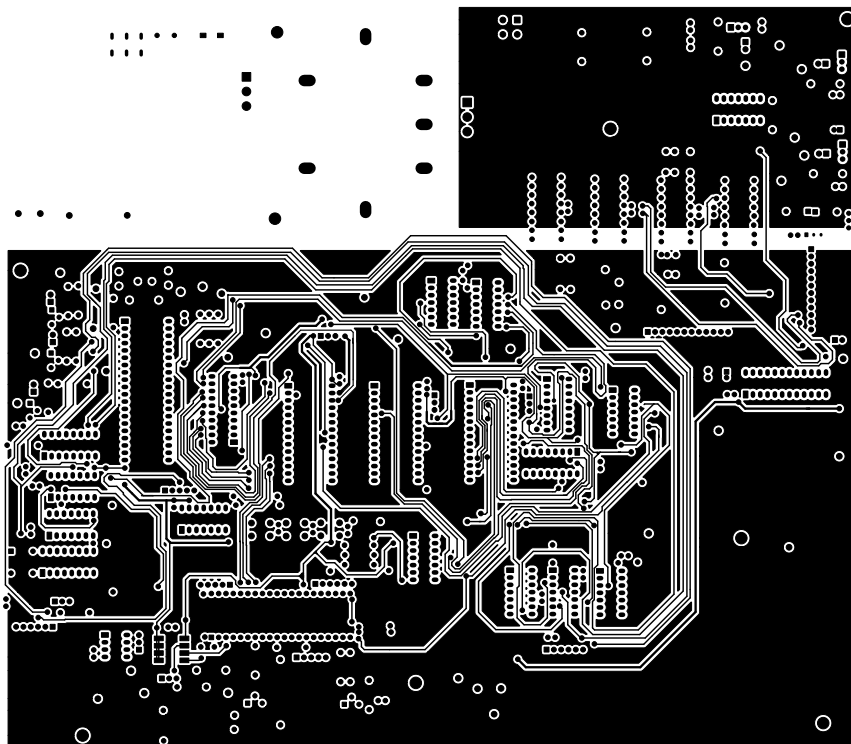


Figura 7.16: Pistas de PCB sistema final parte frontal

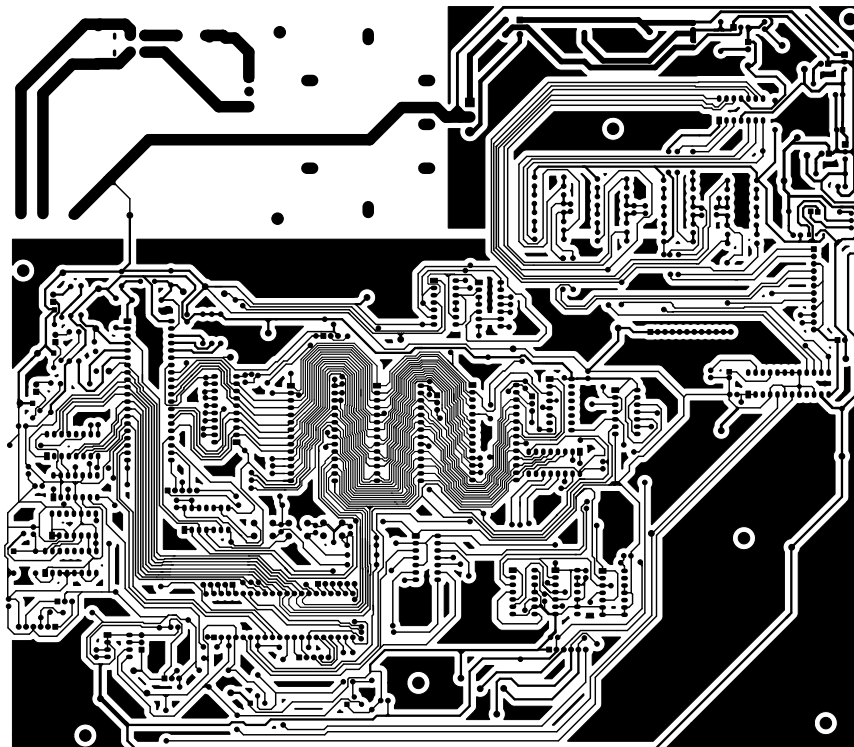
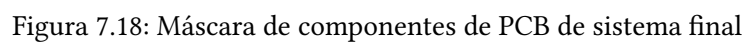


Figura 7.17: Pistas de PCB sistema final parte posterior



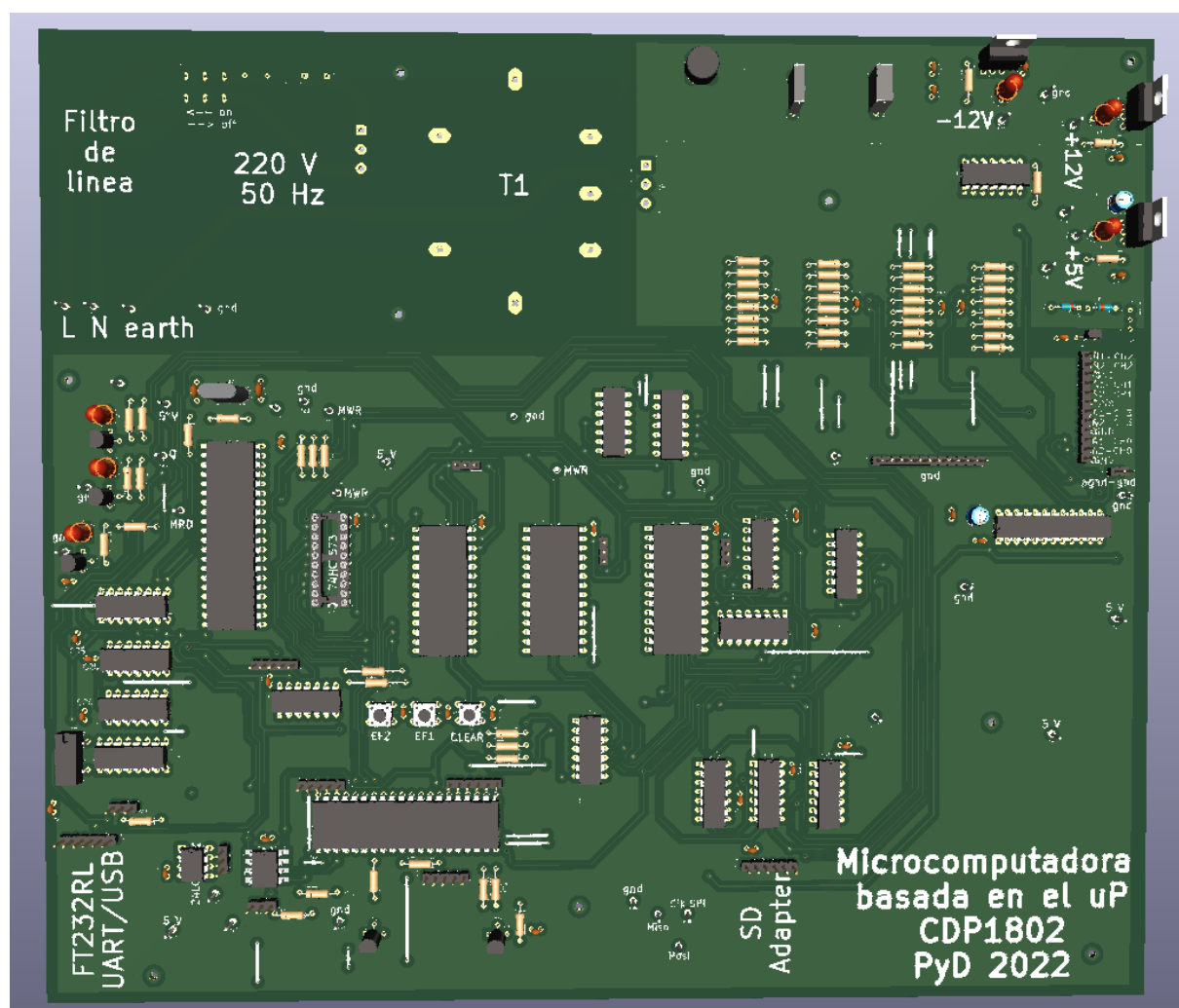


Figura 7.19: Visión en tres dimensiones de sistema final

Conclusiones

Se han estudiado las características correspondientes al microprocesador CDP1802 y entendido el principio de funcionamiento del mismo, conocimiento que fue aplicado para el desarrollo de diferentes circuitos de complejidad progresiva que se encuentran explicados en el presente documento. Se han reconocido los fundamentos y funcionalidades de los elementos correspondientes a una microcomputadora, como ser memorias, interfaz de entrada salida, conversor analógico digital, entre otras, de tal manera de poder proyectar subsistemas dedicados a funciones específicas que tengan interoperabilidad entre los mismos, seleccionando los diferentes dispositivos adecuadamente para su compatibilidad en las diferentes aplicaciones. Se analizaron circuitos y placas de desarrollo con el microprocesador utilizado, que sirvieron de base para los sistemas proyectados en este trabajo.

Se han desarrollado competencias en el análisis de costos, pedidos de presupuesto y toda la logística involucrada en el proyecto e implementación de un circuito electrónico, comprendiendo las utilidades presentadas por un proceso de planificación y administración del tiempo, como el diagrama de Gantt, y también se ganó experiencia con herramientas de redacción de informes que permiten plasmar los resultados obtenidos en cualquier ámbito de manera profesional.

Se han estudiado, analizado, comprendido y simulado diferentes arquitecturas que utilizan el microprocesador CDP1802, cuyos resultados brindaron entendimiento respecto a las topologías propuestas. Se utilizaron herramientas de simulación (Emma02, Proteus) que presentan herramientas de depuración, las cuales fueron sumamente importantes para detectar errores que presentan complejidad para detectarse en el circuito físico y corroborar el funcionamiento de las placas de desarrollo ya existentes con arquitecturas similares a las implementadas.

Se han proyectado, diseñado e implementado dos microcomputadoras (sistema mínimo y sistema medio) con arquitecturas diferentes y se ha logrado su correcto funcionamiento al ser probadas y testeadas. Simples programas se han implementado en el sistema mínimo y en el sistema medio, corroborando el correcto funcionamiento de los mismos. En el sistema medio se han ejecutado un intérprete de un lenguaje de programación y un sistema operativo sin disco, y se ha comprobado su correcto funcionamiento. Se ha proyectado y diseñado una microcomputadora (sistema final) que posea todas las aplicaciones planteadas desde un principio, como ser el conversor analógico digital y el uso de memorias seriales, por nombrar algunas. Se diseñaron el esquemático y el PCB del sistema final los cuales se encuentran presentes en el documento. Se logró integrar todas las funcionalidades pensadas en un principio y plasmarlas sobre el esquemático del sistema.

Durante el desarrollo del trabajo se han utilizado y aplicado con éxito gran parte de los conocimientos obtenidos en la carrera de ingeniería electrónica y se adquirieron otros relacionados con llevar a la práctica los conceptos teóricos ya estudiados. Uno de ellos es el uso de software de diseño KiCad, en el cual se desarrollaron todos los circuitos planteados para el trabajo y la construcción de placas PCB de doble faz, entre otros.

Como conclusión en vista de las facultades adquiridas con el desarrollo del presente trabajo, se destaca la interdependencia de software-hardware, sobre todo cuando el desarrollo de software es en bajo nivel como en este caso. Es de carácter fundamental para los diseñadores de este tipo de circuitos conocer el funcionamiento claro, uno del otro, para poder realizar un avance significativo en el desarrollo del proyecto.

Las principales dificultades en el desarrollo del trabajo se relacionan con el entendimiento del microprocesador, el cual presenta una arquitectura particular. Se destaca principalmente el modo en el que opera con las instrucciones de entrada-salida, sin embargo, el uso de registros y su programación difieren mucho de las arquitecturas estudiadas en la carrera.

Se han cumplimentado en gran medida los objetivos planteados para este proyecto en un principio, logrando integrar los conceptos estudiados a lo largo de la carrera correctamente y diseñar e implementar diferentes sistemas de microcómputo que sirven para sus determinadas aplicaciones. Los alumnos a cargo del proyecto han adquirido herramientas específicas para la integración de conceptos, proyecto y diseño de sistemas integrados, comprensión cabal del funcionamiento de diversos dispositivos, entre las más importantes. Estas herramientas, adquiridas por los alumnos para su próxima inserción laboral, cumplimentan así los objetivos de la materia Proyecto y Diseño Electrónico.

Trabajos a futuro

En este capítulo presentaremos una lista de actividades que pueden ser desarrolladas a partir de las bases presentadas en este trabajo. Las actividades propuestas buscan ser alcanzables con un nivel medio de dificultad considerándose como punto de partida las herramientas teórico-prácticas presentadas.

9.1. Construcción de sistema final

Se considera que el primer paso lógico a seguir, luego de lo presentado, es la construcción del sistema final. La construcción del circuito puede proveer información útil, verificando los esquemas planteados o identificando fallas del mismo. En dicha construcción se deben realizar pruebas de alimentación, comunicación entre circuitos integrados, coordinación en instrucciones simples, entre otras pruebas.

9.2. Programas de prueba

Una vez construido el sistema final, este puede ser puesto a prueba con los programas diseñados para el sistema medio y mínimo, sin embargo, estos no explotarían el potencial del circuito presentado en el sistema final. Es por ello que se requiere el diseño de programas de prueba que en principio serán simples para verificar el funcionamiento del circuito y luego podrán ser modificados para realizar tareas más complejas.

9.2.1. Coordinación de interfaz entrada-salida

El esquema presentado en la figura 7.14 presenta una interfaz de entrada-salida basada en el circuito integrado 82C55. Para el correcto funcionamiento de la interfaz, se debe diseñar un software capaz de controlar los puertos disponibles en el chip. Este software debería ser uno de los primeros en probarse en el sistema final, ya que sirve para comunicarse con gran parte del resto del circuito.

9.2.2. Comunicación I2C

El circuito final cuenta con circuitos integrados que funcionan a través de la comunicación I2C. Para lograr un funcionamiento óptimo del circuito, es indispensable el planteamiento de un software que permita enviar y recibir instrucciones a estos integrados.

9.2.3. Conversión Analógico-Digital

Una vez diseñado el sistema de comunicación I2C, se puede plantear un software que permita controlar el sistema de conversión A/D. Para ello debe poder realizar la configuración de los niveles de tensión asociados a la lectura de cada uno de los canales disponibles, así como también realizar la lectura de un canal deseado.

9.2.4. Rutina de cambio en el mapa de memoria

Para desbloquear un mayor potencial en el funcionamiento del sistema final, se debe plantear un programa de booteo que permita transmitir la memoria de programa a la memoria RAM y luego realice el cambio en el mapa de memoria poniendo en la parte baja del mismo (0000h a 7FFFh) otra memoria RAM, permitiendo almacenar mayor volumen de información en ejecución.

9.2.5. Escritura de memoria SDcard

Aprovechando la rutina planteada en la sección 7.1.8, diseñar un programa capaz de escribir en una memoria del tipo SDcard. A estas memorias se puede acceder mediante un protocolo SPI, aun así debe ser analizado el funcionamiento específico del grabado en memorias SPI, para lograr modificar la rutina de comunicación SPI y lograr un sistema de escritura satisfactorio.

9.2.6. Copia de memoria

Una vez que los sistemas de escritura en memoria, tanto los de comunicación I2C, como los de comunicación SPI, funcionen correctamente, se puede desarrollar un software capaz de guardar el contenido de una sección o la totalidad del mapa de memoria en estas memorias no volátiles.

9.3. Rediseño de circuito

Una vez probado y verificado el sistema final, puede ser rediseñado con el fin de optimizar en algún aspecto su funcionamiento. En esta sección presentamos algunas propuestas.

9.3.1. Rediseño de placa PCB

En caso de que se desee replicar el sistema final, manteniendo su topología fundamental, puede establecerse un nuevo layout que reduzca la distancia entre componentes y el número de puentes. Dado el caso, puede analizarse el uso de GALs o PALs para reemplazar las compuertas lógicas y simplificar la disposición del circuito.

9.3.2. Conversión Digital-Analógico

Para dotar de aún más herramientas al sistema, se puede plantear la incorporación de un circuito de conversión digital-analógico. Esto permitiría a la computadora interactuar con mayor número de periféricos.

9.3.3. Separación galvánica de entradas analógicas

Para un mejor desempeño de la etapa de conversión analógico-digital (o digital-analógico) puede plantearse la separación galvánica de estas para reducir interferencias que puedan existir.

9.3.4. Traslado del circuito SPI del mapa de memoria principal al mapa de memoria de entrada y salida del CDP1802

En el diseño realizado del sistema final, el circuito SPI interrumpe la continuidad de la RAM en el mapa de memoria, utilizando 400h direcciones de memoria; sin embargo, solo necesita una dirección.

Como alternativa a esto, se propone modificar el circuito SPI para que el mismo esté en el mapa de memoria de dispositivos de entrada y salida del CDP1802, esto liberaría el espacio de memoria del mapa principal actualmente utilizado por el circuito SPI y, además, se utilizarían las instrucciones INP y OUT para el proceso de lectura y escritura de datos. Es necesario recordar que el 82C55 utiliza las direcciones 4, 5, 6 y 7 de los pines N0, N1 y N2, por lo tanto, quedan libres las direcciones 1, 2 y 3 que pueden utilizarse.

Aunque pareciera una complicación, realizar esta modificación simplificaría el decodificador del mapa de memoria, lo cual debe tenerse en cuenta a la hora de comparar las dos alternativas.

9.4. Desarrollo de controlador lógico programable

Combinando algunas de las soluciones propuestas tanto en software como en hardware, se puede plantear el uso del sistema como PLC. Para poder avanzar en el desarrollo de este sistema se debe contar con un hardware probado y libre de errores a fin de enfocarse en la tarea de desarrollo de software que puede resultar compleja.

9.5. Topología con dos microprocesadores

A modo didáctico puede ser planteado un nuevo sistema que presente dos microprocesadores, esto permitiría expandir el mapa de memoria, realizar tareas en paralelo, entre otras cosas. El planteamiento del software debería contemplar un proceso de comunicación entre ambos microprocesadores. Las combinaciones y utilidades que pueda presentar son diversas y deben ser desarrolladas con detenimiento. Sin embargo, presentamos una propuesta de funcionamiento.

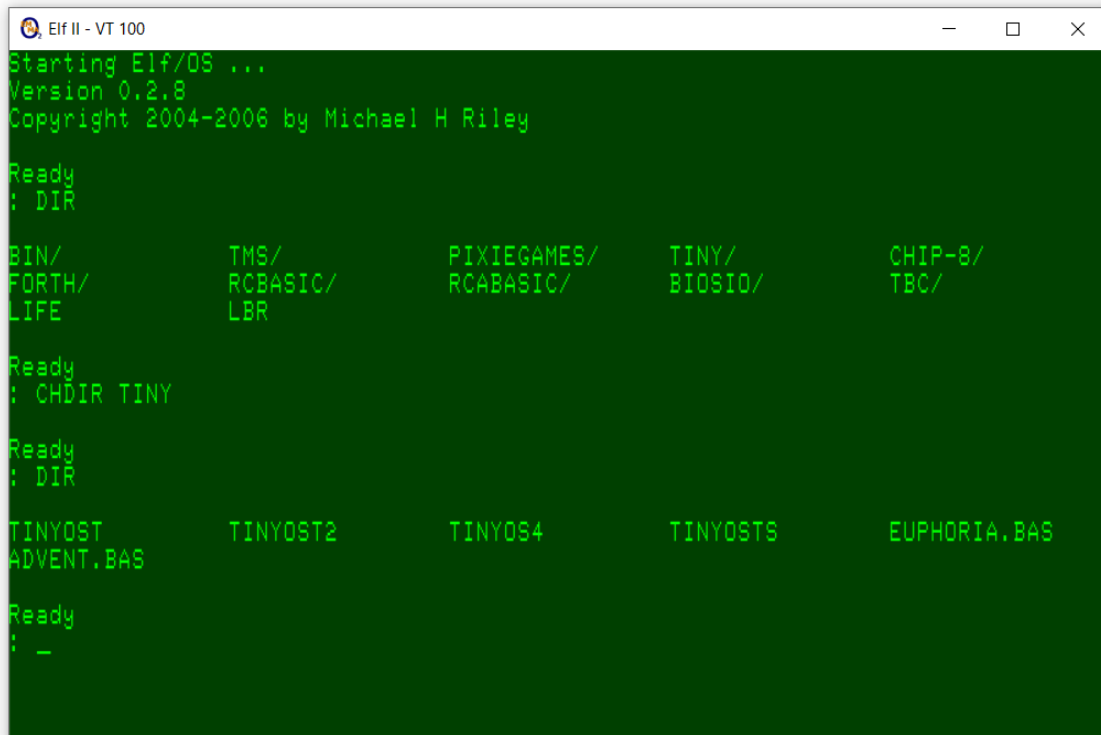
Establecer un microprocesador maestro el cual delega tareas preestablecidas al segundo microprocesador; la comunicación entre ellos se realizaría mediante los flags EF1-EF4. Para la interacción entre

los buses de datos se utilizará un sistema basado en un latch de 8 líneas, que disponga de un flag indicador de disponibilidad de dato.

Lineamientos para el desarrollo del sistema operativo

El CDP1802 ya cuenta con algunos sistemas operativos funcionales, los mismos pueden probarse en el simulador Emma02. Entre ellos está el ElfOS[36], el cual utilizamos como guía para describir algunas características a esperarse de tal sistema.

El sistema operativo a desarrollarse debe contar con un sistema de archivos, el cual incluya directorios y la posibilidad de navegar en ellos. Como ejemplo, el ElfOS, posee los comandos DIR y CHDIR, los cuales tienen la función de visualizar y cambiar el directorio, respectivamente. En la figura 10.1 puede verse como ejemplo la ejecución de estos dos comandos.



```

Elf II - VT 100
Starting Elf/OS ...
Version 0.2.8
Copyright 2004-2006 by Michael H Riley

Ready
: DIR

BIN/          TMS/          PIXIEGAMES/   TINY/          CHIP-8/
FORTH/        RCBASIC/        RCBASIC/      BIOSIO/        TBC/
LIFE

Ready
: CHDIR TINY

Ready
: DIR

TINYOST       TINYOST2       TINYOS4       TINYOSTS       EUPHORIA.BAS
ADVENT.BAS

Ready
: -

```

Figura 10.1: Sistema de directorios del ElfOS

El sistema operativo también debe incluir programas para crear y editar archivos de texto. El ElfOS posee este programa, que además brinda la posibilidad de escribir código en el lenguaje ensamblador del CDP1802, el cual puede luego ser ejecutado dentro del mismo sistema operativo, retomando su operación una vez ejecutado el código.

El sistema debe funcionar mediante el envío y recepción de caracteres (sin interfaz gráfica) mediante la comunicación con una terminal externa. Existen módulos conversores de protocolo Serial a USB que permiten realizar esta tarea.

El sistema no debe incluir conexión a internet ni ser multiusuario.

Debe contemplarse la posibilidad de que el sistema operativo a desarrollar sea de tiempo real (*Real-Time Operating System RTOS*). Esto significa que debe ser capaz de aceptar y responder a eventos externos con cierto grado de fiabilidad, y en un margen de tiempo específico. Esta cualidad permite utilizar el sistema en aplicaciones industriales, donde se requiere atender a gran número de eventos externos y además controlar variables externas. La implementación de un sistema operativo de estas características permitiría utilizar la microcomputadora como un PLC.

Bibliografía

- [1] RCA, *RCA COS/MOS Memories, Microprocessors, and Support Systems*, ASIN: B000KF7M0Q, CDP1802 datasheet, Page 17, 1979. dirección: <https://www.datasheetarchive.com/pdf/download.php?id=696f0d0fbeeada82c6995adde9562efbb29f2&type=O&term=RCA-CDP1802> (Accedido: 30/9/2022).
- [2] «RCA Laboratories - Research Report 1973 (online). »dirección: http://www.retrotechnology.com/memship/RCA_Res_1973_COSMAC.pdf (Accedido: 30/9/2022).
- [3] «Chip Hall of Fame: RCA CDP 1802 - IEEE Spectrum magazine (online article). »dirección: <https://spectrum.ieee.org/chip-hall-of-fame-rca-cdp-1802> (Accedido: 30/9/2022).
- [4] J. E. Tomayko, «Computers in Spaceflight: The NASA Experience,» en Alpha Books, 1994, cap. 6, ISBN: 9781567614633. dirección: <https://history.nasa.gov/computers/Ch6-3.html> (Accedido: 30/9/2022).
- [5] J. Weisbecker. «Popular Electronics Magazine - Build the COSMAC ELF Article. »dirección: <https://worldradiohistory.com/Archive-Poptronics/70s/1976/Poptronics-1976-08.pdf> (Accedido: 30/9/2022).
- [6] «Alan Turing: How His Universal Machine Became A Musical Instrument - IEEE Spectrum magazine (online article). »dirección: <https://spectrum.ieee.org/alan-turing-how-his-universal-machine-became-a-musical-instrument> (Accedido: 30/9/2022).
- [7] «The 1802 Membership Card Computer Website (online). »dirección: <http://www.sunrise-ev.com/1802.htm> (Accedido: 30/9/2022).
- [8] RCA Solid State Division, *User Manual For The CDP 1802 COSMAC Microprocessor*. RCA Corporation, 1977. dirección: http://bitsavers.trailing-edge.com/components/rca/cosmac/MPM-201A_User_Manual_for_the_CDP1802_COSMAC_Microprocessor_1976.pdf
- [9] «Entorno de desarrollo Visual Studio Code. »dirección: <https://code.visualstudio.com/> (Accedido: 5/11/2022).
- [10] «Git - programa para el control de versión. »dirección: <https://git-scm.com/downloads> (Accedido: 5/11/2022).
- [11] «Git - configuración para comparar versiones. »dirección: <https://youtu.be/8JJ101D3knE?t=3034> (Accedido: 5/11/2022).
- [12] «A18 1802 series cross-assembler. »dirección: <https://www.retrotechnology.com/memship/a18.html> (Accedido: 5/11/2022).

- [13] «Emulador Emma 02. »dirección: <https://www.emma02.hobby-site.com/> (Accedido: 31/10/2022).
- [14] «Emulador Emma 02 - sección de ayuda. »dirección: <https://www.emma02.hobby-site.com/help/Main.htm> (Accedido: 7/11/2022).
- [15] «Tera Term - Programa emulador de terminal. »dirección: <https://ttssh2.osdn.jp/index.html.en> (Accedido: 5/11/2022).
- [16] «Página de descarga de Multi Commander. »dirección: <http://multicommander.com/downloads> (Accedido: 5/11/2022).
- [17] «Download Newest XGecu T56/T48/TL866II software. »dirección: <http://forums.xgecu.com/viewthread.php?tid=20&page=1&extra=#pid23> (Accedido: 13/11/2022).
- [18] *Atmel*, «64K (8K x 8) Parallel EEPROMs», *AT28C64 datasheet*. dirección: <http://ww1.microchip.com/downloads/en/devicedoc/doc0001h.pdf> (Accedido: 13/11/2022).
- [19] *Xicor*, «5 Volt, Byte Alterable EEPROM 256K (32K x 8)», *X28C256 datasheet*. dirección: <https://docs.rs-online.com/eac9/0900766b800263a1.pdf> (Accedido: 13/11/2022).
- [20] «KiCad EDA - A Cross Platform and Open Source Electronics Design Automation Suite. »dirección: <https://www.kicad.org/> (Accedido: 14/11/2022).
- [21] «Diseño y Programación de Computadoras. »dirección: <http://diseno-programacion-computadoras.blogspot.com/2014/01/construccion-de-una-computadora-con.html> (Accedido: 26/10/2022).
- [22] *Future Technology Devices International*, «USB UART IC», *FT232R datasheet*. dirección: https://ftdichip.com/wp-content/uploads/2020/08/DS_FT232R.pdf (Accedido: 13/11/2022).
- [23] RCA Solid State Division, *Design Ideas Book for the CDP1802 COSMAC Microprocessor*, ISBN: 979-8475533178. RCA Corporation, 1977. dirección: <http://www.cosmacelf.com/publications/books/design-ideas-book-cdp1802.pdf>
- [24] «Programador de EEPROM 28Cxx (con Arduino). »dirección: <https://8bit65.wordpress.com/2014/09/04/programando-la-eprom/> (Accedido: 26/10/2022).
- [25] *Texas Instruments*, «Octal Transparent D-Type Latches With 3-State Outputs», *74HC373 datasheet*. dirección: https://www.ti.com/lit/ds/symlink/cd74hc373.pdf?ts=1669082180400&ref_url (Accedido: 21/11/2022).
- [26] *Maxim Integrated*, «Multirange, +5V, 12-Bit DAS with 2-Wire Serial Interface», *MAX127 datasheet*. dirección: <https://datasheets.maximintegrated.com/en/ds/MAX127-MAX128.pdf> (Accedido: 10/11/2022).
- [27] *Burr-Brown*, «High Precision Quad Operational Amplifier», *OPA4277 datasheet*. dirección: http://pdf-html.ic37.com/pdf_file_A/20200531/pdf_pdf/pdf5/BB/OPA4277PA_datasheet_548235/893974/OPA4277PA_datasheet.pdf (Accedido: 20/11/2022).
- [28] *Philips*, «Octal D-type transparent latch; 3-state», *74HC573 datasheet*. dirección: <http://eeshop.unl.edu/pdf/74HCT573.pdf> (Accedido: 20/11/2022).

- [29] Texas Instruments, «High Speed CMOS Logic 3-to-8 Line Decoder», 74HC138 datasheet. dirección: https://www.ti.com/lit/ds/symlink/cd74hc138.pdf?ts=1668867982221&ref_url (Accedido: 20/11/2022).
- [30] Motorola, «8-Stage Static Shift Register», MC14014B datasheet. dirección: <https://datasheetspdf.com/pdf-file/397464/Motorola/MC14014B/1> (Accedido: 20/11/2022).
- [31] Texas Instruments, «8-Bit Shift Registers With 3-State Output Registers», 74HC595 datasheet. dirección: https://www.ti.com/lit/ds/scls041j/scls041j.pdf?ts=1668955342444&ref_url (Accedido: 20/11/2022).
- [32] Fairchild, «Synchronous Binary Counter with Asynchronous Clear», 74HC161 datasheet. dirección: <https://www.farnell.com/datasheets/59462.pdf> (Accedido: 20/11/2022).
- [33] National Semiconductor, «Dual J-K Flip-Flops with Clear», 74HC73 datasheet. dirección: <https://pdf1.alldatasheet.com/datasheet-pdf/view/157172/NSC/MM74HC73N.html> (Accedido: 20/11/2022).
- [34] Texas Instruments, «Dual D-Type Positive-Edge-Triggered Flip-Flops With Clear and Preset», 74HC74 datasheet. dirección: https://www.ti.com/lit/ds/symlink/sn54hc74-sp.pdf?ts=1668943140964&ref_url (Accedido: 20/11/2022).
- [35] RCA, «Decodificador de 3 a 8 líneas», CDP1853 datasheet. dirección: <http://www.cosmacelf.com/publications/data-sheets/cdp1853.pdf> (Accedido: 20/11/2022).
- [36] «Sistema operativo ElfOS, desarrollado para el CDP1802. »dirección: <https://web.archive.org/web/20220918120828/http://www.elf-emulation.com/elfos.html> (Accedido: 20/11/2022).
- [37] J. Mayné, *Sistemas de Comunicaciones*. Silica, 2003. dirección: https://bairerobotics.tripod.com/data/Sistemas_Comunicaciones_R3_Silica.pdf (Accedido: 9/10/2022).
- [38] Intel Corporation, *8086 Family Utilities User's Guide - Appendix D. MCS-86 Absolute Object File Formats: Hexadecimal Object File Format*, Manual Order Number: 9800639-04 Rev. E, 1982. dirección: https://www.bitsavers.org/pdf/intel/ISIS_II/9800639-04E_8086_Family_Uilities_Users_Guide_May82.pdf (Accedido: 30/9/2022).

Protocolos de Comunicación Serial

A.1. UART

UART (Universal Asynchronous Receiver-Transmitter) [37] es un protocolo de comunicación serial asincrónica y bidireccional full-duplex desarrollado en el año 1960 por la empresa Digital Equipment Corporation. Esta interfaz posee dos líneas de datos, RX y TX, las cuales se interconectan como lo muestra la figura A.1. UART generalmente es usada para establecer una comunicación bidireccional full-dúplex sólo entre dos dispositivos (punto a punto).



Figura A.1: Interconexión de interfaz UART

La información intercambiada entre los dispositivos es mediante paquetes de datos con un formato definido como se muestra en la figura A.2. La comunicación está sincronizada mediante los bits de START y STOP que posee el paquete, considerando que los relojes de los dispositivos transmisor y receptor operan a la misma frecuencia.



Figura A.2: Paquete de datos UART

El paquete de datos indicado posee las siguientes configuraciones que deben efectuarse en los dispositivos que intervienen en la comunicación:

Bit de START Es el primer bit del paquete de datos y permite que el receptor reconozca el inicio de la transmisión de un paquete. Normalmente el transmisor (TX) mantiene la línea de datos en “1” cuando se encuentra en reposo. Para iniciar la transmisión pone a “0” la línea y el receptor, al detectar esto, comienza con la lectura del DATO.

DATO Este campo del paquete puede configurarse de 5 a 9 bits. La configuración debe ser la misma para ambos dispositivos en la comunicación. Los bits son enviados uno a uno, empezando por el menos significativo y terminando con el más significativo.

Bit de Paridad Permite al receptor verificar si los datos recibidos son correctos o no. La paridad es un sistema de comprobación de errores de bajo nivel y puede ser de paridad par o paridad impar. Debe configurarse la misma paridad en ambos dispositivos involucrados en la comunicación.

Bit de STOP Marca el final del paquete de datos al receptor. Puede tener uno o dos bits de longitud. Debe configurarse lo mismo en ambos dispositivos. Al finalizar la transmisión, es habitual que se mantenga la línea de datos en “1” (línea en reposo).

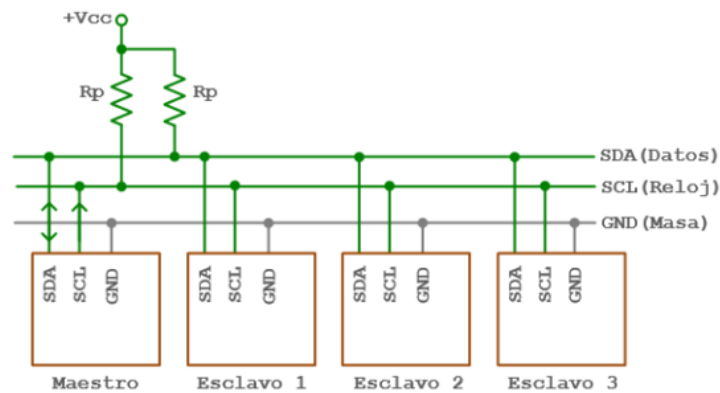
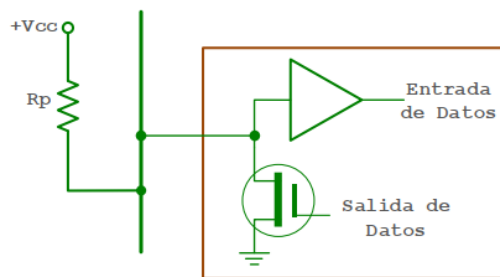
Baud Rate (bits por segundo) Es un parámetro que indica la velocidad a la que se transmiten los datos y se relaciona con el reloj interno que poseen los dispositivos para la lectura/escritura de los bits que conforman el paquete de datos. Todos los dispositivos deben tener el mismo baud rate. Algunas de las velocidades estándares son 4800 bps, 9600 bps, 19200 bps, 115200 bps, etc. De estas, 9600 bps es la más utilizada.

A.2. I²C

I²C (Inter-Integrated Circuit) [37] es un bus de comunicación serie sincrónica bidireccional half-duplex desarrollado en el año 1982 por la empresa Philips Semiconductors. El bus I²C también es conocido con el nombre de TWI (Two Wire Interface). Este bus también posee una arquitectura Maestro-Eslavo, donde el dispositivo Maestro es el único que puede iniciar y finalizar la transmisión y también proveer la señal de reloj. En I²C los dispositivos Esclavos pueden convertirse en Maestros, pero en el bus sólo puede existir un Maestro a la vez.

El bus está compuesto de dos líneas y la conexión común de GND, como puede apreciarse en la figura A.3a. Existe una línea bidireccional para el intercambio de datos que se denomina SDA (Serial Data) y una para la señal de reloj denominada SCL (Serial Clock). Como en este bus no existen líneas de selección de esclavo, cada dispositivo Esclavo debe contar con una dirección única (código de identificación) la cual debe incluirse al dato. Esto resulta en una reducción de la velocidad de transferencia de cada byte de dato.

Las líneas SDA y SCL son del tipo drenador abierto (figura A.3b), por lo cual cada una requiere una resistencia pull-up R_p ($1,8 \text{ k}\Omega \leq R_p \leq 10 \text{ k}\Omega$).

(a) Interconexión de interfaz I²C

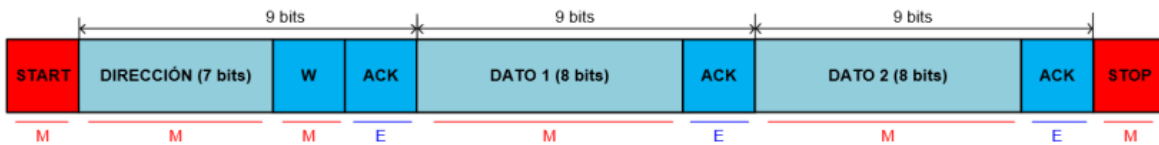
(b) Líneas SDA y SCL

Figura A.3: Interfaz I²C

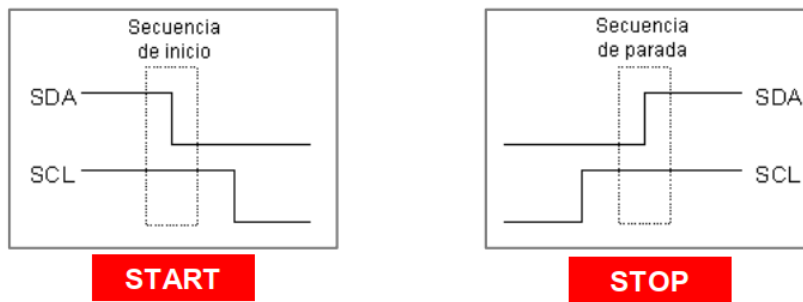
A continuación se describe un ejemplo de trama del protocolo I²C, la cual se puede observar en la figura A.4:

- El Maestro inicia la comunicación con la secuencia de inicio (START), acompañada de 7 bits que corresponden a la dirección del Esclavo con el que se desea comunicar. A esto el Maestro agrega un bit que indica si la operación será de escritura ($W = 0$) o de lectura ($R = 1$). En este caso de escritura.
- Si el Esclavo está disponible en el bus, responde con un bit de reconocimiento (ACK) a lo anterior. Si no hay respuesta del Esclavo en cuestión, el Maestro terminará la comunicación con una secuencia de parada (STOP).
- Luego de que el Maestro recibe el bit de reconocimiento, el mismo envía el Dato 1 (de 8 bits) que corresponde a la dirección del registro del Esclavo donde se desea escribir. A esto el Esclavo vuelve a responder con un bit de reconocimiento (ACK). Si esto no sucede el Maestro finaliza la comunicación con una secuencia de parada (STOP).

- Recibido el bit de reconocimiento, el Maestro envía el Dato 2 (8 bits) que corresponde al valor que se cargará en el registro del Esclavo. Luego, este último responde con un bit de reconocimiento (ACK). Caso contrario, el Maestro finaliza la comunicación.
- Una vez que el Maestro ha escrito el valor correspondiente (Dato 2) en el registro deseado (Dato 1) del Esclavo, finaliza la comunicación con una secuencia de parada (STOP).



(a) Ejemplo de trama I²C: Escritura en el registro de un Esclavo (M: Maestro; E: Esclavo)



(b) Secuencia de inicio y parada I²C

Figura A.4: Trama y secuencia de inicio y parada I²C

En la figura A.5 se encuentra ilustrada en forma más detallada la trama I²C. A continuación se describen cada una de las subfiguras.

Reconocimiento de bits (figura A.5a): El nivel lógico de la línea SDA debe ser estable cuando la línea SCL es alta. La única excepción a esta regla es la generación de secuencias de START y STOP.

Secuencias de START y STOP (figura A.5b): El bus está inactivo cuando SDA = 1 y SCL = 1. El Maestro inicia la transmisión con una secuencia START (SDA = ↓ con SCL = 1) y finaliza la misma con una secuencia de STOP (SDA = ↑ con SCL = 1). Entre una condición de START y STOP, el bus se considera ocupado y ningún otro maestro debe intentar tomar el control del bus. Si el Maestro genera una nueva secuencia de START antes de la secuencia de STOP, esto se conoce como INICIO REPETIDO (*Repeated START*) y se utiliza cuando el Maestro desea iniciar una nueva transferencia sin renunciar al control del bus.

Paquete de Dirección (figura A.5c): Todos los paquetes de direcciones transmitidos tienen una longitud de 9 bits y están constituidos por:

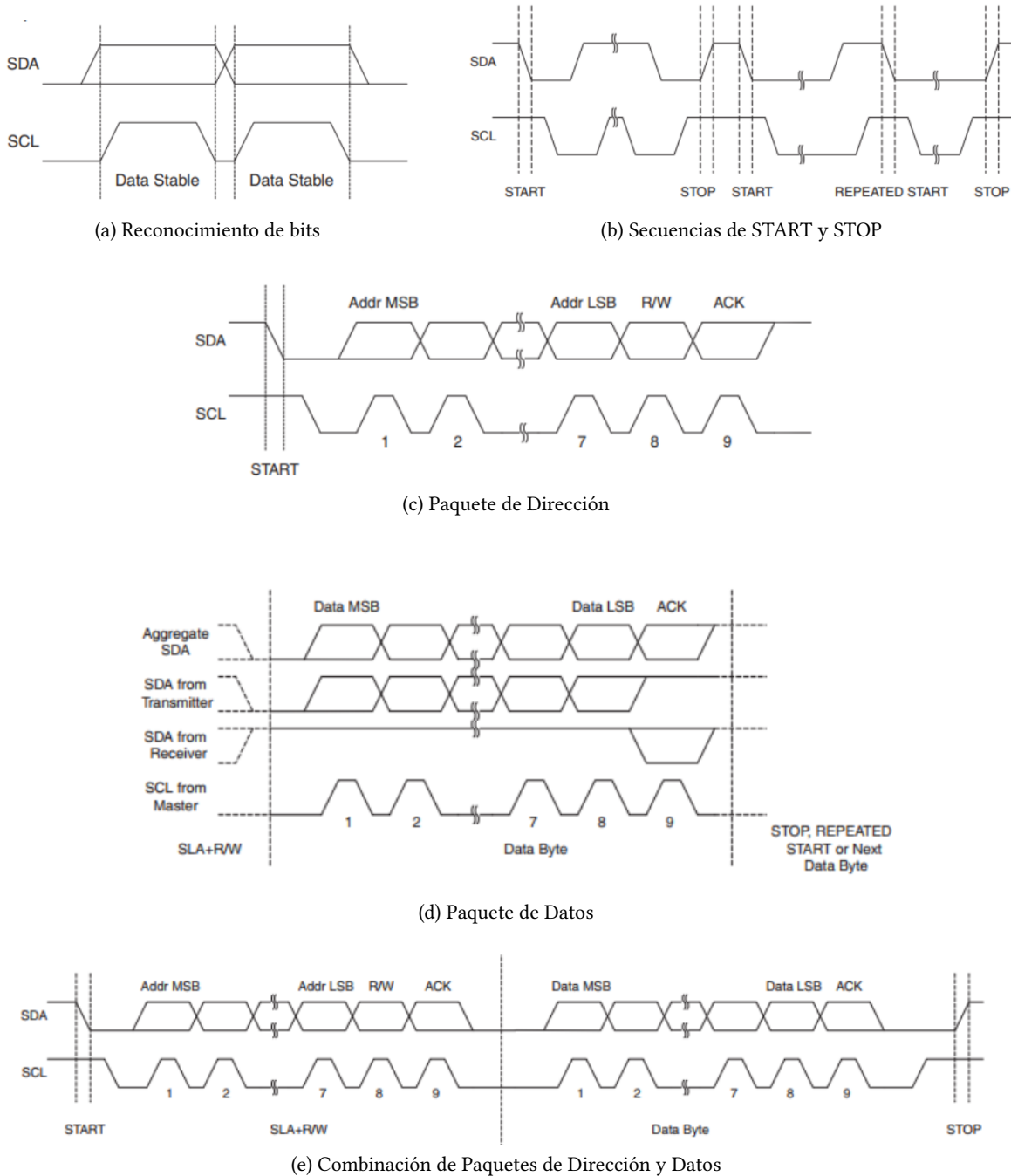
- 7 bits más significativos para la dirección del Esclavo con el cual el Maestro desea comunicarse.
- 1 bit para definir la operación que desea realizar el Maestro. Puede ser Lectura (R) o Escritura ($\overline{W}$). Este bit debe ser “1” para el caso de lectura y “0” para escritura.
- 1 bit de reconocimiento (ACK). Este bit es generado por el Esclavo cuando reconoce la dirección enviada por el Maestro. El esclavo debe poner a “0” la línea SDA si reconoce la dirección.

Paquete de Datos (figura A.5d): Todos los paquetes de datos transmitidos poseen una longitud de 9 bits y están constituidos por:

- 8 bits para datos.
- 1 bit de reconocimiento (ACK). Este bit es generado por el receptor de los datos. El reconocimiento de la recepción se efectúa poniendo en “0” la línea SDA durante el noveno ciclo de reloj en línea SCL. Si el receptor deja la línea SDA en “1”, se señala un NACK. Esta señalización permite al receptor avisar que ha recibido el último byte, o por alguna razón no puede recibir más bytes.

Combinación de Paquetes de Dirección y Datos (figura A.5e): Una transmisión consiste básicamente en una secuencia de START, una dirección más bit R o W ($SLA+R/W$)¹, uno o más paquetes de datos y una secuencia de STOP. Pueden existir varios bytes de datos entre $SLA+R/W$ y STOP, dependiendo del protocolo implementado por el programa del usuario.

¹SLA significa dirección de esclavo (Slave Address)

Figura A.5: Trama I²C en detalle

A.3. SPI

SPI (Serial Peripheral Interface) [37] es un bus de comunicación serie síncrona bidireccional full-duplex desarrollado en el año 1980 por la empresa Motorola. En el bus SPI hay un dispositivo Maestro y uno o más Esclavos. El Maestro es el único encargado de iniciar y finalizar la comunicación en cualquiera de las direcciones y también genera la señal de reloj de sincronización. Este bus se encuentra compuesto por las siguientes cuatro líneas más la línea común de GND:

MOSI (Master Out Slave In): Esta línea lleva los datos desde el Maestro a los Esclavos.

MISO (Master In Slave Out): Lleva los datos desde los Esclavos al Maestro.

CLK (Clock): Esta línea lleva los pulsos de reloj desde el Maestro a los Esclavos.

$\overline{SS}$ (Slave Select): Es la línea mediante la cual el Maestro selecciona al Esclavo con el cual quiere comunicarse. Si hay varios esclavos, pueden existir varias líneas de selección de esclavos.

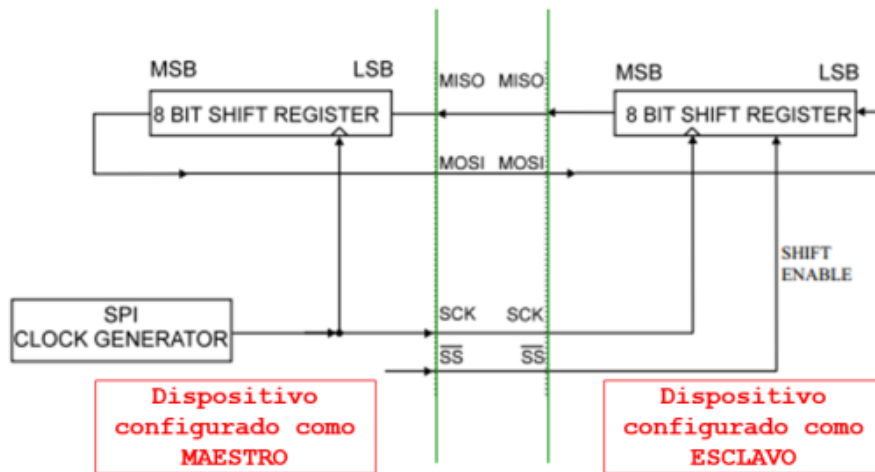


Figura A.6: Interconexión interfaz SPI

Operación: El sistema de comunicación SPI de la figura A.6 posee dos registros de desplazamiento (de 8 bits c/u) y un generador de reloj maestro. El maestro SPI inicia el ciclo de comunicación cuando pone en bajo el pín $\overline{SS}$. El maestro y el esclavo preparan los datos para ser enviados en sus respectivos registros de desplazamiento, y el maestro genera los pulsos de reloj requeridos en la línea CLK para intercambiar los datos. Estos últimos van de Maestro a Esclavo a través de la línea MOSI y de Esclavo a Maestro a través de MISO. Después de cada paquete de datos (8 bits), el maestro sincronizará al esclavo poniendo en alto la línea de selección $\overline{SS}$.

Formato Intel (Archivo .hex)

El formato de archivo de objeto hexadecimal de Intel, el formato hexadecimal de Intel o Intellect Hex es un formato de archivo que transmite información binaria en forma de texto ASCII [38]. Se usa comúnmente para programar microcontroladores, EPROM y otros tipos de dispositivos lógicos programables y emuladores de hardware. En una aplicación típica, un compilador o ensamblador convierte el código fuente de un programa (como en lenguaje C o ensamblador) en código de máquina y lo envía a un archivo .hex. Algunos también lo usan como un formato contenedor que contiene paquetes de datos de transmisión. Las extensiones de archivo comunes utilizadas para los archivos resultantes son .hex o .H86. Luego, un programador lee el archivo .hex para escribir el código máquina en una memoria o se transfiere al sistema de destino para su carga y ejecución.

<pre> :10010000214601360121470136007EFE09D2190140 :100110002146017EB7C20001FF5F16002148011988 :10012000194E79234623965778239EDA3F01B2CAA7 :100130003F0156702B5E712B722B732146013421C7 :00000001FF </pre>	 Código de inicio Longitud Dirección Tipo de registro Datos Checksum
--	--

Figura B.1: ejemplo de archivo .hex

Código de inicio El código de inicio utilizado es dos puntos ":"

Longitud del registro Dos dígitos hexadecimales con la cantidad de bytes del campo de datos.

Dirección Cuatro dígitos hexadecimales, con la dirección de inicio de los datos.

Tipo de registro Dos dígitos hexadecimales, de 00 a 05, definen el tipo del campo de datos.

Datos Duplas de dígitos hexadecimales que contienen los datos

Checksum Dos dígitos hexadecimales con el complemento a dos de la suma de todos los campos anteriores, salvo el ":"

Hay seis tipos de registros, los mismos se describen a continuación:

- 00 Datos:** La *Longitud de registro* especifica el número de bytes de datos en el registro. El ejemplo tiene **0B** (once) bytes de datos. La dirección inicial de 16 bits para los datos (en el ejemplo, en direcciones que comienzan en **0010**) y los datos (**61**, **64**, **64**, **72**, **65**, **73**, **73**, **20**, **67**, **61**, **70**).

Ejemplo: : **0B** **0010** **00** **6164647265737320676170** **A7**

- 01 Fin de archivo:** Debe ocurrir exactamente una vez por archivo en el último registro del archivo. La Longitud siempre es **00**, el campo de dirección suele ser **0000** y se omite el campo de datos.

Ejemplo: : **00** **0000** **01** **FF**

- 02 Dirección Extendida de Segmento:** El Tipo de Registro siempre es **02**, el campo de dirección (normalmente **0000**) se ignora y el campo de datos contiene una dirección base de segmento de 16 bits. Esto se multiplica por 16 y se suma a cada dirección de registro de datos subsiguiente para formar la dirección inicial de los datos. Esto permite direccionar hasta un mebibyte (1048576 bytes) de espacio de direcciones.

Ejemplo: : **02** **0000** **02** **1200** **EA**

- 03 Dirección de Comienzo de Segmento:** Para procesadores 80x86, especifica la dirección de ejecución inicial. El Tipo de Registro siempre es **04**, el campo de dirección es **0000** y los dos primeros bytes de datos son el valor CS, los dos últimos son el valor IP.

Ejemplo: : **04** **0000** **03** **00003800** **C1**

- 04 Dirección Lineal Extendida:** Permite direccionamiento de 32 bits (hasta 4 GiB). La Longitud de bytes siempre es **02** y el campo de dirección se ignora (normalmente **0000**). Los dos bytes de datos especifican los 16 bits superiores de la dirección absoluta de 32 bits para todos los registros de tipo **00** posteriores; estos bits de dirección superiores se aplican hasta el siguiente registro **04**. La dirección absoluta para un registro de tipo **00** se forma combinando los 16 bits de dirección superiores del registro **04** más reciente con los 16 bits de dirección inferiores del registro **00**. Si un registro de tipo **00** no está precedido por ningún registro de tipo **04**, sus 16 bits de dirección superiores se establecen de forma predeterminada en **0000**.

Ejemplo: : **02** **0000** **04** **FFFF** **FC**

- 05 Comienzo de Dirección Lineal:** La Longitud de bytes siempre es **04**, el campo de dirección es **0000**. Los cuatro bytes de datos representan un valor de dirección de 32 bits. En el caso de las CPU que lo soportan, esta dirección de 32 bits es la dirección de ejecución inicial).

Ejemplo: : **04** **0000** **05** **000000CD** **2A**

Programas escritos para el sistema mínimo

En este anexo se encuentra el código fuente de los programas escritos para el sistema mínimo.

C.1. Programa 1: Parpadeo LED Q

Listado C.1: Programa 1

```
1 ; CPU Type:
2     CPU 1802
3
4 ; Register Definitions:
5 R0     EQU 0
6 R1     EQU 1
7 R2     EQU 2
8 R3     EQU 3
9 R4     EQU 4
10 R5     EQU 5
11 R6     EQU 6
12 R7     EQU 7
13 R8     EQU 8
14 R9     EQU 9
15 RA     EQU 10
16 RB     EQU 11
17 RC     EQU 12
18 RD     EQU 13
19 RE     EQU 14
20 RF     EQU 15
21
22 ; Start code segment
23     ORG 00000H
24 INICIO
25     REQ          ; reset Q
26 START
27     LDI 010H     ; carga el valor inmediato 10 hex en D
28     PHI R1       ; carga el valor de D a R1.1
29 LOOP
30     DEC R1       ; decrementa R1.1
31     GHI R1       ; carga R1.1 en D
32     BNZ LOOP     ; salta si D distinto de 0
33     BQ INICIO    ; salta si Q == 1 (para que Q conmute)
34     SEQ         ; set Q
35     BR START     ; salto incondicional
36     END
```

C.2. Programa 2: LED Q se enciende al presionar pulsador

Listado C.2: Programa 2

```
1 ; CPU Type:
2     CPU 1802
3
4 ; Register Definitions:
5 R0     EQU 0
6 R1     EQU 1
7 R2     EQU 2
8 R3     EQU 3
9 R4     EQU 4
10 R5     EQU 5
11 R6     EQU 6
12 R7     EQU 7
13 R8     EQU 8
14 R9     EQU 9
15 RA     EQU 10
16 RB     EQU 11
17 RC     EQU 12
18 RD     EQU 13
19 RE     EQU 14
20 RF     EQU 15
21
22 ; Start code segment
23     ORG 00000H
24 START
25     REQ                ; reset Q
26 ENCENDIDO
27     BN4 START          ; salta si EF4 esta en estado bajo
28     SEQ                ; set Q
29     BR ENCENDIDO       ; salto incondicional
30     END
```

C.3. Programa 3: LED Q parpadea al presionar pulsador

Listado C.3: Programa 3

```

1  ; CPU Type:
2      CPU 1802
3
4  ; Register Definitions:
5  R0      EQU 0
6  R1      EQU 1
7  R2      EQU 2
8  R3      EQU 3
9  R4      EQU 4
10 R5      EQU 5
11 R6      EQU 6
12 R7      EQU 7
13 R8      EQU 8
14 R9      EQU 9
15 RA      EQU 10
16 RB      EQU 11
17 RC      EQU 12
18 RD      EQU 13
19 RE      EQU 14
20 RF      EQU 15
21
22 ; Start code segment
23 ORG 00000H
24 START
25     REQ          ; reset Q
26 INICIO
27     LDI 10H      ; carga 10 en D
28     PHI R1       ; carga el valor de D en R1.1
29 LOOP
30     DEC R1       ; decremanta R1
31     GHI R1       ; carga el valor de R1.1 en D
32     BNZ LOOP    ; salta si D = 0
33     BQ  START   ; salta si Q = 1
34     SEQ          ; set Q
35 ENCENDIDO
36     BN4 START   ; salta si EF4 esta en estado bajo
37     SEQ          ; set Q
38     BR  INICIO  ; salto incondicional
39     END

```

C.4. Programa 4: Transmisión serial de caracteres

Listado C.4: Programa 4

```

1  ; CPU Type:
2      CPU 1802
3
4  ; Register Definitions:
5  R0      EQU 0
6  R1      EQU 1
7  R2      EQU 2
8  R3      EQU 3
9  R4      EQU 4
10 R5      EQU 5
11 R6      EQU 6
12 R7      EQU 7
13 R8      EQU 8
14 R9      EQU 9
15 RA      EQU 10
16 RB      EQU 11
17 RC      EQU 12
18 RD      EQU 13
19 RE      EQU 14
20 RF      EQU 15
21 ; definicion de constantes
22 TX      EQU 8
23 DLY1     EQU 9          ; DELAY1 es un delay corto
24 DLY2     EQU 10         ; DELAY2 es un delay largo
25 CONT1BT  EQU 123D       ; contador para el delay de 1 bit-time
26 CONTLONG EQU 050H       ; contador para el delay largo (delay 2)
27
28 ; Start code segment
29
30 ; antes de empezar suponemos que el dato a enviar esta cargado en el registro R2
   .1 (porque R0 es el contador de programa)
31     ORG 00000H
32 INICIO          ; esta parte son solo definiciones
33     LDI 00H
34     PHI TX
35     LDI 40H
36     PLO TX      ; se cargo 0040H en TX = R8, que es la SUB de transmision
37     LDI 00H
38     PHI DLY1
39     LDI 60H
40     PLO DLY1    ; se cargo 0060H en DLY1 = R9, que es la SUB de DELAY1 (para
                   la SUB de transmision)
41     LDI 00H
42     PHI DLY2
43     LDI 70H
44     PLO DLY2    ; se cargo 0070H en DLY2 = RA, que es la SUB de DELAY2 (para
                   el programa principal)

```

```

45 ;aca empieza la rutina principal
46     SEQ                ; empezamos ponindo la linea estado inactivo
47 LOOP  LDI 65D          ; caracter ascii A (mayuscula)
48     PLO R4            ; cargo el valor de D a R4 (para tener guardado el
                        ; ultimo caracter enviado)
49     PHI R2            ; cargo el valor de D a R2 (para la subrutina)
50     LDI 07H           ; cargo 7 en D
51     PLO R5            ; cargo D en R5.0 (para hacer 7 repeticiones en la
                        ; rutina SEND)
52 SEND  SEP TX          ; llamo a la SUB que transmite lo que esta en R2
53     SEP DLY2          ; llamo a la SUB DELAY2
54     INC R4            ; incremento R4 (paso al siguiente caracter)
55     GLO R4            ; carga R4.0 en D
56     PHI R2            ; carga D en R2.1
57     DEC R5            ; decremento R5
58     GLO R5            ; carga la parte baja de R3 en D
59     BNZ SEND          ; salta si D != 0 (va a saltar 7 veces, enviando ABCDEFG
                        ; )
60     ; envio CRLF para hacer el salto de linea
61     LDI 13D           ; caracter ascii CR (carriage return)
62     PHI R2            ; cargo el valor de D a R2 (para la subrutina)
63     SEP TX            ; llamo a la SUB que transmite lo que esta en R2
64     SEP DLY2          ; llamo a la SUB DELAY2
65     LDI 10D           ; caracter ascii (line feed)
66     PHI R2            ; cargo el valor de D a R2 (para la subrutina)
67     SEP TX            ; llamo a la SUB que transmite lo que esta en R2
68     SEP DLY2          ; llamo a la SUB DELAY2
69     BR LOOP           ; salto incondicional
70
71 ;-----subrutina de trasmision de datos-----
72     ORG 0003FH
73 BACK1 SEP R0          ; retorna a la subrutina principal
74 TRANSM
75     SEQ                ; Q = 1 (linea en estado inactivo)
76     REQ                ; Q = 0 (bit de start)
77     SEP DLY1          ; llamar subrutina de demora de 1 bit-time
78     LDI 08H           ; carga el valor inmediato en D
79     PLO R3            ; carga el valor de D en R3.0 (este es el contador para
                        ; iterar cada bit del dato en R2)
80
                        ; tiene que ser la parte baja de R3, porque DEC R3
                        ; decrementa el registro entero de 16 bits
81 SHIFT GHI R2          ; carga el valor de R2.1 en D (aca se supone que esta el
                        ; byte que queremos transmitir)
82     SHR                ; desplaza D a la deracha, cargando el LSB en el bit DF
83     PHI R2            ; carga el valor de D en R2.1
84     BNF ESCERO        ; salta si DF = 0
85     SEQ                ; salida Q igual a 1
86     SKP                ; salta la siguiente instruccion de manera incondicional
87 ESCERO REQ            ; salida Q igual a 0
88     SEP DLY1          ; llamar subrutina de demora de 1 bit-time
89     DEC R3            ; decremento el contador

```

```

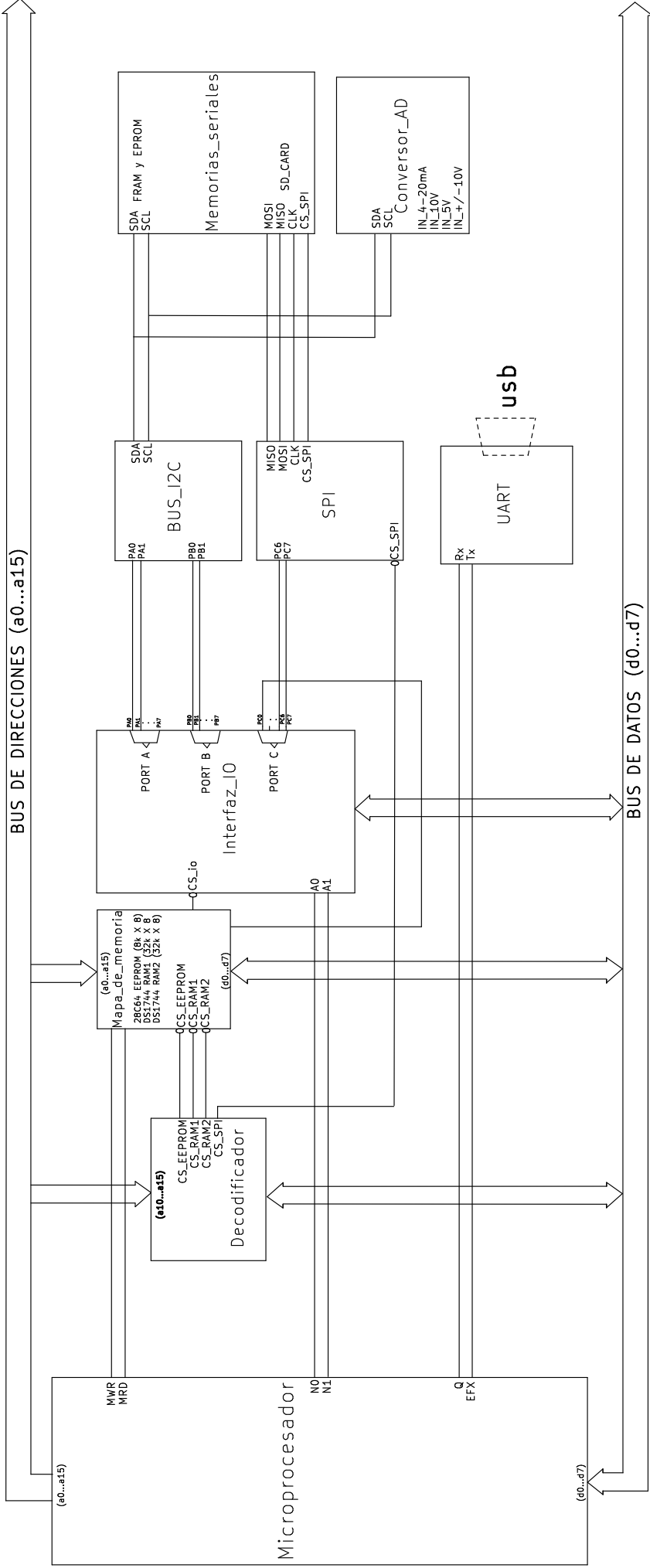
90      GLO R3      ; carga la parte baja de R3 en D
91      BNZ SHIFT  ; salta si D != 0
92      SEQ        ; Q = 1 (bit de stop)
93      SEP DLY1   ; llamar subrutina de demora de 1 bit-time dos veces
94      SEP DLY1
95      BR BACK1   ; salto incondicional
96
97 ;-----subrutina de ratardo de tranmision-----
98      ORG 0005FH
99 BACK2 SEP TX      ; retorna a subrutina de transmision
100 DELAY1
101      LDI CONT1BT  ; carga el valor inmediato en D
102      PLO R1       ; carga el valor de D en R1.1
103 TIMER1
104      DEC R1       ; decrementa R1
105      GLO R1       ; carga la parte alta de R1 en D
106      BNZ TIMER1  ; salta si D != 0
107      BR BACK2    ; salto incondicional
108
109 ;-----subrutina de retardo del programa principal-----
110      ORG 0006FH
111 BACK3 SEP R0      ; retorna a la rutina principal
112 DELAY2
113      LDI CONTLONG ; carga el valor inmediato en D
114      PHI R1       ; carga el valor de D en R1.1
115 TIMER2
116      DEC R1       ; decrementa R1
117      GHI R1       ; carga la parte alta de R1 en D
118      BNZ TIMER2  ; salta si D != 0
119      BR BACK3    ; salto incondicional
120      END

```

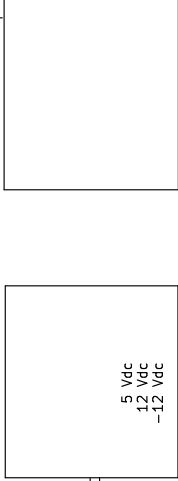

Esquemas circuitales y circuitos impresos

En este apéndice se incluyen los esquemáticos presentados dentro del cuerpo del informe, pero en el tamaño de hoja original.

Microcomputadora basada en el microprocesador CDP1802



Fuente_de_alimentacion Alimentacion_compuertas



Tutor: Ing. Korpys Ricardo

Alumnos: Alegre Chamorro Felipe, Boher Atilio, Vera Uriel

Sheet: /

File: SISTEMA_FINAL_esquemático_segmentado.kicad_sch

Title: Microcomputadora basada en el microprocesador_CDP1802

Size: A4

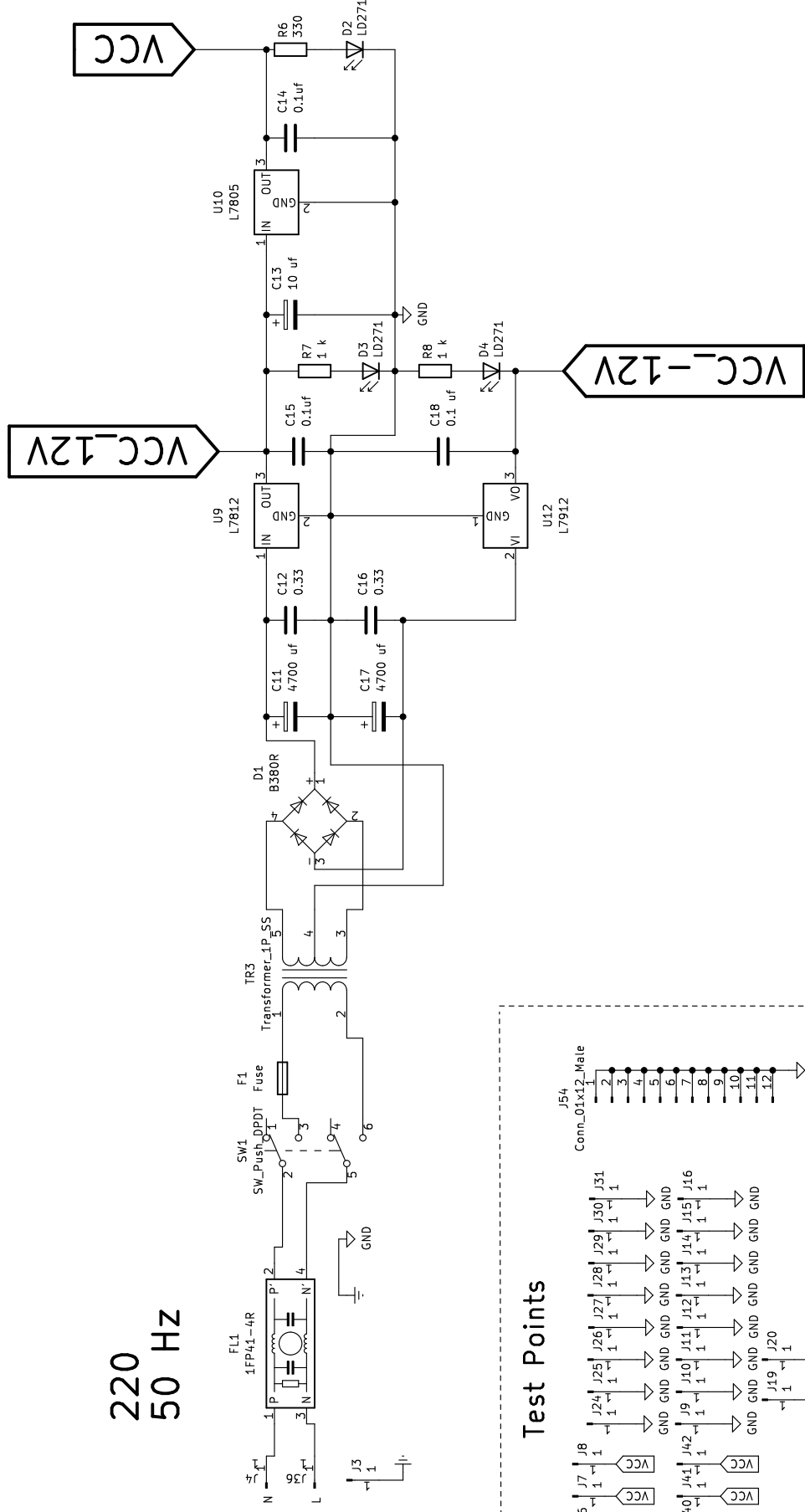
Date: 22/11/2022

KiCad E.D.A. kicad (6.0.6)

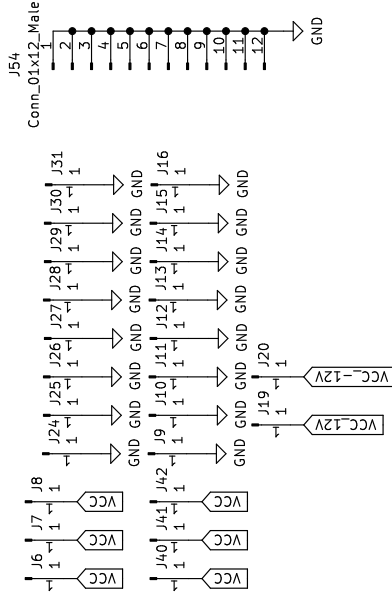
Rev: Id: 1/12

Fuente de alimentación

220
50 Hz



Test Points



- Las tensiones de continua disponibles son de 5V, 12V y -12V
- Los reguladores de tensión (78XX y 79XX) poseen un encapsulado TO220 y deben ir con su correspondiente disipador
- El filtro de línea deberá ir engrapado a la placa
- El puente de diodos es de 1A

Tutor: Ing. Korpys Ricardo
Alumnos: Alegre Chamorro Felipe, Boher Atlio, Vera Uriel
Sheet: /Fuente_de_alimentacion/
File: Fuente_de_alimentacion.kicad_sch
Title: Fuente_de_alimentacion
Size: A4 | Date: 22/11/2022
KiCad E.D.A. kicad (6.0.6)

Rev:

Id: 2/12

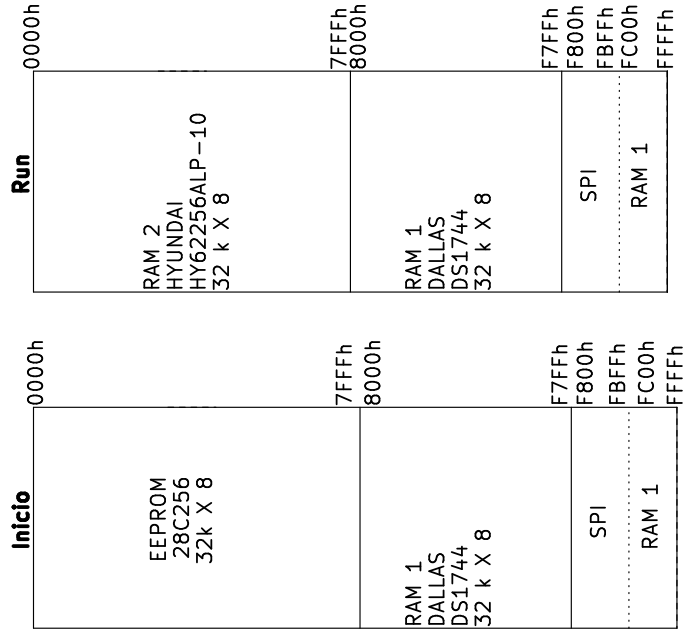
Mapa de memoria

BUS DE DATOS (D0..D7)

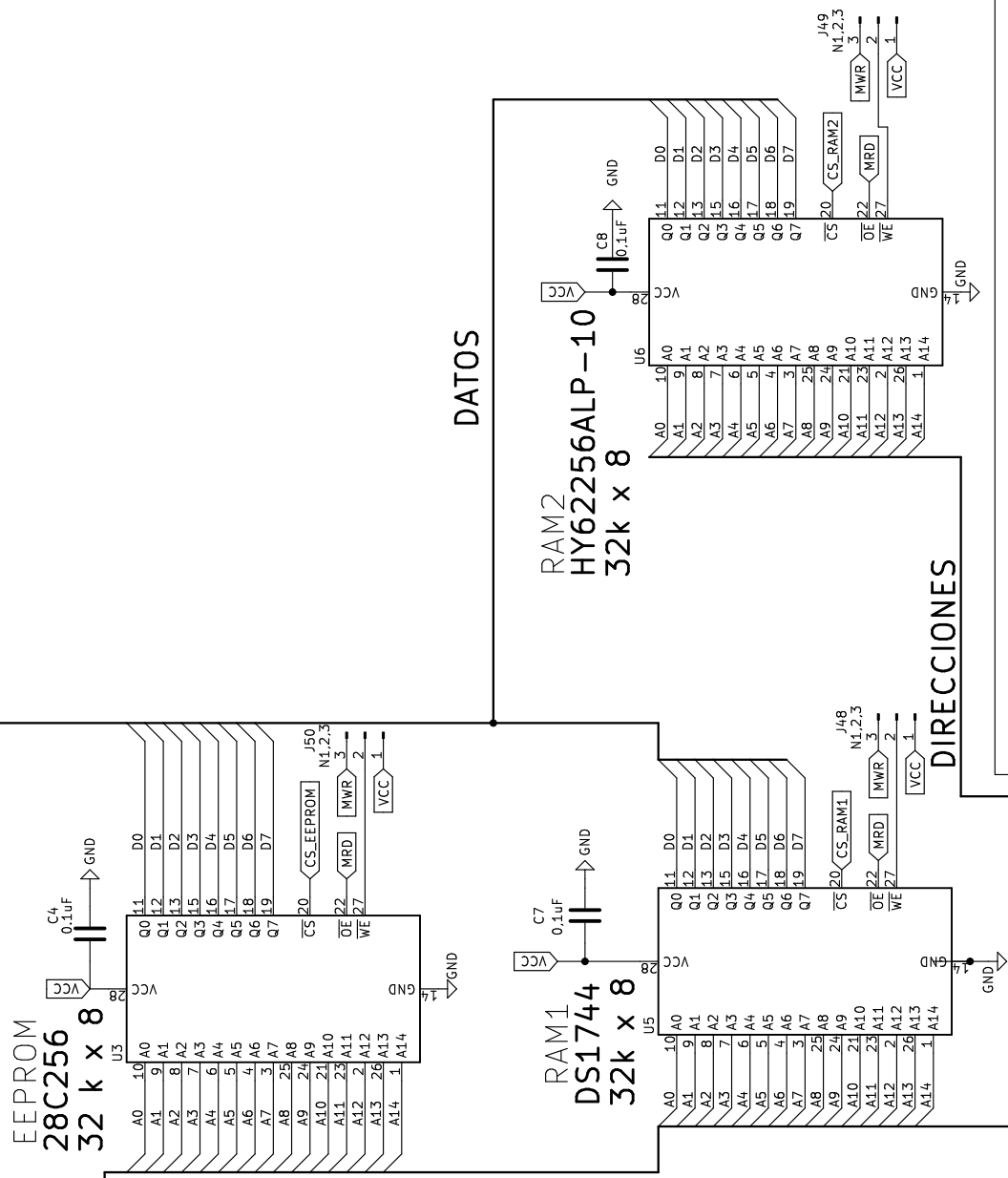
BUS DE DATOS

BUS DE DIRECCIONES (A0...A15)

BUS DE DIRECCIONES



- El modo inicio corresponde al uso de la EEPROM donde se almacena el programa a utilizar. En esta secuencia inicial, se copia el programa en la parte alta de la RAM1
- El modo RUN es el modo normal de funcionamiento, que sucede luego de la secuencia de inicio.
- y se puede utilizar la totalidad del mapa de memoria
- El decodificador se utiliza solamente para el SPI, para las memorias A15 es suficiente
- El memory Write (MW) de cada memoria puede ponerse microprocesador. Esto es para deshabilitar la escritura de operación al momento de la microcomputadora
- En lugar de la memoria EEPROM 28C64 se puede conectar los mismos comparten encapsulado



es suficiente

- El memory write (MW) de cada memoria puede ponerse a estado alto o conectarse al MW del microprocesador. Esto es para deshabilitar la escritura de las memorias al momento de la operación al momento de la microcomputadora
- En lugar de la memoria EEPROM 28C64 se puede conectar una memoria EEPROM 28C64. Los mismos comparten encapsulado

Tutor: Ing. Korpys Ricardo

Alumno: Alegre Chamorro Felipe, Boher Atilio, Vera Uriel

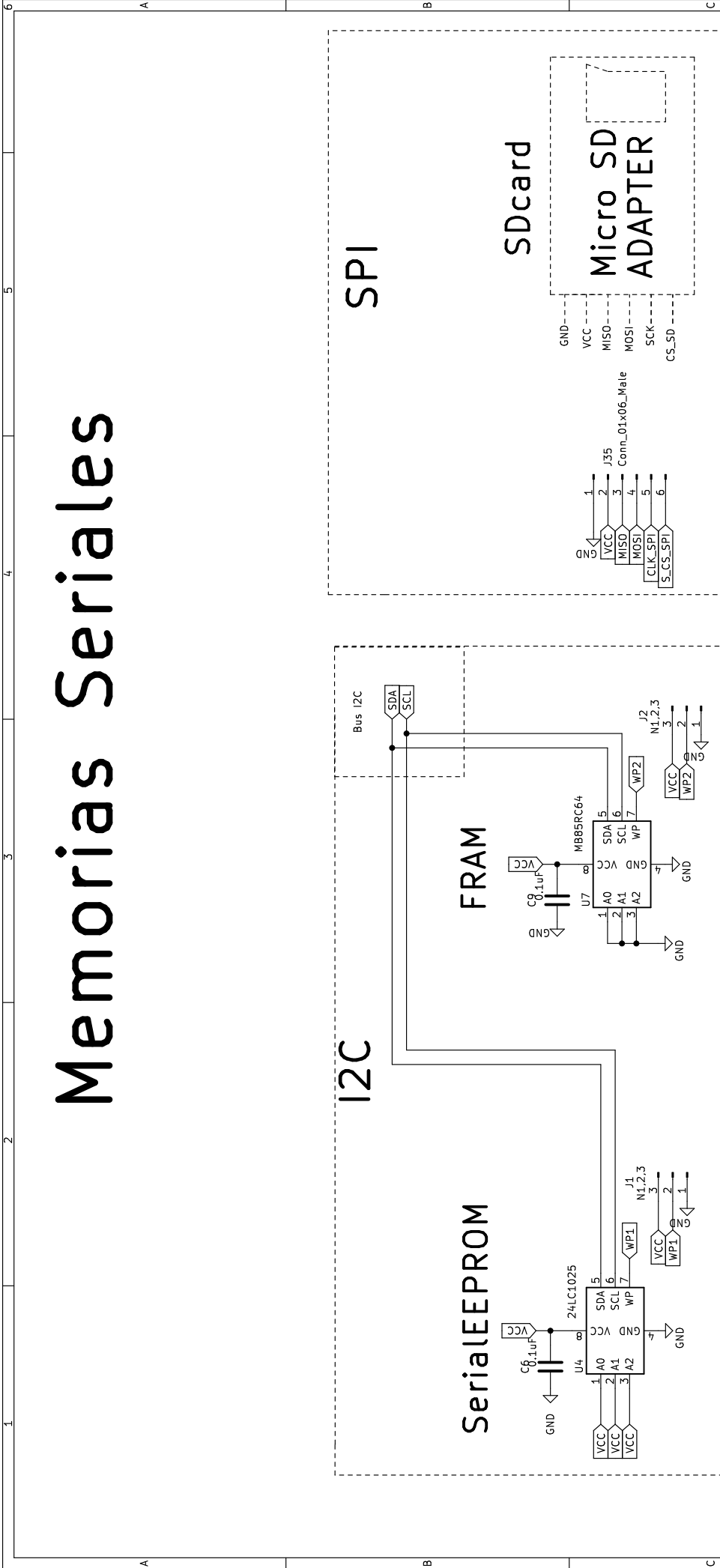
Sheet: / Mapa de memoria /

File: Mapa_de_memoria_y_decodificador.kicad_sch

Title: Mapa_de_memoria

Size: A4	Date: 22/11/2022
KiCad E.D.A. kicad (6.0.6)	

1	2	3	4	5
---	---	---	---	---



Tutor: Ing. Korpys Ricardo Alumnos: Alegre Chamorro Felipe, Boher Attilio, Vera Uriel	
Sheet: /Memorias_seriales/ File: Memorias_seriales_sch.kicad_sch	
Title: Memorias_seriales	D

Tutor: Ing. Korpys Ricardo
Alumnos: Alegre Chamorro Felipe, Boher Attilio, Vera Uriel

Sheet: /Memorias_seriales/ File: Memorias_seriales_sch.kicad_sch	D
---	---

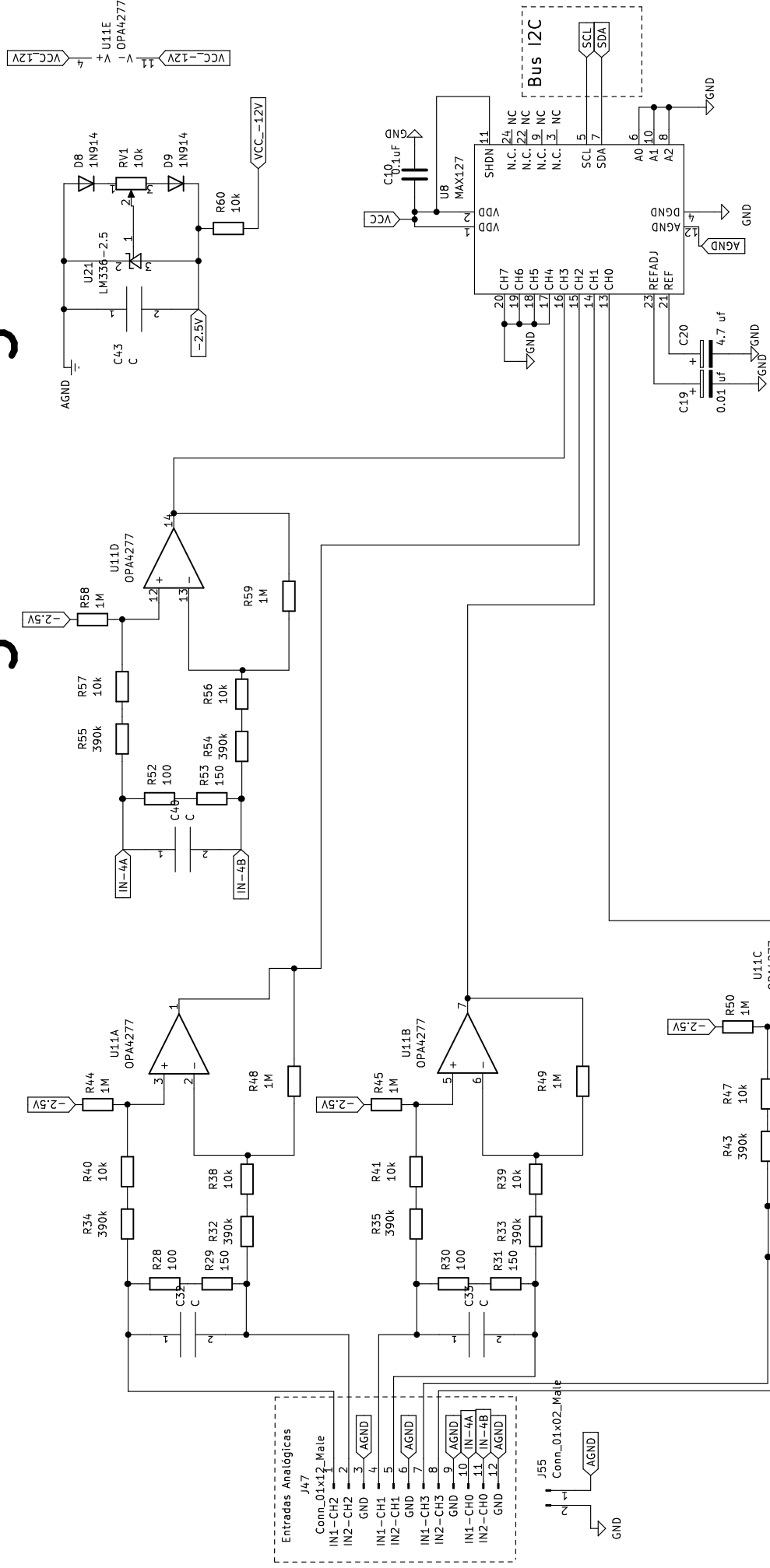
Title: Memorias_seriales

Size: A4	Date: 22/11/2022	Rev:
----------	------------------	------

KiCad E.D.A. kicad (6.0.6)	Id: 4/12
----------------------------	----------

[illegible]

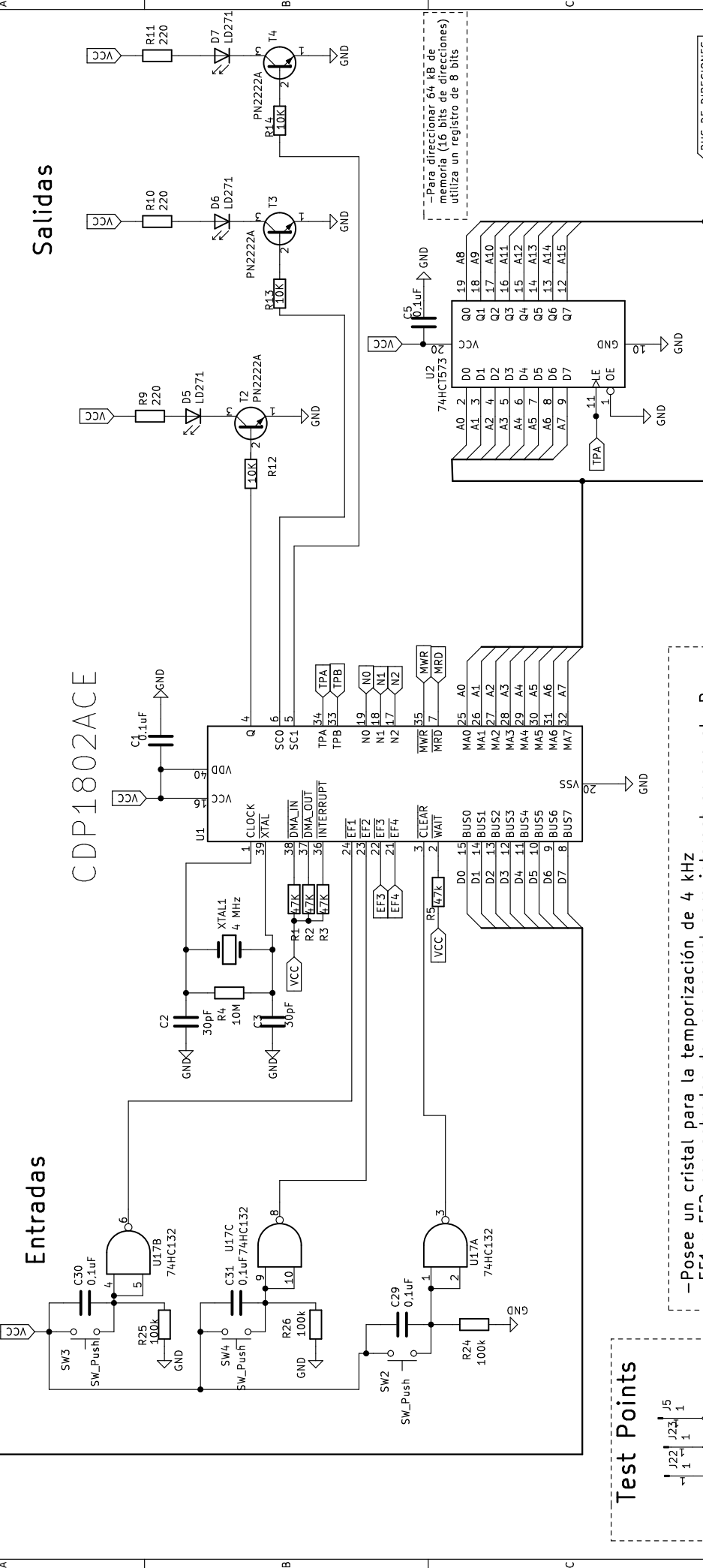
Conversor Analógico-Digital



Tutor: Ing. Korpys Ricardo
Alumnos: Alegre Chamorro Felipe, Boher Attilio, Vera Uriel
 Sheet: /Conversor_AD/
 File: Conversor_AD.kicad_sch
Title: Conversor Analógico-Digital
 Size: A4 | Date: 22/11/2022
 KiCad E.D.A. kicad (6.0.6) **Rev:**
 Id: 5/12

-Todas las entradas al conversor AD son conversores de 4 a 20 mA
 -Se utiliza un amplificador de precisión del fabricante Analog Devices
 -Para la implementación de la placa, en la parte del conversor analógico/digital se coloca un jumper para poder conectar las masas analógicas y digitales.

BUS DE JAIROS



se utiliza para supervisar el funcionamiento de los programas instalados.

- Posee un cristal para la temporización de 4 kHz
- EF1 y EF2 son entradas de uso general para interactuar con el uP
- Las salidas SC1 y SC2 son para control de debugging y la salida Q se utiliza para supervisar el funcionamiento de los programas instalados. Además el pin Q se utiliza para la comunicación uart

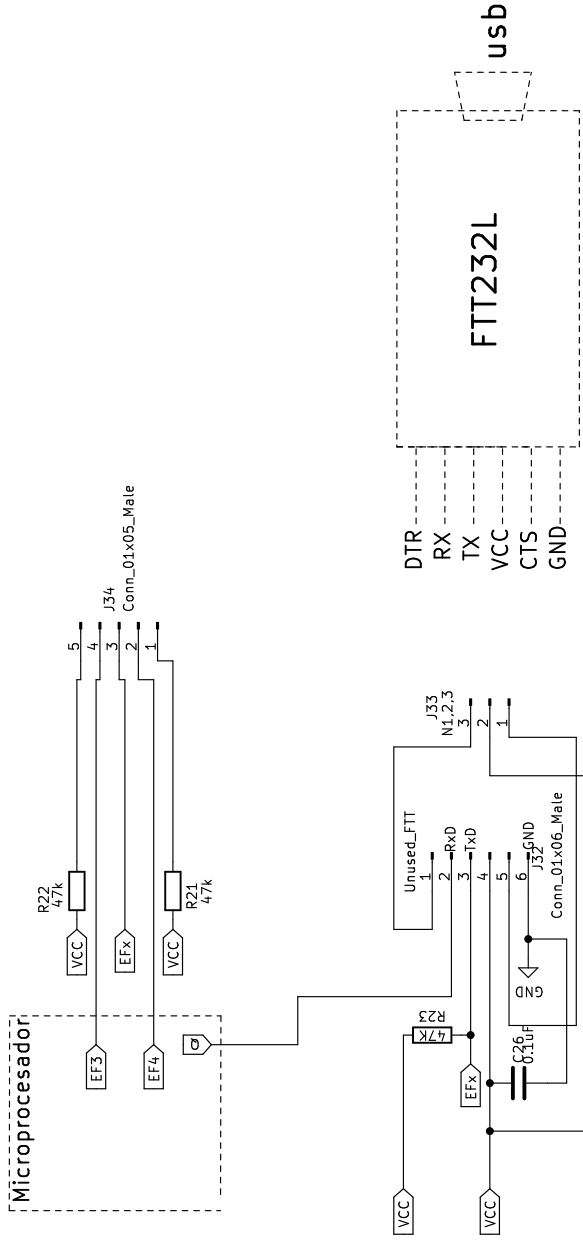
Test Points

Tutor: Ing. Korpys Ricardo	
Alumnos: Alegre Chamorro Felipe, Boher Attilio, Vera Uriel	
Sheet: /Microprocesador/	
File: Microprocesador_sch.kicad_sch	
Title: Conexiones del Microprocesador	
Size: A4	Date: 22/11/2022
KiCad E.D.A.	kicad (6.0.6)
Rev:	
Id: 6/12	



- El modo RUN es el modo normal de funcionamiento, que sucede luego de la secuencia de inicio, y se puede utilizar la totalidad del mapa de memoria
- El decodificador se utiliza solamente para el SPI, para las memorias RAM y EEPROM A15 es suficiente.
- El SPI utiliza desde la dirección F800h a la FBFFh debido a que es la sección del mapa de memoria que activa el decodificador
- Todas las memorias tienen un encapsulado DIP-28 y el decodificador posee encapsulado DIP-16

Comunicación UART



- La comunicaciónUART se realiza mediante software.
- Se utiliza la placa conversora de uart a usb (FTT232L) para conectarse a una hipterminal (PC)
- Existen programas y sistemas operativos que utilizan Ef3 y Ef4 para su funcionamiento y es por ello debe poderse intercambiar según se desee

Tutor: Ing. Korpys Ricardo
Alumnos: Alegre Chamorro Felipe, Boher Attilio, Vera Uriel

Sheet: /UART/
File: UART.kicad.sch

Title: Interfaz_UART-USB

Size: A4 | Date: 22/11/2022
KiCad E.D.A. kicad (6.0.6)

Rev:
Id: 8/12

Comunicación SPI

The diagram illustrates the SPI communication setup for a Micro SD Storage expansion Board. The components and their connections are as follows:

- Microprocesador (U28A, 74HC00):** A 74HC00 NAND gate used for signal processing. It is connected to the SPI transfer signal (PC7) and the CS_SPI signal.
- Contador 8 bits (U25, 74HC161):** An 8-bit counter that generates the CS_SPI signal. It is connected to the SPI transfer signal (PC7) and the CS_SPI signal.
- Registro de desplazamiento serial-in-serial-out (U31, 74HC595):** A serial-in-serial-out shift register that receives data from the microcontroller and outputs it to the SD card. It is connected to the SPI transfer signal (PC7) and the CS_SPI signal.
- Registro de desplazamiento parallel-in-serial-out (U32, CD4014):** A parallel-in-serial-out shift register that receives data from the microcontroller and outputs it to the SD card. It is connected to the SPI transfer signal (PC7) and the CS_SPI signal.
- Dispositivo SPI (SD card adapter):** The SD card adapter is connected to the CS_SPI signal and the SPI transfer signal (PC7).
- Test Point:** A test point for the SPI signals, showing the CS_SPI, MOSI, and MISO signals.
- Micro SD Storage expansion Board:** The board is connected to the CS_SPI signal and the SPI transfer signal (PC7).

The circuit is designed to allow the microcontroller to communicate with the SD card via SPI. The CS_SPI signal is used to select the SD card, and the SPI transfer signal (PC7) is used to transfer data between the microcontroller and the SD card.

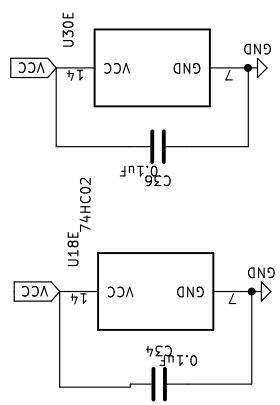
Tutor: Ing. Korpys Ricardo
Alumnos: Alegre Chamorro Felipe, Boher Atilio, Vera Uriel
 Sheet: /SPI/
 File: SPI.kicad_sch
Title: Comunicación SPI

Size: A4 Date: 22/11/2022
 KiCad E.D.A. kicad (6.0.6)

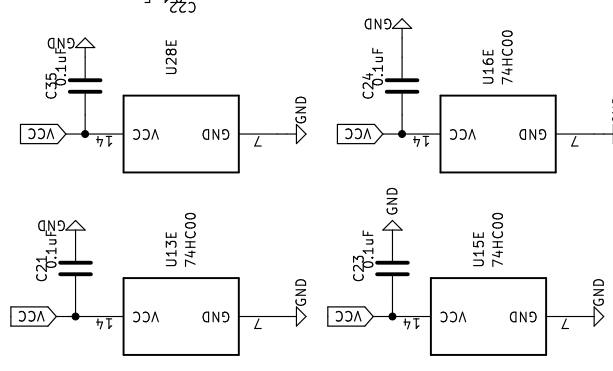
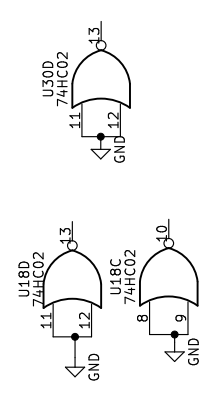
Rev:
Id: 9/12

Alimentación de circuitos integrados

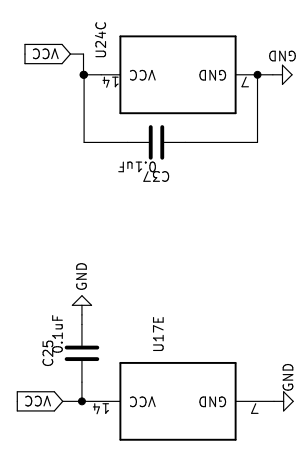
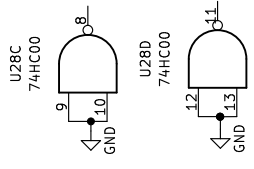
74HC02 74HC00 74HC132 74HC74 74HC73



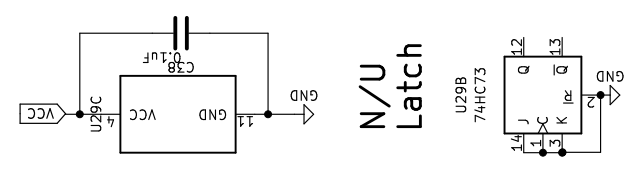
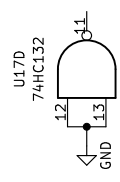
N/U
Compuertas



N/U
Compuertas



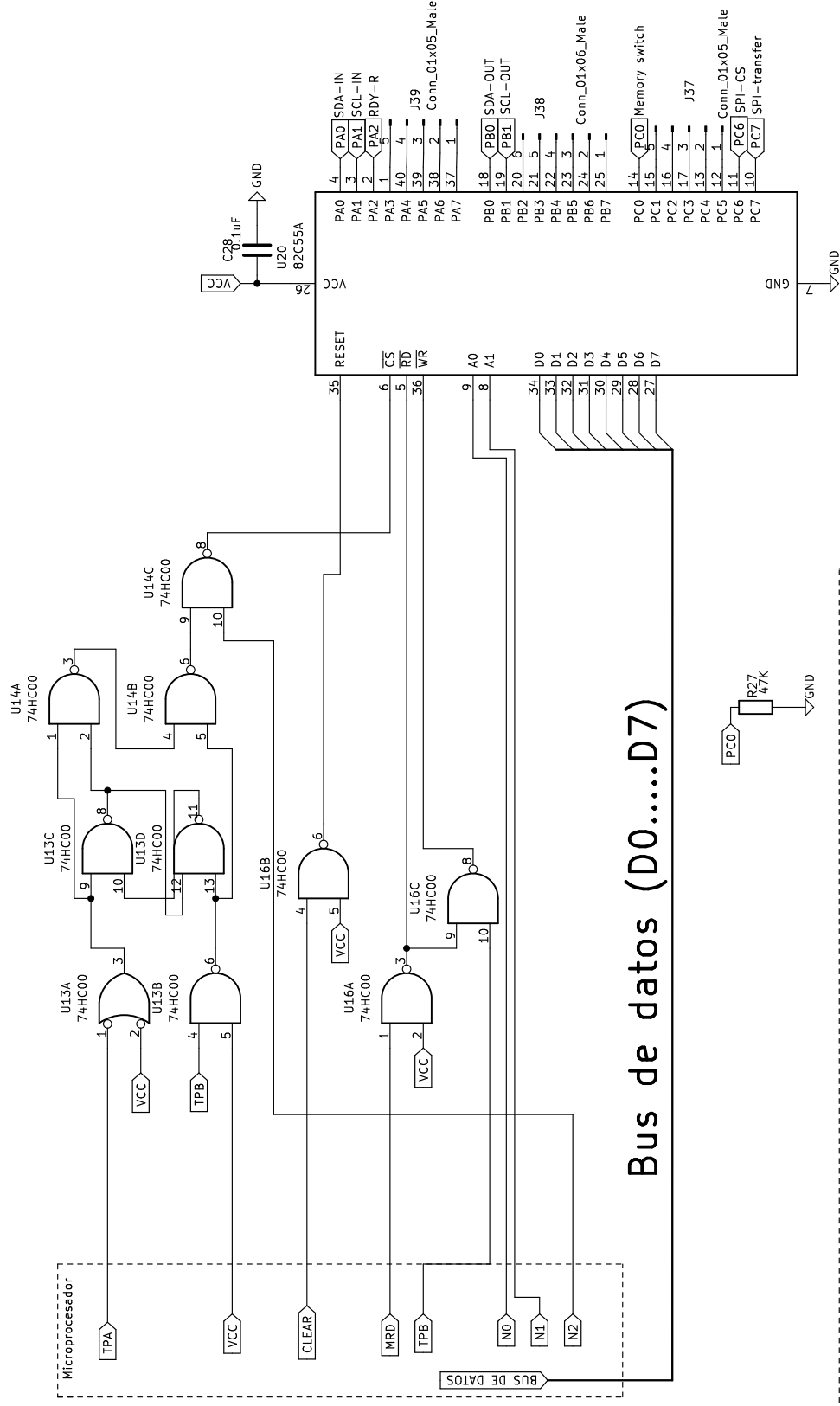
N/U
Compuertas



N/U
Latch

Las compuertas y circuitos no utilizados se conectan a masa debido a que los dispositivos son de la familia CMOS.

Interfaz I/O

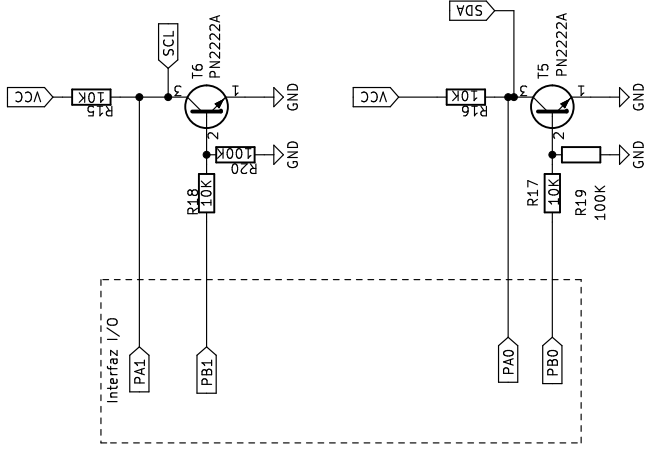


Bus de datos (D0.....D7)

La interfaz de entrada salida provee a la microcomputadora de entradas y salidas que puedan ser seteados según se requiera.
Posee 3 puertos A,B y C y son para diversas interacciones con el microprocesaodr.
Se configura mediante el bus de datos y los pines CS,RD, WR

Tutor: Ing. Korpys Ricardo
Alumnos: Alegre Chamorro Felipe, Boher Attilo, Vera Uriel
Sheet: /Interfaz_IO/
File: Interfaz_IO.kicad_sch
Title: Interfaz_Entrada-Salida
Size: A4 | Date: 22/11/2022
Kicad E.D.A. kicad (6.0.6) | Rev: Id: 11/12

Bus I2C



- La comunicacion I2C se realiza mediante una solución por hardware y software. Se utilizan las entradas de la interfaz de entrada salida y mediante una secuencia programada se ejecuta la comunicación.
- Los dispositivos que utilizan el bus I2C son el convertor A/D y las memorias seriales
- Cada uno de ellos tiene una direccion diferente configurada según corresponda.

Direcciones I2C	
Dispositivo	Dirección
ADCMAX127	0101 000 W/R
FRAM	
MB85RC64	1010 000 W/R
EEPROM	
24LC1025	1010 111 W/R

Tutor: Ing. Korpys Ricardo
Alumnos: **Alegre Chamorro Felipe, Boher Atilio, Vera Uriel**

Sheet: /BUS_I2C/
File: BUS_I2C_sch.kicad_sch

Title: Acondicionamiento_BUS-I2C

Size: A4 | Date: 22/11/2022

KiCad E.D.A. kicad (6.0.6)

Rev:

Id: 12/12

